Creda: Technical Specification

Creda: Technical Specification

Version: 0.1.0-draft **Status:** Draft **Audience:** Engineering Team **Last Updated:** 2026-04-28

---

1. Overview

Creda is a decentralized, peer-to-peer substrate for cross-institutional patient identity provenance and portable authorization. Each institution operates a Creda peer that participates in a vetted network governed by a legal coordinator (the admission control authority) but operates without runtime coordination. Identity events — assertions, links, contestations, attestations, amendments, tombstones, and deceased declarations — and authorization events — grants, revocations, and export receipts — form a directed acyclic graph that records who asserted what about a patient, what that patient authorized, when, and based on what prior assertions. The graph is replicated asynchronously across thousands of peers via gossip and anti-entropy, with FHIR R4 as the integration surface for institutional systems.

Identity continuity and portable authorization are **co-primary capabilities**. Identity continuity establishes who a patient is across institutions; portable authorization establishes what they have permitted — as a signed, scoped artifact that travels with data references and is verifiable at any point of use, not only at the moment a query is served. Authorization is enforced through a dual-control model: a source-side Export Gate validates authorization before data leaves a source system, and a relying-side Verifier independently confirms authorization, identity continuity, and provenance integrity at the point of use, locally and offline if needed.

Creda solves a problem that today's MPI and exchange ecosystem does not: cross-institutional patient identity with cryptographic provenance, persistent and revocable authorization, no central authority, and no vendor lock-in. Institutions retain sovereignty over their own assertions; patients can participate directly through signed self-attestations; and the entire history is tamper-evident by construction. The architecture builds on existing standards and components — UDAP, SMART on FHIR, HAPI FHIR, libp2p, SPIRE, TEFCA tokenization — rather than reinventing them, and is designed for incremental adoption alongside existing MPIs rather than as a forklift replacement.

2. Design Principles

These are the system-level architectural principles that govern every design decision in this spec. They apply across identity, authorization, security, networking, deployment, and governance — broader than the identity-specific tenets enumerated in Section 3.2. Where a proposed change would conflict with one of these principles, the principle wins.

Verification, not mediation. Creda verifies authorization, provenance, and state. It does not itself grant, deny, broker, or mediate access to clinical data. Access decisions remain with the responding institution under applicable law and policy. This boundary is structural, not aspirational — no Creda component has the capability to grant or deny access to health records. It confirms that an authorization is signed, scoped, unexpired, and unrevoked; the institution decides what to do with that confirmation. This principle is what keeps Creda on the right side of the line between "infrastructure that enables trust decisions" and "a system that makes access decisions for institutions."

Decentralization is structural, not aspirational. No runtime coordinator, no central data store, no privileged peer roles. Admission control exists — participants are vetted and admitted to a trust framework — but once admitted, operations are peer-to-peer. Any architectural decision that would reintroduce a central runtime authority is rejected, even when centralization would be operationally simpler. The model parallels DirectTrust, the non-profit trade association established in 2012 that operates the trust framework underlying Direct Secure Messaging in US healthcare: DirectTrust accredits and governs participants but does not mediate the messages they exchange.[1] Creda follows the same pattern — vetted participation, peer-to-peer operations.

Provenance is first-class, not metadata. Every fact about a patient's identity is traceable to who asserted it, when, and based on what prior assertions. Provenance is the structural backbone of the system, not an audit log bolted on afterward. Where existing systems return match scores without explanation, Creda returns the full evidentiary chain. Where existing systems treat audit as a separate compliance system, Creda's data structure *is* the audit trail.

Institutional sovereignty. Each institution owns and controls only what it created. No institution can modify, censor, or override another institution's assertions. The system has no privileged actor that can rewrite history on behalf of others. Trust between institutions is established through cryptographic signatures and reputation, not through deference to a centralized authority. This principle is what makes peer-to-peer participation politically viable — institutions adopt the network because participation does not require ceding control.

Standards over invention. Build on UDAP, SMART on FHIR, HAPI FHIR, HL7 Implementation Guide processes, libp2p, SPIFFE, TEFCA tokenization, and NIST post-quantum cryptography. Reinvent only what is genuinely Creda-specific: the healthcare event semantics, the consent model, the disambiguation algorithm, the integration glue. New code is reserved for the healthcare-domain layer; everything else is assembled from existing components. Appendix C documents the specific component for each technical decision.

Additive, not invasive. Creda extends the existing FHIR and US Health IT ecosystem rather than replacing it. Non-Creda consumers see standard FHIR resources with extensions they can ignore. Institutions retain their existing MPIs, EHRs, and FHIR endpoints; Creda joins as a complementary identity provenance layer. Adoption is incremental —

institutions can start as Observers (consuming through a participating QHIN), graduate to Light participants, and eventually become Full participants over multi-year arcs. The architecture is designed for a decade of coexistence with legacy infrastructure, not a forklift cutover.

Privacy by structure, not policy. Data minimization, demographic tokenization, and consent enforcement are architectural properties of the system, not administrative rules layered on top. Cleartext PHI never traverses the gossip network. Consent events are enforceable predicates evaluated at every responding peer, not policies in a separate database. The system makes it structurally difficult to leak PHI rather than relying on operators to follow procedures. When a privacy-preserving option requires more engineering than the alternative, Creda chooses the privacy-preserving option.

Honest about tradeoffs. Every significant design decision documents what it trades off. The right-to-be-forgotten / cryptographic integrity tension is named explicitly, not hidden. The bottom-up adoption strategy acknowledges the chicken-and-egg problem of network effects. The vetted-network model acknowledges the tension between openness and admission control. Section 13 enumerates unresolved questions rather than pretending the spec is complete. This honesty is not a presentation choice — it is how the spec earns the trust required for institutional adoption of the resulting system.

Designed for longevity. Patient identity provenance must remain verifiable for decades. The architecture treats long-term durability as a baseline requirement, not a future enhancement: post-quantum cryptography readiness from day one, algorithm-agile signatures with explicit migration paths, periodic salt rotation, extensible event type enums, versioned FHIR Implementation Guides, and a coordinator role designed to be transferable across organizational successors. The spec assumes Creda will outlive its founding institutions, founding coordinator, and founding cryptographic primitives.

3. Identity Model

3.1 Principles

The following principles are foundational to Creda's identity model. They are not guidelines — they are invariants that every design decision must satisfy.

An identity is a subgraph, not a record. There is no single golden record for a patient. A patient's identity is the totality of assertions made about them by institutions over time, represented as a directed acyclic subgraph. Each institution's view of a patient is a projection of the subgraph visible to them. The subgraph is the source of truth; any flat record is a derived artifact.

Provenance is first-class. Every fact about a patient's identity is traceable to the institution that asserted it, the method by which it was verified, and the prior assertions it was derived from. Provenance is not metadata attached after the fact — it is the structural backbone of the graph.

Institutions are sovereign over their own assertions. Each institution owns and controls only the nodes it created. No institution can modify, amend, tombstone, or overwrite another institution's assertions. An institution may contest a link between subgraphs, but this creates a new event — it does not alter the original. This preserves institutional autonomy and eliminates the need for a central authority to arbitrate ownership.

Multiple roots are the norm. A patient's subgraph may originate from multiple independent institutions that encountered the patient with no prior knowledge of each other. The subgraph is a forest of independent roots until link events connect them. The system must never assume or require a single canonical origin.

Patients may participate. Patients can hold their own signing key and create assertion events about themselves (address changes, preferred name, contact information). Patient-originated assertions carry a distinct verification-method tag and a lower default confidence than institutionally-verified assertions. Patient participation is optional and additive — the system functions fully without it.

3.2 Tenets

Tenets are operational commitments the engineering team adheres to when building and evolving the identity model.

Append-forward by default, mutable by exception. The natural operation is appending new events that reference prior events. Mutation (amendment, tombstoning) is a supported but exceptional path, required for regulatory compliance (right to be forgotten) and error correction. Mutation never silently rewrites history — it always leaves an auditable trace of what changed and why.

Fail open for reads, fail closed for writes. A node that cannot reach peers should still serve identity queries from its local subgraph (potentially stale). A node that cannot verify a write's signature or structural validity must reject it. Availability for reads; integrity for writes.

Carry the weakest assumption. When evaluating a provenance chain, consumers should weight their trust on the weakest link, not the strongest. A chain with one unverified self-report assertion is weaker than a chain of government-ID-verified assertions, regardless of how many strong assertions surround the weak one.

Design for the deceased. Death is a first-class identity lifecycle event, not an afterthought. The system must handle provenance chain closure, retention windows, and eventual data disposition from day one.

Interoperability is not optional. Every identity event must be expressible as a FHIR resource. Every Creda node must expose a standard FHIR API. Non-Creda participants must be able to interact with the system through standard FHIR operations without understanding the underlying DAG. Creda extends the ecosystem — it does not fork it.

Privacy by structure, not policy. Data minimization, demographic tokenization, and consent enforcement are architectural properties of the system, not administrative policies layered on top. The system should make it structurally difficult to leak PII, not merely against the rules.

3.3 What Is an Identity

In Creda, a patient identity is not a single record in a database. It is a directed acyclic subgraph composed of identity events — discrete, signed assertions made by institutions (or patients) over time. Each event references zero or more prior events, forming a provenance chain that records how the identity was established, verified, linked, corrected, and eventually closed.

The subgraph for a single patient may have multiple independent roots, created by institutions that encountered the patient without knowledge of each other. These roots remain independent until a link event connects them. Even after linking, the original roots and their downstream chains retain their independent provenance — the link is an assertion of equivalence, not a merge that destroys history.

To evaluate a patient's current identity, a consumer traverses the subgraph from the most recent events backward through parent references, respecting amendments and contestations along the way. This traversal produces an **effective identity** — the current best understanding of who this patient is, given all visible assertions. Different institutions may compute slightly different effective identities depending on which portions of the subgraph they can see, and this is by design.

3.4 Identity Event Types

Every node in the DAG is an event. All events share a common structure (defined in Section 5.1) and are distinguished by their event type. Creda's event types span two co-primary concerns — identity continuity (Section 3) and portable authorization (Section 4) — within a single shared enum:

```
enum IdentityEventType {
    // Identity continuity events (Section 3)
    Assert,
    Link,
    Contest,
    Attest,
    Amend,
    Tombstone,
    DeceasedDeclaration,

    // Portable authorization events (Section 4)
    AuthorizationGrant,        // supersedes the earlier Consent event type
    AuthorizationRevocation,   // withdraws a Grant (distinct from Tombstone)
    ExportReceipt,             // records data release / receipt under a Grant
}
```

The identity event types are described in this section; the authorization event types are described in Section 4.3. They share one enum, one node schema, and one replication fabric, but answer different questions: identity events are evaluated to compute who a patient is (advisory), while authorization events are evaluated to determine what is permitted (enforced). The `AuthorizationGrant` type supersedes the simpler `Consent` type from earlier Creda drafts — a consent directive is now expressed as an `AuthorizationGrant` with a minimal scope. Note that `AuthorizationRevocation` (withdrawing a permission) and `Tombstone` (scrubbing PII for right-to-be-forgotten) are deliberately distinct event types; Creda does not overload a single "revocation" concept across permission-withdrawal and content-destruction, because conflating them creates ambiguity in enforcement and audit.

This enum is **extensible**. New event types can be introduced via FHIR Implementation Guide versioning without breaking existing nodes. During replication, nodes that encounter an unknown event type must preserve the event and propagate it, but may ignore it during local subgraph traversal. This ensures forward compatibility as the protocol evolves.

3.4.1 Assert

An institution claims demographic facts about a patient. This is the foundational event type — most subgraphs begin with one or more Assert events.

An Assert event carries tokenized demographics: name, date of birth, sex, address, SSN fragment, MRNs (as an array, since a patient may have multiple), insurance member IDs (as an array), and an extensible key-value bag for additional identifiers that may aid in matching. The extensible bag allows institutions to include identifiers specific to their context (e.g., state Medicaid IDs, tribal enrollment numbers, military service numbers) without requiring schema changes.

Each Assert event also carries **verification method metadata** describing how the demographics were verified: government-issued photo ID scan, insurance card presentation, self-report, biometric capture, birth certificate, vital records lookup, or other. This metadata feeds directly into downstream confidence scoring — a government-ID-verified assertion carries more weight than a self-reported one.

An Assert event with no parent references is a **root node** — the beginning of an independent identity subgraph for this patient at this institution.

3.4.2 Link

An institution or matching algorithm determines that two independent subgraphs represent the same real-world person. The Link event references the head nodes of both subgraphs being connected.

Critically, a Link **does not merge** the subgraphs. Both retain their independent roots, their own provenance chains, and their own institutional ownership. The Link is an assertion of real-world equivalence — "we believe these two subgraphs refer to the same person" — and it carries metadata about why:

- **Confidence score**: a numeric weight expressing the strength of the match.
- **Method**: how the determination was made — manual clinician review, algorithmic demographic match, referral-based (patient presented with a reference to the other subgraph), insurance crosswalk, or other.
- **Linking institution**: the institution that made the determination, which may differ from the institutions that created either subgraph.

Link events are the primary mechanism by which the fragmented patient identity landscape consolidates. A patient who has been seen at five unrelated institutions will have five independent root subgraphs; Link events (potentially created by an HIE or by the institutions themselves) connect them into a single composite subgraph without any institution ceding ownership of its assertions.

3.4.3 Contest

A Contest event declares that a prior Link event is incorrect — the two subgraphs it connected do not, in fact, represent the same person. The Contest references the specific Link node being invalidated and carries a reason (e.g., manual review determined distinct patients, demographic conflict discovered, duplicate record identified during clinical care).

Downstream consumers that encounter a Contest must treat the referenced Link as invalid during subgraph traversal. The two subgraphs revert to being independent. The original Link event is **not deleted** — it remains in the graph for audit purposes, but is superseded by the Contest.

Contestation is **limited to institutions that are party to the linked subgraphs** — meaning the institution that created the Link, or any institution that has previously created an Assert, Attest, or Amend event within either of the two linked subgraphs. This scoping ensures that only institutions with direct knowledge of the patient(s) in question can challenge a link, preventing spurious contestations from unrelated parties while still allowing downstream consumers who have relied on the identity chain to flag errors they discover during clinical care.

3.4.4 Attest

An Attest event signals that an institution has relied on a provenance chain for a clinical or administrative purpose. It is a vote of confidence — "we used this identity chain to treat a patient, file a claim, or make a clinical decision."

Attestations do not carry demographic data. They reference one or more nodes in the subgraph that the institution relied upon, plus the purpose (treatment, payment, operations, public health reporting, etc.).

The accumulation of attestations from independent institutions is a strong signal of identity correctness. A subgraph with attestations from ten unrelated institutions is far more trustworthy than one with a single assertion, even if that single assertion was verified against a government ID. Attestation count and institutional diversity are inputs to confidence scoring (see Section 5.3).

3.4.5 Amend

An institution corrects a prior assertion it made. The Amend event references the original node being corrected and provides updated demographic data. The original node remains in the graph for audit purposes but is superseded — traversal logic treats the amendment as the current truth.

Only the originating institution may amend its own assertions. This is a hard constraint enforced by signature verification. An Amend event must be signed by the same institutional key that signed the original Assert, or by a successor key with a valid rotation chain. No institution can amend another institution's assertions — if Institution B believes Institution A's assertion is wrong, Institution B's recourse is to create its own Assert with different demographics and let the confidence scoring reflect the disagreement.

3.4.6 Tombstone

A Tombstone event implements the right to be forgotten under applicable law (e.g., state privacy laws, GDPR for international extensions). When a valid deletion request is received, a Tombstone is created that targets specific nodes in the subgraph.

The Tombstone replaces the demographic content of targeted nodes with a deletion marker. The graph structure — edges, event types, timestamps, and the fact that a node existed — may be retained for audit and structural integrity, but all PII is irreversibly scrubbed. What remains is a shell: the provenance chain shows that Institution A made an assertion on a given date, and that assertion was later tombstoned at the patient's request, but the content of the assertion is gone.

Tombstoning breaks content-addressed hashing, since the hash of a node changes when its content is replaced. To handle this, Creda uses **stable UUIDs as the primary addressing scheme**. Each identity event is assigned a UUID at creation time, and all references between events (parent pointers, link targets, contest targets, etc.) use UUIDs, not content hashes. Content hashes serve as an **optional integrity check** — they can verify that a node's payload has not been tampered with, but they are not load-bearing for graph traversal or replication. After tombstoning, the content hash is voided and marked as such. The graph structure remains intact because UUID-based references are unaffected by content changes.

Tombstone events propagate through the gossip network like any other event. Peers that receive a Tombstone must scrub the targeted content from their local stores. Failure to propagate tombstones is a compliance violation.

> **Authorization event types** (AuthorizationGrant, AuthorizationRevocation, ExportReceipt) were formerly modeled here as Consent and RevokeConsent. They have been promoted to the co-primary portable authorization layer and are specified in Section 4.3. A consent directive is now expressed as an AuthorizationGrant with a minimal scope.

3.4.7 DeceasedDeclaration

A DeceasedDeclaration is created by an authoritative institution — a hospital, coroner's office, state vital records agency, or similar — asserting that the patient is deceased. It carries the date of death, the certifier's identity, and a cause-of-death flag (present or absent — the actual cause of death is **not** stored in the identity graph, as it is clinical data, not identity data).

A DeceasedDeclaration triggers **provenance chain closure**:

- New Assert or Link events referencing a subgraph with a DeceasedDeclaration are **not rejected**, but the system issues a **soft warning** to the creating institution indicating that the subgraph is associated with a deceased patient. The creating institution may proceed if they have a valid reason (e.g., post-mortem correction to demographics for vital records, insurance reconciliation, organ donation coordination). The warning and the institution's acknowledgment are logged as metadata on the new event. This approach avoids blocking legitimate post-mortem workflows while ensuring that accidental identity activity against deceased patients is flagged.
- The subgraph enters a **retention window**. The retention period is configurable per regulatory jurisdiction and defaults to alignment with HIPAA's requirement of 6 years from the date of last activity. During the retention window, the subgraph is effectively read-only — existing events are preserved and queryable, but only administrative events (Amend for corrections, Tombstone for right-to-be-forgotten) are accepted.
- After the retention window expires, nodes may be archived to cold storage or tombstoned in bulk, depending on institutional policy and jurisdictional requirements.

DeceasedDeclaration is one of the few event types where the identity of the asserting institution carries exceptional weight. A declaration from a state vital records office is treated as authoritative. A declaration from a hospital is strong but potentially preliminary (patient may have been misidentified). The confidence model (Section 5.3) must account for this.

3.5 Temporal Ordering

Identity events carry two forms of timestamp:

- **Wall-clock timestamp**: the real-world time the event was created, as reported by the originating institution. Used for human readability, retention policy calculations, and regulatory compliance. Not relied upon for causal ordering, since institutional clocks may drift.
- **Logical clock**: a per-subgraph sequence number (Lamport timestamp or vector clock component) that establishes causal ordering among events within a subgraph. Incremented by the creating institution at write time. During replication, logical clocks enable consumers to reconstruct the causal order of events even if wall-clock timestamps are inconsistent.

The combination of wall-clock and logical ordering ensures that the system can answer both "when did this happen in the real world?" and "what was the order of events as understood by the system?" — and can do so correctly even under asynchronous replication with clock skew.

3.6 Signature Model

Every identity event is signed by the creating institution's private key. The signature covers the entire event payload: event type, demographics (if present), parent references, timestamps, and all metadata. The signing key chains back to the institution's UDAP certificate, which is anchored in the existing US Health IT trust framework.

Signature verification is **mandatory** during replication. A peer receiving an event via gossip must verify the signature against the claimed institution's public key before accepting the event into its local store. Events with missing, invalid, or unverifiable signatures are dropped and logged.

For key rotation — which is inevitable over the lifetime of the system — institutions publish key rotation events that chain the new key to the old one. Amend events signed by a rotated key are valid if the rotation chain is intact.

Patient-originated events (when patients hold their own keys) use a separate signing scheme. Patient keys are not UDAP certificates — they may be issued by a patient-facing application or identity provider. The trust model for patient keys is weaker by default, reflected in lower confidence scores for patient-originated assertions.

4. Portable Authorization

Identity provenance answers *who* a patient is. Portable authorization answers *what they have authorized* — and, critically, makes that answer verifiable at the point where data is actually used, not only at the moment a query is served. These are co-primary capabilities. Identity continuity without authorization is incomplete: you know who the patient is but cannot confirm what they permitted. Authorization without identity continuity is meaningless: you hold an authorization artifact but cannot confirm it belongs to this patient. Creda exists because cross-institutional health data exchange requires both, and neither exists today as shared infrastructure.

This section defines authorization as a first-class primitive alongside the identity model of Section 3. The two share the same DAG, the same event-node schema (Section 5), the same replication fabric (Sections 6–7), and the same signing and trust model. Authorization is not a separate system bolted onto identity — it is a parallel set of event types and an enforcement model that travels on the same rails.

4.1 The Problem Portable Authorization Solves

Current health data exchange verifies authorization at the moment of transfer and then forgets it. Once data leaves a source system, the authorization that permitted the exchange becomes invisible to downstream consumers. If a patient revokes consent an hour later, systems that already received the data have no way to know — and no obligation to check. Authorization is treated as a transient gate, checked once and discarded.

Creda treats authorization as a **persistent, verifiable state** that travels with the data reference and can be confirmed at any point of use. When a patient authorizes an exchange, the result is a signed, scoped artifact recorded in the DAG. Any relying system can later re-verify that artifact — its signature, scope, expiration, audience, and revocation status — locally, without contacting the originating institution. Authorization stops being a moment and becomes a checkable, revocable, auditable condition that persists across the full data lifecycle.

4.2 Authorization as Verifiable State

Authorization is represented as a portable, verifiable state object within the DAG — not as a permission controlled by a central authority or mediated by a consent service. When a patient authorizes data exchange, the resulting artifact is:

- **Cryptographically signed** by the institution acting on the patient's directive (or by the patient, if they hold their own key).

- **Scoped**: purpose, audience, duration, volume, and use-mode constraints.
- **Bound to identity context** without requiring a global identifier — the authorization references the patient's subgraph, not a universal patient ID.
- **Verifiable at any point of use** by traversing the provenance chain and checking the signature.
- **Non-transferable**: bound to the specified patient and audience; it cannot be reassigned.

Creda verifies authorization state. It does not itself grant, deny, broker, or mediate access to clinical data. Access decisions remain with the responding institution under applicable law and policy. This boundary is structural, not aspirational — no Creda component has the capability to grant or deny access to health records. It verifies that an authorization is signed, scoped, unexpired, and unrevoked; the institution decides what to do with that verification.

4.3 Authorization Event Types

Authorization introduces three event types that join the identity event types in the shared `IdentityEventType` enum (Section 5.1.3). They are listed here with identity events for completeness; the enum is the single source of truth.

4.3.1 AuthorizationGrant

An AuthorizationGrant records a patient's directive granting specific institutions (or classes of institutions) access to their subgraph and associated data. It supersedes the simpler Consent event type from earlier Creda drafts — a Consent is an AuthorizationGrant with a minimal scope. The Grant payload specifies:

- **Scope**: which subgraph segments, which event types, which data categories.
- **Audience**: a specific institutional identity (UDAP fingerprint), an institutional class (e.g., "any TEFCA QHIN," verified against the Participant Registry), or a constrained wildcard (e.g., "any institution with an active BAA").
- **Purpose**: treatment, payment, operations, public health, research, AI training, AI inference, or federal program eligibility/adjudication. Purpose is enumerated and extensible.
- **Duration**: an explicit expiration date or indefinite.
- **Volume constraints**: rate limits, record counts, or other quantitative bounds.
- **Use-mode constraints**: read-only, read-and-rely, or read-and-export.
- **Non-transferability binding**: the authorization is bound to the specified patient and cannot be reassigned.

An AuthorizationGrant may be created by the patient (if they hold a signing key) or by an institution acting on the patient's documented directive. The richer purpose enumeration is what makes Creda authorization useful beyond clinical TPO — research, AI, and federal program scopes carry distinct enforcement semantics (Section 4.6).

4.3.2 AuthorizationRevocation

An AuthorizationRevocation supersedes a prior AuthorizationGrant. It references the Grant being revoked and takes effect upon propagation, with bounded latency determined by the DAG's synchronization characteristics (Section 4.7). After revocation, participating peers must stop serving the affected subgraph segments to the previously-authorized institution.

Revocation does not delete the original Grant — the Grant remains in the DAG as an audit record showing that authorization was once granted and later withdrawn. This is materially stronger than revocation via an API call to a central service: revocation latency is deterministic and auditable at every peer, and the revocation itself is a signed, permanent event rather than a database mutation that could be silently reversed.

Note the relationship to the identity-side Tombstone (Section 3.4.6): Tombstone scrubs PII content for right-to-be-forgotten compliance; AuthorizationRevocation withdraws a permission. They are distinct operations with distinct event types — Creda deliberately does not overload a single "revocation" concept across both, because conflating "this permission no longer holds" with "this content must be destroyed" creates ambiguity in enforcement and audit.

4.3.3 ExportReceipt

An ExportReceipt records that data was issued from a source system under a specific AuthorizationGrant, and (optionally, in a paired downstream event) that a relying party acknowledged receipt under the same terms. The ExportReceipt carries a reference to the governing Grant, the requesting institution, the scope of what was released, and a timestamp.

ExportReceipts create a non-repudiable chain of custody. The source can prove it released data only under a valid Grant; the recipient's acknowledgment proves they accepted the data under known terms and cannot later claim ignorance of the scope or constraints. This closes a gap the identity model alone does not address: identity events record *who a patient is* and *who relied on that identity*, but not *that data moved, under what authorization, and that the recipient accepted the terms*.

4.4 The Portable Authorization Artifact

The Portable Authorization Artifact is an AuthorizationGrant event in its canonical CBOR serialization (Section 5.1.1), **detachable from the DAG for transport** to relying institutions. It carries the full Grant payload — scope, audience, purpose, expiration, volume constraints, use mode, and non-transferability binding — plus the originating institution's signature.

The Artifact is not a derived or summarized representation of the Grant; it is the Grant event itself, made portable by design. A relying institution that receives the Artifact verifies the signature against the originating institution's UDAP certificate before evaluating the Grant's constraints. Because the Artifact is the signed event, verification is self-contained: the relying party needs the originating institution's public key (available from the Participant Registry) but does not need to contact the originating institution or any central service.

This portability is what enables verification at the point of use. The Artifact can travel alongside a data reference through multiple institutional hops, and at each point a holder can independently confirm that the authorization is signed, scoped to cover the intended use, unexpired, and — by checking for an AuthorizationRevocation in their local DAG view — unrevoked.

4.5 Dual-Control Enforcement

Creda enforces authorization at two independent control points. Neither the source nor the relying party can unilaterally circumvent authorization: the source cannot export without a valid artifact, and the relying party cannot use data without local verification.

4.5.1 Source Side: Export Gate

Before data leaves a source system, the **Export Gate** validates the Portable Authorization Artifact governing the release — confirming that the authorization is signed, unexpired, correctly scoped for the requested data and purpose, addressed to the requesting institution's audience, and unrevoked. The Export Gate enforces source-side policy. If the authorization artifact fails validation, data is not exported.

The Export Gate runs at the source — typically inside or adjacent to the institution's EHR, data warehouse, or FHIR endpoint, wherever data egress occurs. It is a separate enforcement component (Section 10) precisely because it must sit at the egress boundary, not in the peer's query path. The Export Gate also emits the ExportReceipt event (Section 4.3.3) recording that the release occurred under a specific Grant.

4.5.2 Relying Side: Verifier

At the point of use, the **Verifier** validates three things together: that the authorization artifact is valid (signature, scope, expiration, audience, revocation status), that identity continuity holds for the patient (the subgraph traversal confirms the artifact belongs to this patient), and that the provenance chain is intact (no broken signatures or missing parents in the relevant chain).

The Verifier operates locally and can function offline using its most recent synchronized DAG state. It does not require a callback to the source system for routine verification. This is essential for two reasons: resilience (verification continues during network partitions or source outages) and adoption (a consuming system — an EHR, a payer adjudication system, a research platform — can embed the Verifier and check authorization locally without running a full Creda peer).

4.5.3 Why Two Controls

The dual-control model removes a single point of trust failure. In a single-control design, one actor decides whether data may be released and used — and a compromise or error at that actor defeats authorization entirely. With dual control:

- The Export Gate ensures data does not leave the source without a valid, current authorization, regardless of what the relying party claims.
- The Verifier ensures the relying party independently confirms authorization at use, regardless of what the source asserted at export.

Identity continuity is verified at both control points: the Export Gate confirms the authorization artifact is bound to the correct patient before release; the Verifier confirms identity continuity still holds at the point of use. An attacker would have to defeat both independent controls, operated by different parties, to misuse authorized data.

4.6 Authorization Evaluation Algorithm

When a Creda peer at Institution B receives a query from Institution A requesting access to events in a patient's subgraph, the responding peer executes the following algorithm before returning any events. The algorithm is local: it requires no network calls, no callback to the patient, and no contact with a central consent service. It is also the reference logic for the Verifier's evaluation function.

Step 1 — Collect AuthorizationGrants. Walk the patient's local subgraph and collect all AuthorizationGrant events in the responding peer's local store. Only Grants targeting this patient's subgraph are considered. Grants are collected regardless of current status; revoked Grants are filtered in Step 2.

Step 2 — Subtract revoked Grants. For each collected Grant, check whether a validated AuthorizationRevocation exists in the local store referencing that Grant's UUID. A Grant is revoked if and only if a Revocation with a matching `target_grant_id` exists *and* that Revocation has been validated (signature verified, parent references resolved). A Revocation that exists but has not been validated — for example, because its parent references have not yet arrived via replication — does **not** revoke the Grant; it is logged as a warning and the Grant remains active until the Revocation fully validates. This design prefers availability over premature enforcement: enforcing unvalidated Revocations would let an attacker block legitimate access by injecting structurally incomplete Revocation events. The remaining set is the active Grant set.

Step 3 — Match requesting institution against Grant audience. For each active Grant, evaluate whether Institution A matches the Grant's audience: specific institution match (UDAP fingerprint), institutional class match (e.g., "any TEFCA QHIN," verified against the Participant Registry), constrained wildcard (e.g., "any institution with an active BAA"), or no match (discard this Grant).

Step 4 — Evaluate scope, purpose, and use-mode. For each audience-matched Grant, evaluate whether the requested operation falls within the Grant's scope (subgraph segments, event types, data categories), purpose (treatment, payment, operations, public health, research, AI training, AI inference, federal program adjudication), and use-mode (read-only, read-and-rely, read-and-export). Grants not covering the requested scope, purpose, or use mode are discarded.

Step 5 — Check expiration and volume. For each scope-matched Grant, evaluate temporal and quantitative bounds. Expired Grants are discarded. Grants whose volume limits are exhausted are discarded. The responding peer tracks Grant utilization — the count of requests served under each Grant — to enforce volume constraints.

Step 6 — Apply cross-institutional policy honoring. When Institution B holds events that originated at Institution C, and Institution A requests those events, Institution B must honor the intersection of three layers: the patient's Grant (Steps 1–5), Institution C's redistribution policy for events it originated, and Institution B's own posture. The most restrictive of the three governs. Institution B cannot become a laundering point for data from a stricter institution. Redistribution policy is carried as a `redistribution_policy` metadata field on each event node, set by the originating institution at creation time and evaluated per event — a single response may include events from multiple originating institutions with different policies.

Step 7 — Determine outcome. If at least one Grant survives all steps, the request is authorized for the events those Grants cover. If no Grant survives, the responding peer applies its configured default posture (Section 9.3.2 — deny-by-default or treatment-presumed-authorization), with research, AI, and federal program scopes always requiring an explicit Grant regardless of posture.

4.7 Revocation Propagation and Bounded Latency

Because authorization revocation is safety-relevant — a patient who revokes consent expects that revocation to take effect — Creda treats revocation propagation latency as a measurable, bounded property rather than a best-effort hope. Three bounds apply:

- **Bound 1 (gossip propagation):** under normal conditions, an AuthorizationRevocation propagates to subscribed peers within the gossip convergence window (typically 1–2 seconds across the network, Section 6.1.1). This is the common case.
- **Bound 2 (anti-entropy):** a peer that missed the revocation via gossip — due to transient unavailability or partition — receives it during the next anti-entropy cycle for the affected subgraph (15 minutes for active subgraphs, Section 6.2.5).
- **Bound 3 (worst-case convergence):** following an extended partition, revocations reconcile when connectivity is restored and anti-entropy completes. This bound is partition-duration-dependent and therefore not fixed; it is the case operators monitor for.

Bound 1 is realistic and validated by the gossip design. Bounds 2 and 3 are stated as architectural commitments but require pilot validation (Section 13). Conformance tests (Section 10.5 tooling) verify that a revocation injected at one peer is reflected at subscribed peers within Bound 1 under normal conditions. This bounded-latency posture is materially stronger than revocation via a central service, where latency is opaque and unauditable; in Creda, revocation latency is deterministic under normal conditions and auditable at every peer.

4.8 Relationship to the Identity Model

Authorization and identity share one DAG and one trust fabric, but they answer different questions and fail differently:

- **Identity events** (Section 3) are evaluated to compute *who a patient is* — the effective identity projection. They are advisory: the consuming institution decides how much to trust the projection.
- **Authorization events** (this section) are evaluated to determine *what is permitted* — the authorization decision. They are enforced: a responding peer that finds no covering Grant (and no applicable default posture) returns nothing.

The two interact at exactly one point: an authorization is bound to a patient via the subgraph, so evaluating a Grant requires that identity continuity hold for the patient the Grant references. This is why identity is the first primitive and authorization the second — you cannot verify "this patient authorized X" without first establishing identity continuity for "this patient." But neither subsumes the other, and the spec treats them as co-primary throughout.

5. Data Structures

5.1 Identity Event Node

Every identity event in Creda is stored as a node with the following schema:

```
struct IdentityEventNode {
    // Primary key — stable across mutations (tombstoning, amend).
    // UUIDv7: time-ordered for natural storage-layer sorting,
    // namespaced to the creating institution's node ID to prevent
    // cross-institution collisions.
    id: UUIDv7,

    // Optional integrity check. Computed over the serialized payload
    // at creation time. Voided (set to None) after tombstoning.
    // Not used for addressing or graph traversal — purely for
    // tamper detection on untombstoned nodes.
    content_hash: Option<Blake3Hash>,

    // Whether the content hash has been voided by a tombstone.
    // Distinguishes "hash was never computed" from "hash was
    // invalidated by a legitimate tombstone operation."
    content_hash_voided: bool,

    event_type: IdentityEventType,

    // UUIDs of parent events. Empty array = root node.
    // One parent = linear extension. Two+ = merge/link point.
    // Ordered by logical clock of the referenced parent.
    parent_ids: Vec<UUIDv7>,

    // Event-type-specific payload. See Section 5.1.2.
    payload: EventPayload,

    // The institution that created this event, identified by
    // its UDAP certificate fingerprint.
    institution_id: CertificateFingerprint,

    // Signature over the canonical serialization of all fields
    // above, produced by the institution's private key.
    // Algorithm-agile: see Section 5.1.4 for PQC requirements.
    signature: CryptoSignature,

    // Real-world creation time. Not trusted for causal ordering.
    wall_clock_timestamp: RFC3339Timestamp,

    // Per-subgraph causal ordering. Monotonically increasing
    // within the scope of events this institution has observed
    // in this patient's subgraph.
    logical_clock: u64,
}
```

5.1.1 Serialization and Determinism

Signature verification requires that the same logical event always produces the same byte sequence. Creda uses **canonical CBOR (RFC 8949, Core Deterministic Encoding)** as its serialization format. Map keys are sorted lexicographically, floating-point values are avoided (all numeric values are integers or fixed-point), and optional fields that are absent are omitted entirely rather than encoded as null. This ensures that any two implementations serializing the same event produce identical bytes, and therefore identical signatures.

Protobuf was considered but rejected due to its non-deterministic handling of map fields and unknown fields, which would require additional canonicalization layers.

5.1.2 Post-Quantum Cryptography Requirements

Creda is designed for longevity — patient identity subgraphs may persist for decades, and identity provenance must remain verifiable over the full lifetime of the data. This makes post-quantum cryptographic (PQC) readiness a requirement from day one, not a future migration.

Hash function. Blake3 is used for content hashes. Blake3 is not directly threatened by quantum computing — Grover's algorithm reduces the effective security of a hash function by half (256-bit hash → 128-bit equivalent security against quantum search), and Blake3's 256-bit output provides a 128-bit post-quantum security margin, which meets NIST's recommended floor. Blake3 was chosen over SHA-256 (slower, same PQC security margin) and SHA-3 (less mature Rust ecosystem). If future NIST guidance raises the floor above 128-bit quantum security, the content hash field supports algorithm agility — the hash is stored alongside an algorithm identifier, allowing a transition to Blake3 with a longer output or a successor function without schema changes.

Signatures. The `CryptoSignature` type is algorithm-agile, carrying an algorithm identifier alongside the signature bytes:

```
struct CryptoSignature {
    algorithm: SignatureAlgorithm,
    public_key_fingerprint: Vec<u8>,
    signature_bytes: Vec<u8>,
}

enum SignatureAlgorithm {
    Ed25519,          // Current default, classical
    MlDsa65,          // FIPS 204 (ML-DSA-65, formerly Dilithium3) — PQC primary
    SlhDsa256s,       // FIPS 205 (SLH-DSA-SHA2-256s, formerly SPHINCS+) — PQC stateless fallback
    Ed25519MlDsa65,   // Hybrid: classical + PQC, both must verify
}
```

Migration path. Creda launches with `Ed25519` as the default to align with current UDAP certificate infrastructure. Institutions that are PQC-ready may use `MlDsa65` (NIST FIPS 204, the primary post-quantum digital signature standard) or the hybrid `Ed25519MlDsa65` mode, which requires both signatures to verify and provides security against both classical and quantum adversaries during the transition period.

The hybrid mode is recommended for institutions that want PQC protection today without abandoning classical verification that existing tooling understands. SLH-DSA (FIPS 205) is included as a stateless fallback — its signatures are larger but it does not depend on maintaining state, making it suitable for environments where stateful key management is impractical.

Verification policy. A peer's signature verification policy determines which algorithms it accepts. The minimum policy is: accept `Ed25519` (for backward compatibility), accept `MlDsa65` and `SlhDsa256s` (for PQC), accept `Ed25519MlDsa65` (hybrid). A future network-level policy update can deprecate `Ed25519`-only signatures once PQC adoption reaches a sufficient threshold, enforced by a configurable cutoff date after which classical-only signatures are rejected.

"Harvest now, decrypt later" defense. The primary PQC threat to Creda is not real-time forgery but the "harvest now, decrypt later" attack — an adversary captures signed events today and attempts to forge or re-sign them once quantum computers can break classical signatures. The hybrid signature mode defends against this: even if Ed25519 is broken, the ML-DSA-65 component remains secure. For events signed with classical-only Ed25519 before PQC adoption, institutions may optionally re-sign historical events with a PQC algorithm via an Attest event that references the original and adds a PQC signature layer, providing retroactive quantum resistance for critical provenance chains.

5.1.3 Payload Schema per Event Type

The `EventPayload` is a tagged union discriminated by `event_type`:

```
enum EventPayload {
    Assert {
        demographics: Demographics,
        verification_method: VerificationMethod,
    },
    Link {
        target_subgraph_heads: (UUIDv7, UUIDv7),
        confidence_score: u16,       // 0-10000, representing 0.00-100.00%
        method: LinkMethod,          // Manual, Algorithmic, Referral, InsuranceCrosswalk, Other
    },
    Contest {
        target_link_id: UUIDv7,
        reason: ContestReason,       // enum + freetext
    },
    Attest {
        target_event_ids: Vec<UUIDv7>,
        purpose: AttestPurpose,      // Treatment, Payment, Operations, PublicHealth, Other
    },
    Amend {
        target_event_id: UUIDv7,
        updated_demographics: Demographics,
        amendment_reason: String,
    },
    Tombstone {
        target_event_ids: Vec<UUIDv7>,
        legal_basis: TombstoneBasis, // RightToBeForgotten, StateLaw, CourtOrder, Other
    },
    AuthorizationGrant {
        scope: AuthorizationScope,        // subgraph segments, event types, data categories
        audience: GrantAudience,          // InstitutionId, InstitutionClass, or ConstrainedWildcard
        purpose: GrantPurpose,            // Treatment, Payment, Operations, PublicHealth,
                                          //   Research, AiTraining, AiInference, FederalProgram
        expiration: Option<RFC3339Timestamp>,
        volume_constraints: Option<VolumeConstraints>, // rate limits, record counts
        use_mode: UseMode,                // ReadOnly, ReadAndRely, ReadAndExport
        // Non-transferability is implicit: a Grant is bound to the
        // patient subgraph it references and cannot be reassigned.
    },
    AuthorizationRevocation {
        target_grant_id: UUIDv7,
    },
    ExportReceipt {
        governing_grant_id: UUIDv7,       // the Grant under which data was released
        requesting_institution: CertificateFingerprint,
        released_scope: AuthorizationScope, // what was actually released
        // A paired downstream ExportReceipt may acknowledge receipt
        // under the same governing Grant, completing the chain of custody.
    },
    DeceasedDeclaration {
        date_of_death: RFC3339Date,
        certifier_id: CertificateFingerprint,
        cause_of_death_present: bool,  // flag only, not the actual cause
    },
}
```

5.1.4 UUID Generation

Creda uses **UUIDv7** (RFC 9562) for all event node identifiers. UUIDv7 encodes a Unix timestamp in the high bits, providing natural time-ordering at the storage layer without requiring a separate timestamp index. The random component of the UUIDv7 is seeded with the creating institution's node ID, reducing collision probability across institutions generating events concurrently to a level that is negligible at the scale of the system (millions of patients, thousands of nodes).

UUIDv7 was chosen over UUIDv4 (no temporal ordering) and ULID (non-standard, limited ecosystem support in Rust) for its combination of time-ordering, uniqueness guarantees, and broad library support.

5.2 Patient Identity Subgraph

5.2.1 Subgraph Definition

A patient identity subgraph is **not a stored data structure** — it is a query result. It is the transitive closure of all events reachable by traversing parent references and link targets from a given set of entry points. Subgraphs are materialized on demand from the underlying event store, not maintained as a separate unit.

This design avoids the need to keep a separate "subgraph" object in sync with its constituent events. The events are the source of truth; the subgraph is a derived view.

5.2.2 Root Discovery

To find all roots for a patient's subgraph, traverse backward from any known event belonging to that patient until reaching nodes with zero parents. Multiple roots are expected and normal — they represent independent institutional encounters with the same patient before any Link events connected them.

The complete root set defines the full scope of a patient's known identity. A consumer that can see three roots connected by two links has a more complete view than one that can see only a single root — and the system makes this visibility explicit rather than hiding it behind a merged record.

5.2.3 Fork and Split Semantics

Forks occur naturally when multiple institutions create events concurrently against the same subgraph — two new events with the same parent, created independently. This is not a conflict; it is concurrency. Both events are valid and coexist in the subgraph. A subsequent event that references both as parents creates a merge point, similar to a Git merge commit.

Splits occur via Contest events. When a Link is contested, the two subgraphs it connected become logically independent again. They are not physically separated — the Link event and the Contest event both remain in the store — but traversal logic treats the contested Link as a no-op, effectively splitting the composite subgraph back into its constituent parts.

5.2.4 Effective Identity Computation

The effective identity is the current best understanding of a patient's demographics, computed by traversal. The algorithm:

1. Start from all leaf nodes (events with no children) in the subgraph.
2. Walk backward through parent references.
3. At each Amend event, substitute the amended payload for the original, so the original assertion is superseded.
4. At each Contest event, mark the target Link as invalid and exclude the linked subgraph from further traversal through that path.
5. At each Tombstone, treat the targeted nodes as having no demographic content.
6. Collect all uncontested, untombstoned Assert events (including their amendments).
7. Aggregate demographics per field across all collected assertions, weighted by confidence (see Section 5.3).
8. The output is a computed demographic record with per-field confidence scores.

The effective identity is a **projection**, not a stored record. It may differ between institutions depending on which events they have replicated locally. This is by design — the system does not pretend to offer a single universal truth, but rather gives each consumer the best answer derivable from the events available to them.

5.2.5 Index Structures

Efficient subgraph operations require secondary indexes maintained alongside the primary event store:

- **Demographic token → subgraph entry points**: enables matching queries. When an institution needs to find subgraphs for a patient presenting at registration, it tokenizes the patient's demographics and looks up matching entry points. This is the primary interface between Creda and institutional matching logic.
- **Institution ID → event UUIDs**: enables institutional audit. An institution can enumerate all events it has ever created across all patient subgraphs.
- **Event UUID → event node**: primary key lookup for individual event retrieval.
- **Parent UUID → child UUIDs**: forward traversal index. Given an event, find all events that reference it as a parent. Necessary for computing leaf nodes and for propagating tombstone effects forward through the graph.

Indexes are local to each peer and rebuilt from the event store during bootstrap. They are not replicated — each peer maintains its own indexes over its local event set.

5.3 Confidence and Trust Metadata

5.3.1 Demographics Struct

```
struct Demographics {
    name_family: Option<Vec<TokenizedString>>,
    name_given: Option<Vec<TokenizedString>>,
    name_middle: Option<Vec<TokenizedString>>,
    date_of_birth: Option<TokenizedDate>,
    sex: Option<AdministrativeGender>,    // FHIR valueset: male, female, other, unknown
    address: Option<StructuredAddress>,
    ssn_last_four: Option<TokenizedString>,
    mrns: Vec<InstitutionalIdentifier>,   // (institution_id, mrn_value) pairs
    insurance_member_ids: Vec<InsuranceIdentifier>, // (payer_id, member_id) pairs
    extensions: HashMap<String, TokenizedString>,   // extensible key-value bag
}
```

All fields are optional. An Assert event need not carry all demographics — an institution may assert only what it has verified. The `extensions` map allows institutions to include identifiers specific to their context (state Medicaid IDs, tribal enrollment numbers, military service numbers, etc.) without requiring schema changes. Extension keys should follow a namespace convention (e.g., `us-va:veteran-id`, `us-medicaid-ca:beneficiary-id`) to avoid collisions.

5.3.2 Per-Field Confidence Model

Confidence in Creda is computed **per demographic field, not per patient**. A patient's date of birth might be high-confidence (verified by government ID at three independent institutions) while their address is low-confidence (self-reported once, two years ago). This granularity prevents a single strong assertion from inflating confidence in unrelated fields.

For a given demographic field, the confidence score is a function of four inputs:

Verification method weight. Each verification method carries a base weight reflecting its reliability for identity purposes:

- Government-issued photo ID: high
- Birth certificate / vital records: high
- Insurance card: medium
- Biometric: high (but may not be available for all fields)
- Self-report: low
- Referral-based (inherited from another institution's assertion): the referring assertion's confidence, discounted

These weights are configurable at the network level and may be overridden by consuming institutions based on their own trust policies.

Institutional credibility weight. Not all institutions carry equal weight for identity purposes. A state vital records office has higher identity credibility than a walk-in clinic. A large academic medical center with robust identity verification workflows may be weighted higher than a small rural clinic with manual processes. Credibility weights are maintained as network-level configuration, updatable by consensus, and overridable locally.

Attestation amplification. When independent institutions attest to having relied on a provenance chain, confidence in the assertions within that chain increases. The key word is **independent** — three attestations from hospitals within the same health system provide less amplification than three attestations from unrelated institutions. Attestation count and institutional diversity are both inputs.

Agreement amplification. When multiple independent institutions assert the **same value** for a demographic field (e.g., the same date of birth), confidence increases superlinearly. Independence matters — three assertions from the same health system are worth less than three from unrelated institutions. The amplification function should reflect diminishing returns: the tenth independent agreement adds less marginal confidence than the second.

5.3.3 Temporal Decay

Assertions lose confidence over time unless reinforced by attestations or corroborating assertions. A 10-year-old address assertion with no recent attestation should score lower than a 6-month-old one. However, some fields decay differently — a date of birth effectively never decays (it doesn't change), while an address decays relatively quickly.

The decay function is configurable per field type:

- **Non-decaying fields**: date of birth, sex (changes are modeled as amendments, not decay), SSN.
- **Slow-decaying fields**: name (legal name changes happen but are infrequent).
- **Fast-decaying fields**: address, phone number, insurance member ID.

The decay curve is configurable — linear, exponential, or step-function with a cliff at a configurable threshold (e.g., full confidence for 2 years, then linear decay to a floor over the next 5 years). An attestation or corroborating assertion resets the decay clock for the attested fields.

5.3.4 Disagreement Flagging

When institutions assert **conflicting values** for the same demographic field (e.g., different dates of birth, different spellings of a name), the effective identity computation does not pick a winner. Instead, it flags the field as **disputed** and presents all asserted values with their respective confidence scores.

Resolution is left to the consuming institution. Creda's role is to surface the disagreement and provide the evidence (provenance chains for each competing assertion) — not to adjudicate. A consuming institution may choose the highest-confidence value, may flag the patient for manual review, or may apply its own resolution logic. The system makes the disagreement visible rather than silently choosing.

This design reflects the reality that demographic conflicts often indicate real-world complexity (legal name vs. preferred name, old address vs. new address, data entry error at one institution) rather than a system failure. The correct resolution depends on context that only the consuming institution has.

6. Network Architecture

Creda's network is a fully decentralized peer-to-peer overlay. There are no leader nodes, coordinator nodes, or privileged peers. Every peer can accept writes, serve reads, participate in gossip, and contribute to the distributed hash table. This section describes the network components in dependency order: foundational components that the system cannot function without, complementary components that reinforce each other, architectural choices where alternatives exist, and components that can be deferred for initial deployment.

6.1 Foundational Components

These components are non-negotiable. The system does not function without them.

6.1.1 Peer Identity

Each Creda peer is a k8s pod (or set of pods) operated by an institution. Peer identity is established through two independent credentials that serve different purposes:

- **SPIFFE ID** (via SPIRE, the SPIFFE runtime): Authenticates the workload. Proves "this is a legitimate Creda peer process running in an authorized k8s cluster." The SPIFFE ID is issued by the institution's SPIRE server and is scoped to the Creda workload. It is used for transport-layer authentication during the Noise handshake (see Section 6.2.3).
- **UDAP certificate**: Authenticates the institution. Proves "this process belongs to Hospital X, a real healthcare organization registered in the US Health IT trust framework." The UDAP certificate is the same credential the institution uses for FHIR endpoint authentication, tying Creda's trust model to the existing ecosystem.

Both credentials are required. A peer presenting only a SPIFFE ID is an authenticated workload but an unknown institution — it cannot sign identity events. A peer presenting only a UDAP certificate has institutional identity but no workload attestation — it could be a compromised process impersonating a Creda peer.

The peer's UDAP certificate fingerprint is the `institution_id` on every identity event it creates. The SPIFFE ID is ephemeral and per-pod — it does not appear in the identity graph, only in transport-layer authentication.

6.1.2 Homogeneous Peer Roles

Every peer is architecturally equal. There are no designated leaders, no write-coordinators, no routing authorities. Any peer can:

- Accept new identity events from its institution's local systems.
- Propagate events to other peers via gossip.
- Respond to DHT queries for patient subgraph routing.
- Serve identity queries from its local event store.
- Participate in anti-entropy with other peers holding the same subgraphs.

This is the core decentralization guarantee. Introducing any form of role differentiation (e.g., "super-peers" that coordinate writes, or "gateway peers" that bridge between institutions) reintroduces centralization and single points of failure. The system is designed so that no peer's failure degrades the network beyond the temporary unavailability of that peer's locally-created events, which are replicated to other peers via gossip.

An institution may operate multiple peers for redundancy and load distribution, but these peers are identical in capability — they are not sharded by function.

6.1.3 Network Join Protocol

Joining the Creda network requires both technical onboarding and a legal admission step. The legal step is non-optional: under HIPAA, any peer that receives Protected Health Information (PHI) — including the tokenized demographics carried in identity events — must have a Business Associate Agreement (BAA) in place with each other peer's covered entity. Creda formalizes this through a **Network Participation Agreement (NPA)**, a multi-party BAA framework that participating institutions sign once to establish bilateral BAA coverage with all other participants.

The join process:

1. **Legal admission.** The institution executes the NPA with the network's legal coordinator (typically a participating HIE or a designated nonprofit serving this role). The NPA establishes BAA coverage between the new institution and all existing participants. Existing participants are notified of the new participant via an out-of-band registry.
2. **Credential issuance.** Upon NPA execution, the institution's UDAP certificate is registered in the **Creda Participant Registry** — a signed, replicated list of authorized institutional UDAP certificate fingerprints. The registry itself is maintained as a Creda subgraph (using the same DAG mechanics described in this spec, with the legal coordinator as the asserting institution), making it tamper-evident and decentralized in the same way as patient identity data.
3. **Technical deployment.** The institution deploys a Creda peer configured with their UDAP certificate and SPIFFE identity.
4. **Network entry.** The peer connects to a bootstrap peer and performs mutual authentication (Noise handshake with SPIFFE, UDAP certificate exchange). The bootstrap peer verifies the UDAP certificate fingerprint against the Participant Registry. If the fingerprint is not registered or has been revoked, the connection is rejected.
5. **Mesh and DHT bootstrap.** The bootstrap peer shares its partial view of the network. The new peer begins gossipsub mesh joining and Kademlia DHT bootstrap (iterative `FIND_NODE` queries to populate its routing table).
6. **Operational readiness.** Within a few seconds, the peer has an active view, is participating in gossip, and is reachable via the DHT.
7. **Bulk loading.** If the institution has existing patient identity data to load (e.g., from a legacy MPI), it creates Assert events for its patients and publishes them via normal gossip. Rate limiting (Section 6.2.2) applies — bulk loads are throttled to avoid flooding.

Revocation. When an institution leaves the network, withdraws from the NPA, has its UDAP certificate revoked, or is removed for cause (e.g., persistent misbehavior despite reputation downgrades), the legal coordinator publishes a revocation event in the Participant Registry. Existing peers process the revocation, drop active connections to the revoked peer, and reject future connection attempts. Events the revoked peer previously created remain in the network — revocation is forward-looking, not retroactive — but the institution can no longer create new events.

Why this is not centralization. The legal coordinator role is administrative, not architectural. The coordinator does not see patient data, cannot create or modify identity events on behalf of patients, and cannot censor events from existing participants. Their sole technical capability is publishing additions and revocations to the Participant Registry, which is itself transparent and auditable. The coordinator role can be transferred between organizations or distributed across multiple coordinators (e.g., regional coordinators for different HIE jurisdictions) without changing the protocol. This mirrors how DirectTrust operates in the existing Direct messaging ecosystem — a coordinating body for trust framework administration, not a data intermediary.

Bootstrap peers themselves are operationally important but not architecturally privileged. They are normal peers that happen to have well-known addresses. If all bootstrap peers fail, existing network participants continue operating — only new joins are affected. The bootstrap peer list should include at least 3-5 peers operated by independent institutions to avoid a single point of failure for network entry.

6.1.4 Gossip Protocol for Event Propagation

New identity events propagate through the network via epidemic gossip. When an institution's local system creates an event (e.g., a new Assert during patient registration), the local Creda peer:

1. Validates the event (schema, signature, structural integrity).
2. Stores the event in its local event store.
3. Pushes the event to peers in its active view (see Section 6.2.1).

Each receiving peer performs the same validation, stores locally, and pushes to its own active view neighbors. Events propagate exponentially through the network. Deduplication is performed by tracking recently-seen event UUIDs in a bounded set (e.g., a Bloom filter or LRU cache) — peers that have already seen an event UUID ignore subsequent deliveries.

Convergence characteristics. For a network of N peers, epidemic gossip converges in O(log N) rounds. At 1,000 peers with a gossip interval of 100ms, full propagation takes approximately 10 rounds or ~1 second. At 10,000 peers, approximately 13-14 rounds or ~1.3-1.4 seconds. These are theoretical bounds — real-world propagation depends on network latency, peer availability, and fanout configuration.

Consistency guarantee. Gossip is best-effort. It does not guarantee delivery — messages may be dropped due to network issues, peer unavailability, or rate limiting. The anti-entropy protocol (Section 6.4) provides the consistency backstop, ensuring that any events gossip misses are eventually synchronized.

6.1.5 Distributed Hash Table for Subgraph Routing

Creda runs a Kademlia DHT for patient subgraph discovery. The DHT answers the question: "Which peers hold identity events for this patient?"

When Institution A needs to find a patient's subgraph (e.g., a patient presents at registration and the institution wants to check for existing identity provenance), it:

1. Tokenizes the patient's demographics using the standard tokenization scheme (Section 9.2).
2. Derives a DHT key from the demographic tokens (see Section 6.1.6).
3. Queries the DHT for providers of that key.
4. Receives a set of peer IDs that have announced themselves as holders of events in that patient's subgraph.
5. Makes targeted peer-to-peer requests to those peers for the actual events.

The DHT does not store patient data — it stores only the mapping from subgraph keys to peer IDs. Patient demographics never traverse the DHT; only tokenized keys do.

6.1.6 DHT Key Derivation

The DHT key for a patient subgraph is derived from a hash of core demographic tokens:

```
dht_key = Blake3(
    tokenize(name_family) ||
    tokenize(date_of_birth) ||
    tokenize(sex)
)
```

The tokenization scheme must produce the same output across all institutions for the same input demographics — this is defined in Section 9.2 and is critical for the DHT to function. The choice of fields (family name, DOB, sex) balances specificity (enough to narrow results) with availability (these three fields are almost always present at registration).

Institutions may also derive secondary DHT keys from alternative field combinations (e.g., SSN fragment + DOB, insurance member ID) and query multiple keys to improve recall. The DHT supports multiple keys per patient subgraph.

A peer that stores events for a patient announces itself as a provider for that patient's DHT key. Announcements are refreshed every 24 hours and expire if not refreshed, keeping the DHT current as peers join and leave.

6.1.7 Partition Tolerance

Network partitions — whether between k8s clusters, across availability zones, or between on-premises and cloud deployments — are expected, not exceptional. During a partition, each partition side continues operating independently:

- Peers accept new writes from their local institutions.
- Peers serve reads from their local event stores (which may be stale relative to the other side of the partition).
- Gossip continues within each partition side.
- The DHT fragments — peers on each side only see providers on their side.

On reconnection, anti-entropy (Section 6.4) detects divergence and synchronizes. Because the DAG is append-forward (new events reference existing ones by UUID) and acyclicity is guaranteed by construction (you can only reference events that already exist), partitions cannot create structural inconsistencies. The two sides simply merge their event sets — the union of two valid DAGs is a valid DAG.

The only semantic edge case is concurrent Link events that assert conflicting identity relationships (e.g., one side links Patient A's subgraph to Patient B's, while the other side links Patient A's subgraph to Patient C's). These are not system-level conflicts — both Links are valid events. They surface as competing identity assertions in the confidence model (Section 5.3.4), and consuming institutions resolve them locally.

6.1.8 Merkle Root for Anti-Entropy

Anti-entropy requires a fast mechanism for two peers to determine whether their copies of a patient subgraph are in sync. Creda computes a **Merkle root over the sorted set of event UUIDs** in a subgraph.

This is deliberately not a Merkle root over event contents — tombstoned nodes would cause roots to diverge even when both peers have the same event set. By hashing only the sorted UUID set, two peers that hold the same events (regardless of content mutations due to tombstoning) will compute the same root.

The comparison protocol:

1. Peer A sends the Merkle root for a patient subgraph to Peer B.
2. If Peer B's root matches, they're in sync. Done.
3. If roots differ, they exchange their UUID sets (or, for large subgraphs, walk the Merkle tree to identify the differing subtrees).
4. Each side identifies UUIDs it has that the other doesn't, and they exchange the missing events.

This is structurally identical to how Git determines which objects to transfer during a fetch — compare roots, identify deltas, transfer only what's needed.

6.2 Complementary Components

These components reinforce each other and are best adopted together. Each pair is described with its mutual dependency.

6.2.1 Peer Discovery and Networking Layer

Peer discovery determines which peers each node communicates with. **The networking layer** provides the transport, encryption, and protocol primitives.

Peer discovery uses a partial-view protocol where each peer maintains:

- An **active view** of 6-8 peers for direct, frequent communication (gossip, anti-entropy).
- A **passive view** of 30-50 peers as backup candidates if active peers fail.

The active view is maintained by periodic shuffling — peers exchange passive view entries, promoting some to active and demoting others. This provides resilience to churn (peers joining and leaving) and fast recovery from failures (a failed active peer is replaced from the passive view within seconds).

The networking layer is **libp2p** (Rust implementation via `rust-libp2p`). libp2p provides:

- **Gossipsub**: topic-based publish-subscribe with mesh management that subsumes the partial-view protocol. Gossipsub's mesh overlay aligns naturally with Creda's needs — each patient subgraph can be a topic, and peers subscribe to topics for patients they care about.
- **Kademlia DHT**: the DHT implementation described in Section 6.1.5.
- **Noise protocol**: encrypted, authenticated transport (see Section 6.2.3).
- **NAT traversal and relay**: enables peers behind firewalls or NATs to participate, critical for on-premises deployments.
- **Connection multiplexing**: multiple logical streams over a single connection, reducing connection overhead between peers.

libp2p was chosen over a custom networking stack because it is hardened by the IPFS, Filecoin, and Ethereum ecosystems, has a mature Rust implementation, and provides all required primitives in a single dependency. The alternative — building gossip, DHT, encryption, and NAT traversal from scratch on raw QUIC — is viable but represents 6-12 months of additional engineering effort with no functional advantage. If the team has concerns about libp2p (binary size, API stability, health IT compliance review), the networking layer is abstracted behind a trait boundary in Creda Core (Section 10.1) to allow future replacement.

6.2.2 Gossip Batching and Rate Limiting

Gossip batching and **rate limiting** jointly control bandwidth consumption.

Batching. Gossip messages carry batches of events, not individual events. Batches are assembled by a peer's outbound gossip queue using a dual trigger:

- **Time trigger**: flush every 100ms regardless of batch size.
- **Size trigger**: flush when the batch reaches 64 events (configurable) regardless of time elapsed.

Each batch includes the sender's peer ID, a batch sequence number (for deduplication at the batch level in addition to event-level dedup), and the serialized events in canonical CBOR.

Batching amortizes the per-message overhead of Noise encryption, gossipsub framing, and network round trips. At typical event creation rates (a few events per second per institution during normal operations), batching reduces message count by 10-50x compared to per-event gossip.

Rate limiting. To prevent a single peer from flooding the network — whether through misconfiguration, a bulk historical load, or malicious intent — gossip messages are rate-limited per sender. The default rate limit is 100 events/second per peer, sufficient for even large institutions during bulk operations (e.g., initial load of historical identity data from an existing MPI). Events that exceed the rate limit are queued locally and drained as the rate limit permits. They are not dropped — rate limiting introduces latency, not data loss.

Batching without rate limiting still allows floods (just in larger packets). Rate limiting without batching wastes overhead on per-event messages. Together, they keep bandwidth predictable and bounded.

6.2.3 Transport Encryption and Peer Identity

Transport encryption and **peer identity** (Section 6.1.1) jointly ensure confidentiality and authenticity.

All peer-to-peer communication is encrypted using the **Noise protocol framework** (via libp2p's noise transport). The Noise handshake authenticates both sides using their SPIFFE-issued keys, establishing an encrypted channel. Unencrypted connections are rejected — there is no plaintext fallback.

Noise was chosen over TLS for its simplicity (no certificate chain validation — SPIFFE handles that), lower handshake latency (1-RTT for most patterns), and native integration with libp2p. The Noise `XX` handshake pattern is used, providing mutual authentication.

Encryption without peer identity authentication is vulnerable to man-in-the-middle attacks. Identity authentication without encryption leaks PHI in transit. Both are required.

6.2.4 Topic-Based Gossip and Subgraph Announcement

Topic-based gossip controls which events a peer receives. **Subgraph announcement** controls which peers the DHT directs queries to. They are two sides of the same routing problem.

Topic-based gossip with bucketing. Not every peer needs every event. At millions of patients across thousands of peers, full-mesh gossip of all events is bandwidth-prohibitive. A naive design would assign one gossipsub topic per patient subgraph, but topic cardinality in the millions stresses gossipsub's mesh management and creates per-topic overhead that does not amortize well.

Creda instead uses **bucketed topics**. Each patient subgraph's DHT key is hashed into one of 1,024 topic buckets:

```
topic_id = "creda/v1/subgraph/" + (Blake3(dht_key) mod 1024)
```

A peer subscribes to topic buckets containing patients its institution has an active relationship with — typically a small subset of the 1,024 buckets, since each institution interacts with a bounded patient population. Peers receive all events in their subscribed buckets, including events for patients they don't actively care about, and filter locally. This trades a modest amount of unnecessary received traffic (events for irrelevant patients within the same bucket) for dramatically reduced topic cardinality (1,024 stable topics vs. millions).

The bucket count (1,024) is a tuning parameter. It can be adjusted upward if institutions grow large enough that per-bucket traffic becomes excessive, or downward if the network grows small enough that per-bucket sparsity wastes mesh management overhead. The bucket function is fixed at the protocol level — changing it requires a coordinated network upgrade.

Peers that need events for an unsubscribed patient (e.g., a new patient presenting at registration whose bucket the peer has never subscribed to) query the DHT, pull the subgraph from identified peers, subscribe to the relevant bucket for ongoing updates, and unsubscribe from buckets no longer needed during periodic subscription rebalancing.

Subgraph announcement. When a peer stores events for a patient, it announces itself as a provider for that patient's DHT key. This announcement tells the network "I have events for this patient — query me if you need them." Announcements are refreshed every 24 hours and expire if not refreshed.

Topic subscription says "send me future events for this patient." Subgraph announcement says "I have existing events for this patient." Together, they ensure that both real-time propagation and historical retrieval work correctly.

6.2.5 Anti-Entropy and Snapshot Bootstrap

Anti-entropy catches ongoing replication drift. **Snapshot bootstrap** handles cold start. Together they ensure every peer converges to a complete event set.

Anti-entropy protocol. Peers that hold events for the same patient subgraph periodically compare state using the Merkle root mechanism (Section 6.1.8). The protocol runs on a configurable schedule:

- **Active subgraphs** (events created or received in the last 24 hours): anti-entropy every 15 minutes.
- **Warm subgraphs** (events in the last 30 days): anti-entropy every 6 hours.

- **Dormant subgraphs** (no recent activity): anti-entropy every 7 days.

This tiering concentrates anti-entropy bandwidth on subgraphs where drift is most likely (recently active) while still eventually checking cold subgraphs.

Anti-entropy partners are selected from peers that the DHT indicates are providers for the same subgraph. A peer does not run anti-entropy against every other peer — only against peers that should have overlapping event sets.

Snapshot bootstrap. When a new peer joins (new institution onboarding, or a replacement pod after a failure), it needs to catch up on events for its institution's patients. Full replay from gossip history is impractical — gossip is ephemeral and not stored.

The bootstrap process:

1. The new peer connects to bootstrap peers and joins the gossip network.
2. It downloads the most recent snapshot of the event store from its institution's object storage (S3-compatible, institution-operated). Snapshots are produced periodically (e.g., every 6 hours) by existing peers in the same institution.
3. The snapshot is loaded into the new peer's local event store.
4. The peer subscribes to gossip topics for its institution's patients and begins receiving new events.
5. Anti-entropy runs immediately against known peers to catch events created between the snapshot timestamp and now.
6. The peer is considered ready when anti-entropy reports zero deltas for all active subgraphs.

Snapshots are institution-scoped — an institution's snapshot contains only events that institution's peers have stored, not the entire network's event set. This limits snapshot size and avoids sharing events that consent policies might restrict.

6.3 Architectural Choices

Where the design admits alternatives, this section documents the choice made and the rationale.

6.3.1 libp2p vs. Custom Networking Stack

Choice: libp2p.

libp2p provides gossipsub, Kademlia DHT, Noise transport, NAT traversal, and connection multiplexing in a single, well-maintained Rust crate (`rust-libp2p`). It is the networking layer for IPFS, Filecoin, and Ethereum's consensus clients — systems that operate at scales comparable to or exceeding Creda's target.

The alternative is a custom stack built on QUIC (via the `quinn` crate) with bespoke gossip, DHT, and encryption implementations. This offers more control and potentially smaller binary size, but at the cost of 6-12 months of additional engineering, extensive security review, and ongoing maintenance of networking primitives that are not Creda's core value proposition.

The risk of libp2p is dependency on an external project's roadmap and potential compliance concerns in a health IT context (libp2p has not been reviewed for HIPAA-regulated environments). To mitigate this, Creda Core (Section 10.1) abstracts the networking layer behind a trait boundary (`NetworkTransport`), allowing libp2p to be replaced if necessary without restructuring the rest of the system.

6.3.2 Snapshot Storage: Object Storage vs. Peer-to-Peer Transfer

Choice: Institution-operated object storage (S3-compatible) as default, peer-to-peer transfer as fallback.

Most institutions in the US Health IT ecosystem operate cloud infrastructure with S3-compatible storage readily available. Snapshots written to object storage are simple to produce, simple to consume, support resumable downloads, and decouple the bootstrapping peer from any specific source peer.

For fully on-premises deployments without object storage, a fallback peer-to-peer snapshot transfer protocol is provided. An existing peer in the same institution streams its event store to the bootstrapping peer over a libp2p stream. This adds load to the source peer and complicates resumable transfers (if the connection drops mid-transfer, the process must restart or implement chunked transfer with checkpointing). It is functional but operationally inferior to object storage.

The snapshot format is the same regardless of transport: a sorted sequence of canonical CBOR-encoded events, plus a manifest containing the snapshot timestamp, event count, and a Blake3 hash of the full snapshot for integrity verification.

6.4 Deferrable Components

These components add significant value but are not required for an initial deployment among trusted participants. Each includes a trigger condition for when it should be promoted to required.

6.4.1 Peer Reputation

Deferrable until the network opens beyond a single trust boundary (e.g., a single HIE's participants).

Since there is no admission control — any institution with a valid UDAP certificate can join — the network needs a mechanism to handle misbehaving peers. Misbehavior includes: flooding events with invalid signatures, propagating events that fail schema validation, refusing to propagate tombstones (a compliance violation), or consistently failing to respond to anti-entropy requests.

Creda uses a **local reputation score per peer**. Each peer maintains its own reputation table — there is no global blacklist (which would reintroduce centralization). Reputation is computed from:

- **Signature validity rate**: percentage of events received from this peer with valid signatures.
- **Schema validity rate**: percentage of events that pass schema validation.
- **Tombstone compliance**: whether the peer propagates tombstones it receives (verified during anti-entropy — if a peer is missing tombstoned events that other peers have, its reputation decreases).
- **Anti-entropy responsiveness**: whether the peer responds to anti-entropy requests in a timely manner.
- **Rate limit compliance**: whether the peer respects rate limits or consistently attempts to exceed them.

Low-reputation peers are deprioritized in the active view — they are replaced by passive view peers with better reputation. Peers whose reputation drops below a configurable floor are disconnected entirely. Reputation recovers over time if behavior improves.

Trigger for promotion to required: When any institution outside the initial trust boundary joins the network, or when the network exceeds 50 participating institutions.

6.4.2 Cross-Region Deployment

Deferrable until the network spans multiple geographic regions with >50ms inter-region latency.

For a single-region deployment (e.g., one state HIE with all peers in the same cloud region), cross-region concerns are irrelevant. For nationwide or multi-region deployment:

- **Gossip fanout tuning**: Ensure that each peer's active view includes at least one peer per region. This guarantees that events propagate across regions within one gossip round, even if intra-region propagation is faster. libp2p's gossipsub supports peer scoring functions that can prioritize geographic diversity in the mesh.
- **NAT traversal and relay**: Institutions operating on-premises behind corporate firewalls need NAT hole-punching or relay peers to participate. libp2p provides both via its AutoNAT and circuit relay v2 protocols. Relay peers can be operated by willing institutions (e.g., cloud-hosted HIEs) without becoming centralized authorities — any peer can be a relay, and relay selection is opportunistic.
- **Anti-entropy scheduling**: Cross-region anti-entropy should be scheduled less aggressively than intra-region to avoid saturating inter-region links. The tiered scheduling (Section 6.2.5) naturally accommodates this if anti-entropy partners are selected with geographic awareness.

Trigger for promotion to required: When peers are deployed in more than one cloud region or when on-premises peers join the network.

6.4.3 Network Observability

Deferrable in strict technical terms — the system runs without metrics — but strongly recommended to ship with v1.

Each peer exposes Prometheus-compatible metrics for network health:

- **Gossip metrics**: events received/second, events propagated/second, deduplication hit rate, batch size distribution, gossip round-trip time.
- **DHT metrics**: query latency (p50, p95, p99), provider announcement count, key lookup success rate.
- **Anti-entropy metrics**: sync frequency, delta size per sync (events exchanged), time to converge after a detected divergence.
- **Peer health**: active view size, passive view size, connected peer count, connection churn rate, per-peer reputation scores (if reputation is enabled).
- **Replication lag**: time between event creation (wall-clock timestamp) and local receipt, measured per event and aggregated as a histogram. This is the primary indicator of "how real-time is the near-real-time replication."

These metrics feed into the operational monitoring described in Section 11.2. Without them, operators cannot diagnose replication delays, detect misbehaving peers, or plan capacity.

Trigger for promotion to required: Production deployment. Metrics should be present from the first production instance.

7. Replication and Consistency

This section describes how events propagate, how peers stay in sync, and how the system behaves under concurrent writes and failures. It builds on the network architecture (Section 6) and adds the storage layer, the consistency semantics consumers can rely on, and the tooling decisions that govern operational behavior.

A core design goal is **deployability without specialized operations expertise**. A Creda peer must be runnable on a developer laptop for testing, on a small on-premises k8s cluster at a community hospital, or on a managed cloud k8s service at a large health system, with similar configuration and similar operational requirements across all three environments. Tooling choices throughout this section are evaluated against this goal explicitly.

7.1 Consistency Model

7.1.1 Eventual Consistency as Baseline

The network does not provide linearizability or strong consistency. Any peer's view of a patient subgraph reflects the events it has received and validated; views may differ across peers at any instant. All peers converge to the same event set over time, with convergence latency bounded by gossip propagation (typically 1-2 seconds in normal conditions across thousands of peers) and anti-entropy (15 minutes for active subgraphs in the worst case).

This is the right consistency model for the use case. Identity provenance is fundamentally additive — assertions accumulate, are linked, are contested, but the historical record never needs to be globally agreed upon at a single instant. Strong consistency would require coordination on every write, which is incompatible with the decentralized peer-to-peer architecture and the tolerable write latency for a clinical workflow.

7.1.2 Causal Consistency for Patient-Scoped Reads

Within a single patient subgraph, a peer's local view is causally consistent. If event B references event A as a parent, a peer that has B will also have A. Two mechanisms enforce this:

- **Validation order at receipt.** When a peer receives an event via gossip, it checks that all parent UUIDs are present in its local store before accepting the event. If parents are missing, the event is buffered briefly and the peer issues targeted requests to the sender (or to peers identified by the DHT) for the missing parents. The event moves from buffered to validated only when its full ancestry is locally available.
- **Logical clock ordering.** The per-subgraph Lamport-style logical clock (Section 3.5) ensures that traversal logic can reconstruct the causal order of events within a subgraph even when wall-clock timestamps are inconsistent across institutions.

Causal consistency means traversal logic never sees a "dangling" reference. Effective identity computation (Section 5.2.4) can rely on the invariant that any event in the local store has its full provenance chain locally available.

7.1.3 Read-Your-Writes Consistency

A peer that creates an event sees it immediately in subsequent local queries. The local write path bypasses gossip — the event is committed to the local store and indexed before being pushed out to other peers. This means the institution that creates an event has stronger consistency guarantees for its own writes than for events from other institutions.

For consumers querying through the FHIR API (Section 8), this manifests as: a registration system that creates an Assert event will see the resulting Patient resource in the next FHIR query, regardless of replication state. Other institutions see the new event with normal gossip latency.

7.1.4 Bounded Staleness as an Operational Metric

Replication lag — the time between event creation (per the originating institution's wall-clock timestamp) and local receipt — is measured per event and exposed as a histogram metric. Tail latency (p99 replication lag) is the primary operational signal:

- **p99 < 5 seconds**: healthy network.
- **p99 5-30 seconds**: degraded — usually a network issue, a slow peer, or capacity exhaustion.
- **p99 > 30 seconds**: significant problem — gossip is failing to converge in expected time, requires investigation.

Operators can alert on these thresholds. Bounded staleness is not a guarantee the system enforces; it is a property the system exhibits when healthy and that operators monitor to detect degradation.

7.2 Conflict Resolution

7.2.1 Acyclicity by Construction

Cycles in the patient identity DAG are structurally impossible. An event can only reference parent UUIDs that already exist (and therefore have UUIDs that predate the new event in logical-clock terms). There is no concurrent write operation that can create a cycle, so no cycle-detection logic is needed at write time, during gossip propagation, or during anti-entropy reconciliation.

This is a core simplification. Many distributed graph systems spend significant complexity on cycle prevention or detection — Creda gets it for free from the append-forward design.

7.2.2 Append-Forward Structural Mutations

Creda distinguishes two kinds of mutation:

Structural mutations change the graph topology — adding new nodes, modifying parent references, reordering events, removing nodes. **These are forbidden except for adding new nodes.** The graph is structurally append-forward: new events can be added, but existing events cannot be removed and existing parent references cannot be modified. Sending an event to another peer never invalidates that peer's existing topology.

Content mutations change the payload of an existing node. **These are permitted under regulated circumstances:**

- **Amend** events do not actually mutate the original node's stored content. The original Assert remains in the store with its original payload; the Amend is a new node that references the original and provides updated content. Traversal logic treats the amendment as superseding the original for effective identity computation, but both nodes coexist in the store. This is technically still append-forward — Amend is not a content mutation, it is a new node that supersedes another semantically.
- **Tombstone** events trigger actual content mutation in targeted nodes. When a Tombstone propagates, peers replace the payload of targeted nodes with a deletion marker, void the content hash, and set the `content_hash_voided` flag. The node's UUID, parent references, event type, timestamps, and signature *slot* remain — only the demographic content is destroyed. The graph topology is unchanged: the same nodes exist, the same edges exist, the same UUIDs are referenced. What is destroyed is the PII payload within affected nodes.

This distinction matters because it preserves the structural invariants the rest of the system relies on (causal consistency, deterministic Merkle roots over UUID sets, signature verification of references) while still complying with right-to-be-forgotten requirements. After tombstoning, signature verification on the affected nodes will fail (because the signed content is gone), but signature verification on *references to* the tombstoned nodes is unaffected (because those references are over UUIDs, not content hashes). A Link event signed last year that references a node tombstoned today remains a valid Link event — the Link's signature still verifies, the topology is intact, and the only consequence is that traversal through the tombstoned node yields no demographics.

In Git terms, this is closer to `git filter-repo` than to `git rm` — content is rewritten in place while the commit graph topology is preserved. The cost is that tombstoned nodes lose their cryptographic integrity guarantee for content; the benefit is that all references to them remain valid, no orphaning occurs, and the audit trail of "this node existed and was tombstoned on this date" is preserved.

7.2.3 Concurrent Writes

Two institutions creating events for the same patient at the same time is the normal case, not an exception. Both events are valid. If they reference a common parent, they become siblings in the DAG. If they have no common ancestor (e.g., both are Assert events at independent institutions that have never seen the patient before), they are independent roots until a future Link event connects them.

No coordination is required at write time. The system does not need locks, distributed transactions, or consensus on event ordering. Each institution writes locally and gossips outward; convergence is handled by gossip and anti-entropy.

The only semantic conflicts are concurrent Link events asserting incompatible identity relationships — for example, one institution links Patient A's subgraph to Patient B's, while another concurrently links Patient A's subgraph to Patient C's, and Patients B and C are clearly not the same person. These are not system-level conflicts. Both Link events are valid. They surface as competing assertions in the confidence model (Section 5.3.4), and consuming institutions resolve the disagreement using their own logic — typically by treating low-confidence or contested links as advisory rather than authoritative.

7.3 Storage Architecture

7.3.1 Embedded Key-Value Store per Peer

Each peer persists events in an embedded key-value store, mounted on a k8s persistent volume. Events are keyed by UUID; secondary indexes (Section 5.2.5) are maintained either directly via additional column families in the KV store or via a separate index store managed by Creda Core.

An embedded store (rather than a network-attached database) is the right choice because it eliminates a network hop on every read and write, simplifies deployment (no separate database container or managed service to provision), and aligns with the goal of laptop-deployable peers.

7.3.2 Tiered Storage

Active subgraphs live in memory caches over the embedded store. Warm subgraphs live on the persistent volume. Cold subgraphs (events from deceased patients past their retention window, or otherwise dormant) can be archived to object storage with metadata pointers retained in the hot store.

The tiering is opportunistic and access-pattern-driven, not strict. Eviction from hot to warm is driven by LRU policy in the cache. Eviction from warm to cold requires explicit triggering, typically by a scheduled retention task (Section 7.5).

7.3.3 Snapshot Generation and Retention

Each peer writes a snapshot of its event store to institution-operated object storage every 6 hours (configurable). Snapshots are append-only — each snapshot is a sorted sequence of canonical CBOR-encoded events plus a manifest with the snapshot timestamp, event count, and a Blake3 hash of the full snapshot for integrity verification.

Default retention: 7 daily snapshots and 4 weekly snapshots per institution. Older snapshots are deleted once a newer snapshot exists and all peers in the institution have caught up past it. Snapshots are not replicated across institutions — each institution's snapshots cover only events that institution's peers stored, respecting consent boundaries and minimizing cross-institutional data transfer.

7.4 Tooling Decisions

This section evaluates the tooling choices Creda faces in the storage and operational layers. Each tool is rated on three axes:

- **Kubernetes nativity (1-5)**: How naturally the tool integrates with k8s primitives. 5 = first-class k8s citizen with operators, CRDs, native scheduling. 1 = runs on k8s but treats it as a generic VM host.
- **Ease of deployment (1-5)**: How much operator skill and configuration is required. 5 = one command, sensible defaults, runs anywhere. 1 = requires dedicated platform team to operate.
- **Runs on a laptop**: yes / partial / no. Whether a developer or small institution can run the component in a development environment without a real cluster.

7.4.1 Embedded Storage Engine

Tool	K8s Nativity	Ease of Deploy	Laptop	Notes
RocksDB	5	4	yes	Battle-tested (Facebook, CockroachDB, TiKV). Embedded, no separate process. C++ via FFI from Rust. Predictable performance. **Recommended.**
sled	5	5	yes	Pure Rust, no FFI, simpler API. Less mature; some production stability concerns reported. Good for development.
SQLite	5	5	yes	Single-file storage, trivially auditable. Does not scale to millions of events efficiently. Acceptable for very small deployments or development.
redb	5	5	yes	Pure Rust, ACID transactions, embedded. Newer than sled but design is conservative. Worth re-evaluating in 12-18 months.

Recommendation: RocksDB as the default. The storage engine is abstracted behind a `Store` trait in Creda Core, so a peer can be configured to use sled or SQLite for development with a single configuration change.

7.4.2 Workflow Orchestration for Operational Tasks

This is **not** the patient identity DAG — that is persistent data, not a workflow. Workflow orchestration is needed for operational tasks: bulk import jobs from legacy MPIs, periodic snapshot rollover, scheduled anti-entropy sweeps across regions, batch FHIR export jobs, retention-window expirations for deceased patients.

Tool	K8s Nativity	Ease of Deploy	Laptop	Notes
Kubernetes CronJobs	5	5	yes	Built into k8s. No additional dependency. Best for simple scheduled tasks. Limited expressiveness — no DAG semantics, no retries beyond k8s defaults, no observability beyond pod logs. **Recommended for simple periodic tasks.**
Argo Workflows	5	3	partial	Native k8s DAG orchestrator. Excellent for multi-step pipelines (e.g., "snapshot → upload → notify peers → garbage-collect old snapshot"). Requires Argo controller installed in the cluster. Argo CRDs are k8s-native but represent additional surface area. **Recommended for multi-step operational pipelines.**
Temporal	3	2	partial	Powerful workflow engine with durable execution. Language-agnostic SDKs. Overkill for Creda's operational needs and adds a significant dependency (Temporal server cluster). Better suited for application-level business workflows than infrastructure tasks.
Apache Airflow	2	1	no	Heavyweight, Python-centric, designed for data engineering pipelines rather than infrastructure orchestration. Not k8s-native (k8s executor exists but feels bolted on). **Not recommended for Creda.**

Recommendation: Kubernetes CronJobs for simple scheduled tasks (snapshot generation, retention sweeps). Argo Workflows for multi-step pipelines (legacy MPI bulk import, cross-region anti-entropy coordination, large-scale tombstone propagation campaigns). Both are deployable as the institution's needs grow — start with CronJobs, add Argo Workflows when complexity warrants it.

Important clarification on Argo: Argo Workflows orchestrates the *execution* of operational tasks. The Creda patient identity DAG is the *data being managed*, and is implemented within Creda Core (Rust, libp2p, RocksDB) — not in Argo. Confusing these layers would be a category error. Argo Workflows could, for example, run a workflow that triggers a peer to compute a snapshot, upload it to S3, and notify other peers — but Argo does not store, replicate, or query the patient identity graph itself.

7.4.3 Deployment Packaging

Tool	K8s Nativity	Ease of Deploy	Laptop	Notes
Helm chart	5	5	yes	Industry standard. Templated YAML manifests with values files. Most institutions already operate Helm. Easy to customize for site-specific needs. **Recommended baseline.**
Kubernetes Operator	5	3	partial	Custom controller that manages Creda peer lifecycle (snapshot scheduling, certificate rotation, Participant Registry sync). More powerful than Helm for ongoing operations. Requires writing and maintaining the operator. **Recommended at scale (50+ peer institutions or for managed offerings).**
Raw manifests	5	3	yes	Plain YAML. Maximum transparency but no abstraction over environment differences. Suitable for very simple deployments or for embedding Creda into existing GitOps workflows.
Docker Compose	1	5	yes	Not k8s-native at all. Useful for laptop development and demo deployments. Should be provided alongside k8s manifests but not as the primary deployment mode.

Recommendation: Provide a Helm chart as the primary deployment artifact, with a Docker Compose file for laptop development. A Kubernetes Operator should be developed once Creda has more than ~20 production deployments and patterns of operational toil emerge that the operator can automate.

7.4.4 Object Storage for Snapshots

Tool	K8s Nativity	Ease of Deploy	Laptop	Notes
Cloud-managed S3-compatible (AWS S3, GCS, Azure Blob via S3 gateway)	4	5	partial (requires cloud credentials)	Standard for cloud deployments. Institutions typically already operate this. **Recommended for cloud deployments.**
MinIO	5	4	yes	S3-compatible, deployable in-cluster as a StatefulSet. Pure-Go, no external dependencies. **Recommended for on-prem deployments.**
Local filesystem (PersistentVolume)	5	5	yes	Simplest. Snapshots written to a mounted volume. No external service. Acceptable for development and very small deployments. Loses the durability and cross-zone properties of object storage.
Ceph / Rook	4	2	no	Powerful but operationally complex. Worth considering only if the institution already operates Ceph for other workloads.

Recommendation: Configurable per institution. The snapshot interface in Creda Core abstracts over an S3-compatible API; institutions plug in their preferred backend. Default: MinIO bundled in the Helm chart for self-contained on-prem deployments, with cloud S3-compatible storage as a one-line configuration override.

7.4.5 Observability Stack

Tool	K8s Nativity	Ease of Deploy	Laptop	Notes
Prometheus + Grafana	5	4	yes	Industry standard. Prometheus scrapes Creda peer metrics endpoints; Grafana provides dashboards. Helm charts available. **Recommended baseline.**
OpenTelemetry	5	3	partial	Newer, more flexible (metrics + traces + logs unified). Requires a collector deployment. Worth adopting for tracing across peer-to-peer calls. **Recommended as a complement to Prometheus, not a replacement.**
Datadog / New Relic / commercial APM	3	4	no	Closed-source, vendor lock-in, monthly cost. Some institutions standardize on these for organizational reasons. Should be supported via OTLP export but not as the primary path.
Kubernetes-native logging only (kubectl logs)	5	5	yes	Minimal. Acceptable for development; insufficient for production.

Recommendation: Ship Prometheus metric endpoints and OpenTelemetry trace export from Creda peers. Include a default Grafana dashboard set in the Helm chart. Institutions can route the metrics and traces to whatever backend they already operate.

7.4.6 Identity and Certificate Management

Tool	K8s Nativity	Ease of Deploy	Laptop	Notes
SPIRE (SPIFFE Runtime Environment)	5	3	partial	The reference SPIFFE implementation. k8s-native via the SPIRE Kubernetes Workload Registrar. Required for the SPIFFE ID component of peer identity (Section 6.1.1). **Recommended.**
cert-manager	5	5	yes	Automated certificate issuance and rotation in k8s. Handles UDAP certificate lifecycle if the institution's UDAP CA exposes ACME or a supported issuer. **Recommended for UDAP cert management.**
Istio / Linkerd service mesh	4	2	partial	Provide mTLS and identity but at the cost of significant operational complexity. Overkill for Creda — peer-to-peer communication uses libp2p's Noise transport, not k8s service mesh. **Not recommended.**
Manual certificate management	5	1	yes	Provisioning UDAP certs and SPIFFE IDs manually. Acceptable only for very small deployments or development.

Recommendation: SPIRE for SPIFFE ID issuance, cert-manager for UDAP certificate rotation. Both are deployable via Helm and have established operational patterns.

7.4.7 Tooling Summary

The default Creda deployment stack, optimized for "deployable with little to no oversight":

- **Storage**: RocksDB embedded, mounted on a k8s PersistentVolume.
- **Operational orchestration**: Kubernetes CronJobs for simple tasks; Argo Workflows added when multi-step pipelines emerge.
- **Packaging**: Helm chart (primary); Docker Compose (laptop development); Kubernetes Operator (deferred).
- **Object storage for snapshots**: MinIO bundled by default; cloud S3-compatible via configuration override.
- **Observability**: Prometheus + Grafana + OpenTelemetry.
- **Identity**: SPIRE for SPIFFE, cert-manager for UDAP.

This stack runs on:

- **A laptop**: Docker Compose with RocksDB + local filesystem snapshots + minimal Prometheus. Used for development and integration testing.
- **A small on-premises k8s cluster**: Helm chart with bundled MinIO. Used by community hospitals and small HIEs.
- **A managed cloud k8s service**: Helm chart with cloud S3-compatible storage and the institution's existing Prometheus/Grafana. Used by large health systems and HIEs.

The same Helm chart and the same Creda peer container image work in all three environments — only configuration values change.

7.5 Retention and Lifecycle Tasks

Several scheduled tasks operate on the local event store independently of network replication:

- **Snapshot generation**: every 6 hours, write a snapshot to object storage. Implemented as a Kubernetes CronJob.
- **Snapshot retention**: daily, prune snapshots older than the retention policy (default 7 daily + 4 weekly).
- **Cold tier eviction**: weekly, identify subgraphs eligible for archival (deceased patients past retention window, dormant subgraphs with no activity in N years) and move them to cold storage.
- **Index compaction**: monthly, run RocksDB compaction on indexes to reclaim space and improve query performance.
- **Reputation decay**: hourly, apply temporal decay to peer reputation scores so that misbehavior in the distant past does not permanently penalize a peer that has since reformed.

These tasks are deployed alongside the Creda peer in the same Helm chart. Each is a small, idempotent CronJob that can run on any peer in an institution — they do not require a designated leader.

8. FHIR Integration

Creda's identity model is intentionally aligned with FHIR's existing primitives. The patient identity DAG is a natural fit for FHIR Provenance, the effective identity projection maps to FHIR Patient, and the read/write operations align with established FHIR REST patterns and operations. The result is a system that extends the FHIR ecosystem rather than

replacing it: a non-Creda consumer sees normal FHIR resources with optional extensions they can ignore, while a Creda-aware consumer gains access to the full provenance graph and the trust signals derived from it.

This section defines the Creda FHIR Implementation Guide (IG) at a level sufficient for the engineering team. The full IG — including FSH source files, ImplementationGuide resource, generated profiles, examples, and conformance tests — is a separate deliverable maintained alongside the Creda Core source tree.

8.1 Patient Resource Mapping

8.1.1 Patient as Projection, Not Record

A FHIR `Patient` resource returned by Creda is **computed**, not stored. Each query yields a Patient projected from the effective identity computation (Section 5.2.4) over the events visible to the responding peer. Two consequences follow:

- The same `Patient/[id]` query on two peers may return slightly different resources if the peers have different subsets of the patient's subgraph. This is by design — the system does not pretend to offer a single universal truth.
- The Patient's `meta.lastUpdated` reflects the wall-clock timestamp of the most recent event in the subgraph visible to the responding peer, not a database row update time.

The `Patient.id` is an **opaque, randomly-generated UUID** assigned by the responding peer when the patient is first projected and stable thereafter at that peer. This follows conventional FHIR usage — IDs are opaque tokens that consumers should not parse or derive meaning from. Stable Patient.id values at the provider also support local caching, bookmarking, and integration with downstream systems that retain references.

The Creda subgraph identifier — the deterministic root set hash — is exposed via the `Patient.identifier` slicing described in Section 8.1.2, not as `Patient.id`. This separation is important: `Patient.id` is a local handle managed by the responding peer for its consumers, while the subgraph identifier is the global, deterministic key that all Creda-aware peers compute identically from the same subgraph. Two peers serving the same patient will assign different `Patient.id` values but will produce the same subgraph identifier, allowing Creda-aware consumers to recognize the same underlying identity across peers.

When new events expand the root set (e.g., a Link event connects previously-independent roots), the subgraph identifier changes accordingly and the prior identifier is retained as a historical identifier slice. The `Patient.id` itself does not change — it remains stable at the responding peer regardless of subgraph evolution.

8.1.2 Identifier Slicing

`Patient.identifier` is sliced by `identifier.system` to surface the multiple identifier sources Creda aggregates:

- **Slice: institutional MRN.** One slice per asserting institution, with `system` set to the institution's MRN namespace (typically `urn:oid:[institution-OID]`). Each slice carries the MRN and a reference to the asserting Assert event via the `provenance` extension on the identifier.
- **Slice: payer member ID.** One slice per payer, with `system` set to the payer's identifier namespace. Carries member IDs from insurance card presentations.
- **Slice: Creda subgraph identifier.** A single slice with `system` set to `http://creda.health/identifier/subgraph` and `value` set to the deterministic root set hash. This is the global, peer-independent identifier that all Creda-aware peers compute identically from the same subgraph. Creda-aware consumers use this for cross-peer identity resolution and for following the underlying provenance graph.
- **Slice: historical Creda subgraph identifiers.** Zero or more slices with `system` set to `http://creda.health/identifier/subgraph-historical`, carrying prior subgraph identifier hashes from before Link events expanded the root set. Allows consumers to recognize a patient under their previously-known subgraph identifier even after the subgraph has merged with others.

Non-Creda consumers see this as a normal Patient with multiple identifiers from multiple systems — the standard FHIR pattern for cross-institutional identifiers. Creda-aware consumers can follow the subgraph identifier into the full provenance graph.

8.1.3 Per-Field Confidence Extensions

Confidence scores from the Section 5.3 model attach to demographic elements via a custom extension:

```
http://creda.health/StructureDefinition/field-confidence
```

The extension contains:

- `confidence` (integer, 0-10000): the computed confidence score for this field.
- `inputs` (BackboneElement, repeating): the factors that produced the score — `verificationMethod`, `attestationCount`, `independentInstitutionCount`, `decayFactor`, `agreementWeight`.
- `assertingEvents` (Reference to Provenance, repeating): the events that contribute to this field's value, allowing a consumer to drill into the underlying assertions.

The extension can attach to any individual demographic element — `Patient.name`, `Patient.birthDate`, `Patient.address`, `Patient.identifier`, etc. Consumers that don't understand the extension see normal demographic values; Creda-aware consumers see the trust signals.

8.1.4 Disagreement Representation

When demographics conflict across the subgraph (Section 5.3.4), the projected Patient resource includes all asserted values rather than silently picking one. Disagreement is represented two ways:

- **Primary value selection.** The "primary" value for each field is selected per a configurable policy: highest confidence (default), most-recent assertion, most-attested assertion, or institution-specific custom logic. The primary value occupies the standard FHIR element (e.g., `Patient.birthDate`).
- **Alternate values via extension.** Other asserted values are surfaced via a `http://creda.health/StructureDefinition/disputed-value` extension on the same element. Each alternate carries its own value, confidence score, and reference to the asserting event. A `disputed` flag on the parent element signals to consumers that the field has competing assertions.

Consuming institutions are free to ignore the disagreement and use the primary value, or to surface the dispute to clinicians and registrars for manual resolution. Creda's role is to make the disagreement visible — it does not adjudicate.

8.1.5 Effective Identity vs. Raw Subgraph Query Modes

Creda supports two query modes for Patient retrieval, selectable via a query parameter:

- **`_creda-mode=projection`** (default). Returns the computed effective identity as a Patient resource. This is the mode every FHIR consumer uses by default and matches standard FHIR semantics.
- **`_creda-mode=subgraph`**. Returns a Bundle containing the Patient projection plus all Provenance resources representing the events in the subgraph. Useful for Creda-aware consumers that want the full evidentiary chain in a single round trip rather than following links.

For deeper inspection, `Patient/[id]/$creda-provenance` (Section 8.2.5) provides a richer interface than the subgraph query mode.

8.2 FHIR Implementation Guide

The Creda FHIR IG is published at `http://creda.health/fhir/ig/v1` and follows standard HL7 IG conventions. It conforms to FHIR R4 (with R5 conformance planned for v1.1 once US Core publishes its R5 baseline).

8.2.1 US Core Conformance

CredaPatient (Section 8.2.2) conforms to the US Core Patient profile. Every CredaPatient is a valid US Core Patient — the Creda profile adds extensions and slicing constraints but does not loosen any US Core requirements. This is essential for adoption: institutions that already meet US Core requirements (which is most of the US ecosystem post-ONC certification) can produce CredaPatient resources without changing their core data model.

The profile chain is: `Patient (FHIR R4 base) → US Core Patient → CredaPatient`. Other Creda profiles similarly conform to US Core baselines where they exist (US Core Provenance, US Core Consent).

8.2.2 Profile: CredaPatient

CredaPatient constrains FHIR Patient with:

- **Required extensions:** subgraph identifier, root set, last-modified-event UUID. These are `mustSupport` — a Creda-aware consumer that does not handle these extensions cannot correctly interact with CredaPatient. A non-Creda-aware consumer ignoring the extensions still sees a valid US Core Patient.
- **Optional extensions:** per-field confidence, disagreement flags, deceased declaration provenance reference. These are `mustSupport` only for the producing peer (the Bridge must populate them when applicable) but consumers may ignore them without compromising basic clinical use.
- **Required slicing on `Patient.identifier`:** the Creda subgraph identifier slice is `mustSupport`. Other slices are optional based on data availability.

The decision to make subgraph identifier `mustSupport` while making per-field confidence optional reflects a deliberate design judgment: the subgraph identifier is structural (without it, a Creda-aware consumer cannot navigate to provenance), while confidence is informational (a consumer that ignores it gets a valid Patient, just without the trust signals).

8.2.3 Profile: CredaProvenance

Each Creda identity event maps to a CredaProvenance resource. The mapping is direct because FHIR Provenance is already designed for recording who did what, when, on what evidence:

Creda Event Field	FHIR Provenance Element
Event UUID	`Provenance.id` (URN form)
Event type	`Provenance.activity` (custom CodeableConcept)
Parent UUIDs	`Provenance.entity[].what` (each parent as a derivation entity)
Institution ID (UDAP fingerprint)	`Provenance.agent.who` (Reference to Organization)
Wall-clock timestamp	`Provenance.recorded`
Logical clock	extension `http://creda.health/StructureDefinition/logical-clock`
Signature	extension `http://creda.health/StructureDefinition/event-signature`
Verification method (Assert)	`Provenance.signature.type`
Payload (event-specific)	extension `http://creda.health/StructureDefinition/event-payload`

A consumer that understands FHIR Provenance but not Creda gets a meaningful audit-style record of identity events. A Creda-aware consumer can additionally inspect the payload extension and follow the entity references to walk the full DAG.

CredaProvenance conforms to US Core Provenance where the US Core profile applies.

8.2.4 Provenance vs. AuditEvent

Creda events are conceptually closer to **Provenance** than to **AuditEvent**, even though both FHIR resources record actions. The distinction matters because the engineering team will face this question repeatedly:

- **Provenance** records the source of a fact — "this Patient's birth date came from this Assert event by this institution on this date." Creda events are constitutive of the Patient: they ARE the source. Without the events, there is no Patient.
- **AuditEvent** records the access or modification history of a resource — "this user queried this Patient at this time." Creda also generates AuditEvents, but for read-side activity tracking (who queried which subgraph), not for identity events themselves.

The split: identity events are CredaProvenance resources. Reads, queries, and access checks generate FHIR AuditEvent resources via the standard HAPI auditing infrastructure, stored separately from the identity DAG. Consumers looking for "what happened to this patient's identity" query Provenance. Consumers looking for "who looked at this patient" query AuditEvent.

8.2.5 Operation: $creda-provenance

```
GET Patient/[id]/$creda-provenance
GET Patient/[id]/$creda-provenance?event-type=Link&since=2025-01-01
```

Returns a Bundle of CredaProvenance resources representing the full provenance graph for the patient, optionally filtered by event type, date range, asserting institution, or depth from leaf nodes. This is the primary interop surface for non-Creda consumers who want to inspect the chain via standard FHIR rather than speaking the underlying gRPC protocol.

The Bundle includes a `Bundle.link` of type `next` for pagination when the subgraph is large. Bundle entries are sorted by logical clock to produce a causally-coherent traversal order.

8.2.6 Operation: $creda-attest

```
POST Patient/[id]/$creda-attest
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "purpose", "valueCode": "treatment"},
    {"name": "targetEvents", "valueReference": {"reference": "Provenance/[uuid]"}}
  ]
}
```

Records an Attest event on the patient's identity chain. This is the FHIR-side write interface for attestations — saves consumers from having to implement Creda's native event-creation API for what will be one of the most common write cases (every clinical encounter that relies on the identity should attest).

The operation returns the newly-created CredaProvenance resource representing the Attest event.

8.2.7 Operation: $creda-link and $creda-contest

```
POST Patient/[id]/$creda-link
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "target", "valueReference": {"reference": "Patient/[other-id]"}},
    {"name": "confidence", "valueInteger": 9200},
    {"name": "method", "valueCode": "manual"}
  ]
}
```

Creates a Link event between two Patient subgraphs. The institution invoking this operation must be party to one of the linked subgraphs (Section 3.4.3 enforcement is performed by Creda Core, not the Bridge).

```
POST Provenance/[id]/$creda-contest
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "reason", "valueCode": "demographic-conflict"},
    {"name": "narrative", "valueString": "Manual review identified distinct patients with similar demographics."}
  ]
}
```

Creates a Contest event against a specific Link Provenance. Same party-of-the-subgraphs constraint applies.

8.2.8 Operation: $creda-tombstone

```
POST Patient/[id]/$creda-tombstone
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "legalBasis", "valueCode": "right-to-be-forgotten"},
    {"name": "targetEvents", "valueReference": {"reference": "Provenance/[uuid]"}}
  ]
}
```

Triggers a Tombstone event after the institution has validated the underlying right-to-be-forgotten request. The institution is responsible for the legal validation; Creda enforces only the structural and signature requirements. The `legalBasis` parameter maps to the Tombstone payload's `legal_basis` field.

This is one of the more sensitive operations and requires elevated authorization (typically a privacy officer's credentials, mediated by the institution's existing access control), enforced at the institution's FHIR endpoint before reaching the Bridge.

8.2.9 Operation: $creda-authorize, $creda-revoke, $creda-verify, $creda-export

These four operations expose the portable authorization layer (Section 4) through FHIR. They operate on the FathomAuthorization-equivalent profile (CredaAuthorization, based on FHIR Consent) and are the FHIR-side surface for the authorization event types.

```
POST Patient/[id]/$creda-authorize
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "audience", "valueString": "any-tefca-qhin"},
    {"name": "purpose", "valueCode": "treatment"},
    {"name": "scope", "valueString": "full-subgraph"},
    {"name": "useMode", "valueCode": "read-and-rely"},
    {"name": "expiration", "valueDateTime": "2027-05-11"}
  ]
}
```

`$creda-authorize` creates an AuthorizationGrant. `$creda-revoke` creates an AuthorizationRevocation referencing a prior Grant. `$creda-verify` runs the authorization evaluation algorithm (Section 4.6) for a given requesting institution and returns an authorization decision plus the governing Grant — this is the FHIR-accessible form of the Verifier's check. `$creda-export` records an ExportReceipt when data is released under a Grant and is typically invoked by the Export Gate (Section 10.2) rather than directly by a clinical user.

`$creda-verify` returns a Parameters resource with the decision (`authorized` / `denied` / `denied-revoked` / `denied-expired` / `denied-out-of-scope`), the governing Grant reference, and — when authorized — the scope and use-mode the Grant permits. Because the Verifier operates locally (Section 10.3.3), `$creda-verify` can be served from stale state during a partition, in which case the response includes the age of the responding peer's DAG view.

These operations make portable authorization usable by FHIR consumers that do not embed the native Verifier SDK — a payer adjudication system or an EHR can `POST $creda-verify` to a Creda Bridge and act on the decision, rather than linking the Rust Verifier library directly.

8.2.10 Operation: $creda-disambiguate

The standard FHIR `Patient/$match` operation returns scored candidates but does not support interactive verification. When `$match` returns multiple candidates with similar scores — a common situation at front-desk registration when two patients have similar demographics — the registrar today resorts to ad-hoc, unaudited out-of-band questions ("when was your last visit?", "what's your insurance member ID?") drawn from whatever their local system happens to show.

Creda's provenance chains contain rich, signed history that is uniquely suited to provenance-grounded disambiguation. The `$creda-disambiguate` operation formalizes this: given a set of ambiguous candidates, the operation returns differentiating questions whose answers exist in the candidates' provenance chains but differ between them. The patient's answers are scored against each candidate's chain, producing a refined match with significantly higher confidence. Critically, the verification itself is recorded as an Attest event in the chain, creating durable provenance for the manual disambiguation.

Direct patient verification is the preferred mechanism. When the patient holds their own Creda signing key (the patient-as-participant model from Section 3.1), the patient signs a self-verification event directly — confirming their own identity from a phone or other authenticated client without a registrar intermediary. Registrar-mediated challenge questions via this operation are the fallback for patients without their own key, which today is most patients but should diminish over time as patient-side keys become more available.

8.2.10.1 Operation Flow

The disambiguation flow has three stages:

Stage 1: Request questions.

```
POST Patient/$creda-disambiguate
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "operation", "valueCode": "request-questions"},
    {"name": "candidate", "valueReference": {"reference": "Patient/[id-1]"}},
    {"name": "candidate", "valueReference": {"reference": "Patient/[id-2]"}},
    {"name": "candidate", "valueReference": {"reference": "Patient/[id-3]"}},
    {"name": "registrarContext", "valueCode": "front-desk-registration"}
  ]
}
```

The Bridge invokes Creda Core, which inspects the candidates' subgraphs and selects questions designed to disambiguate. The response is a Parameters resource containing a question set:

```
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "sessionId", "valueString": "[opaque-session-token]"},
    {"name": "question", "part": [
      {"name": "questionId", "valueString": "q1"},
      {"name": "questionType", "valueCode": "multiple-choice"},
      {"name": "prompt", "valueString": "Which of these institutions have you visited?"},
      {"name": "option", "valueString": "Mercy General Hospital"},
      {"name": "option", "valueString": "St. Luke's Medical Center"},
      {"name": "option", "valueString": "Memorial Regional"},
      {"name": "option", "valueString": "None of these"}
    ]},
    {"name": "question", "part": [
      {"name": "questionId", "valueString": "q2"},
      {"name": "questionType", "valueCode": "temporal-range"},
      {"name": "prompt", "valueString": "When did you last visit a healthcare provider?"},
      {"name": "option", "valueString": "Within the past 3 months"},
      {"name": "option", "valueString": "3-12 months ago"},
      {"name": "option", "valueString": "More than a year ago"},
      {"name": "option", "valueString": "I don't recall"}
    ]}
  ]
}
```

Stage 2: Submit answers.

```
POST Patient/$creda-disambiguate
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "operation", "valueCode": "submit-answers"},
    {"name": "sessionId", "valueString": "[opaque-session-token]"},
    {"name": "answer", "part": [
      {"name": "questionId", "valueString": "q1"},
      {"name": "selectedOption", "valueString": "Mercy General Hospital"}
    ]},
    {"name": "answer", "part": [
      {"name": "questionId", "valueString": "q2"},
      {"name": "selectedOption", "valueString": "Within the past 3 months"}
    ]}
  ]
}
```

Stage 3: Receive refined match and recorded provenance.

```
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "match", "part": [
      {"name": "patient", "valueReference": {"reference": "Patient/[id-1]"}},
      {"name": "score", "valueDecimal": 0.97},
      {"name": "verificationMethod", "valueCode": "manual-disambiguation"},
      {"name": "attestEvent", "valueReference": {"reference": "Provenance/[uuid]"}}
    ]}
  ]
}
```

The refined match includes a reference to the newly-created Attest event, which carries verification method `manual-disambiguation`, the questions asked, the answers given (or a hash of them — see below), and the registrar's institutional identity as the asserter. The operation returns a single match if confidence exceeds a configurable threshold; otherwise it returns no match (rather than a low-confidence guess) and the registrar must escalate to a different verification path.

8.2.10.2 Question Selection

Question selection is performed by Creda Core based on the candidates' subgraphs. The selection algorithm:

1. Identify facts in each candidate's subgraph that are signed by trusted institutions and recent enough to be memorable.
2. Find facts that **differ** between candidates — facts where each candidate has a different answer or where one candidate has the fact and others don't.

3. Filter for cognitive accessibility: temporal ranges over exact dates, institution names over MRN numbers, broad categories over precise values.
4. Prefer questions whose answers are stable (institution names, payer names) over volatile ones (specific addresses that may have changed).
5. Generate a question set of 2-4 questions, balancing disambiguation power against patient burden.

The algorithm avoids questions that would leak PHI to the registrar regardless of patient response. Multiple-choice questions include plausible distractors drawn from common institutions or payers in the region, not from other candidates' actual answers — using another candidate's answer as a distractor leaks that candidate's PHI.

8.2.10.3 Privacy and Anti-Phishing Protections

The operation has several built-in protections against misuse:

- **Authenticated registrar context.** The operation can only be invoked under an authenticated institutional identity. Anonymous or weakly-authenticated invocations are rejected. This ties every disambiguation attempt to a specific registrar at a specific institution.
- **Per-session rate limiting.** A single registrar session can invoke `request-questions` a bounded number of times for the same candidate set within a window. Repeated invocations against the same candidates are flagged for audit review — this pattern is consistent with phishing.
- **Per-candidate rate limiting.** A given Patient/[id] can be the subject of disambiguation requests at a bounded frequency across the network. Aggressive querying of a single candidate from multiple registrars in a short window is flagged.
- **Answer hashing in audit trail.** The Attest event records a hash of the question-answer pairs rather than the cleartext answers. This preserves audit integrity (the hash proves the patient answered consistently with the chain) without storing the answers themselves in the provenance graph.
- **Patient notification.** Optionally, the patient can be notified out-of-band (via their patient portal, an SMS to a registered number, or via the IAS interface in Section 8.4.4) that a disambiguation was performed against their identity. This gives patients the ability to detect misuse.
- **Failed-disambiguation logging.** When the patient's answers do not match any candidate, this is logged as a security event, not silently discarded. Repeated failed disambiguations against the same candidates from the same registrar pattern indicate possible abuse.

8.2.10.4 Patient-Direct Verification (Preferred)

When the patient holds their own Creda signing key, registrar-mediated questions are unnecessary. Instead, the patient performs self-verification directly:

```
POST Patient/$creda-self-verify
{
  "resourceType": "Parameters",
  "parameter": [
    {"name": "candidate", "valueReference": {"reference": "Patient/[id]"}},
    {"name": "patientSignature", "valueAttachment": {
      "contentType": "application/cbor",
      "data": "[base64-encoded-signed-verification-event]"
    }}
  ]
}
```

The patient signs an event asserting "I am the subject of this subgraph" using their own key (typically held in a patient-facing application authenticated via OAuth2/OIDC). Creda Core verifies the signature against the patient's registered public key, and on success creates an Attest event with verification method `patient-self-attestation`. This is structurally stronger than registrar-mediated challenge questions: the patient has cryptographically proven possession of their key, and no one else needs to be in the loop.

The two paths are complementary:

- **Patient-self-verification**: highest confidence, lowest friction once patient keys are established. Should be the default whenever the patient has a registered key.
- **Registrar-mediated disambiguation via `$creda-disambiguate`**: fallback for patients without their own key. Should diminish over time as patient-side key infrastructure matures.

The IG includes both operations and a CapabilityStatement element indicating which the peer supports. New deployments should support both from day one — patient-self-verification for forward-looking workflows, registrar-mediated disambiguation for the long tail of patients without keys.

8.2.11 SearchParameter: identity-token

```
GET Patient?_creda-token=[tokenized-value]
```

A custom SearchParameter that allows querying by demographic tokens — the same tokens used for DHT keys (Section 6.1.6). Enables matching workflows where an institution computes tokens from registration demographics and queries for any patients whose subgraphs match those tokens.

The SearchParameter is defined in the IG with token format and tokenization scheme documented in Section 9.2 (privacy and tokenization). Multiple tokens can be combined with standard FHIR search composition: `?_creda-token=[name-token]&_creda-token=[dob-token]`.

8.2.12 CapabilityStatement

Each Creda peer's HAPI FHIR Bridge advertises its Creda capabilities via the standard `CapabilityStatement` resource at `metadata`:

- **Implements**: the Creda IG (via `CapabilityStatement.implementationGuide`).
- **Profiles**: CredaPatient, CredaProvenance, CredaAuthorization (FHIR Consent base; Section 9.3 and Section 4).
- **Operations**: `$creda-provenance`, `$creda-attest`, `$creda-link`, `$creda-contest`, `$creda-tombstone`, `$creda-disambiguate`, `$creda-self-verify`, `$creda-authorize`, `$creda-revoke`, `$creda-verify`, `$creda-export`.
- **Extensions**: subgraph identifier, root set, per-field confidence, disagreement flag, etc.
- **Search parameters**: `_creda-token`, plus standard FHIR Patient search parameters.

A consumer probes a FHIR endpoint with `GET metadata` to determine if it speaks Creda before attempting Creda-specific operations, exactly as the existing FHIR ecosystem handles capability discovery.

8.2.13 Subscription Support

HAPI's FHIR Subscription mechanism is mapped to Creda's gossip topic subscriptions. A FHIR client creates a Subscription resource targeting `Patient/[id]` or a search criterion; the Bridge translates this into a Creda gossip subscription for the relevant topic bucket. When matching events arrive via gossip, the Bridge generates Subscription notifications as FHIR Bundles delivered to the subscriber's notification endpoint.

This gives FHIR clients a standard interface for real-time updates without needing to understand gossipsub or subscribe directly to libp2p topics. The translation is unidirectional — Subscriptions support reads, not writes.

8.2.14 Bulk Data Export

FHIR Bulk Data ($export) is increasingly important for population health, payment, and operations workflows. Creda's event log is a natural fit for bulk export — events are timestamped, signed, and chronologically ordered.

Creda supports `$export` at three levels:

- **System-level (`/$export`)**: not supported. Exporting all events for all patients across the network would violate consent and BAA scoping. Institutions wanting a full event dump for their own patients use the institutional snapshot mechanism (Section 7.3.3) instead.
- **Patient-level (`Patient/[id]/$export`)**: returns the patient's projected resources (Patient, related CredaProvenance, related Consent). Subject to consent enforcement — only resources the requesting institution is authorized to see are included.
- **Group-level (`Group/[id]/$export`)**: returns resources for all patients in a defined Group, useful for cohort export. Same consent enforcement applies per-patient.

Exports return data in the standard FHIR Bulk Data NDJSON format. Because Creda's events are signed, the exported CredaProvenance resources retain their signatures, and a downstream consumer can verify the cryptographic integrity of the exported data without trusting the exporter — a property that standard FHIR Bulk Data lacks.

8.3 HAPI FHIR Bridge Architecture

8.3.1 Process Boundary

The HAPI FHIR Bridge runs as a separate process from Creda Core. Both run in the same k8s pod and communicate via gRPC over a Unix domain socket within the pod's filesystem.

- **HAPI FHIR Bridge**: Java/Kotlin, builds on HAPI FHIR R4. Handles all FHIR REST routing, validation, capability advertisement, subscription management, bulk data, and resource serialization.
- **Creda Core**: Rust. Handles DAG operations, signature verification, gossip, DHT, anti-entropy, and storage.

The split gives us the maturity and ecosystem of HAPI FHIR (the de facto standard for FHIR servers) for the FHIR layer, and the performance and memory safety of Rust for the network and storage layers, without compromising either. The Unix socket avoids network overhead for the in-pod RPC and aligns with k8s sidecar patterns.

8.3.2 Bridge as Translator, Not Reasoner

The Bridge's job is purely translation: incoming FHIR requests map to Creda Core RPCs, and Creda Core responses map to FHIR resources. All identity logic — confidence computation, traversal, signature verification, conflict detection — happens in Creda Core. The Bridge has no business logic of its own beyond the mapping.

This separation is enforced by Creda Core's gRPC interface, which exposes operations like `GetEffectiveIdentity(subgraph_id) → ProjectedIdentity`, `GetSubgraph(subgraph_id, depth) → Vec<Event>`, `CreateAssert(payload) → Event`, etc. The Bridge translates these to FHIR but does not implement them.

The benefit is operational: the Bridge can be replaced or supplemented (e.g., a different FHIR server implementation, or a non-FHIR API like SMART on FHIR backend services) without re-implementing identity logic.

8.3.3 HAPI Plain Server, Not JPA

HAPI FHIR has multiple deployment modes. Creda uses **Plain Server** mode where resource providers are custom Java classes that delegate to Creda Core. The HAPI **JPA Server** mode (which stores FHIR resources in a relational database) is explicitly **NOT used**.

The reason: Creda's source of truth is the event store managed by Creda Core, not a parallel relational schema. Using HAPI JPA would introduce a second persistence layer that has to be kept in sync with the event store, doubling complexity and creating consistency hazards. Plain Server with custom resource providers has a longer initial implementation cost (writing the providers) but a much cleaner long-term architecture.

The Bridge's resource providers handle the FHIR REST verbs:

- `read` and `vread`: project an effective identity from the event store via Creda Core RPC.
- `search`: translate FHIR search parameters into Creda Core queries.
- `create` and `update`: route through the appropriate Creda event-creation operation. Direct PUT/POST to `Patient` is rejected — Patient is a projection, not a resource; clients must use `$creda-link`, `$creda-attest`, etc.
- `delete` on Patient: rejected. Patient deletion is a `$creda-tombstone` operation.
- Resource history (`_history`): returns the chronological sequence of Provenance resources for the patient subgraph.

8.4 TEFCA / QHIN Interoperability

8.4.1 QHIN as a Creda Peer

The most natural integration path: a QHIN runs a Creda peer and uses it for identity resolution. The QHIN's existing FHIR `Patient/$match` endpoint queries Creda internally — the QHIN becomes a Creda-aware resolver while presenting its standard QHIN interface to participants.

Non-Creda QHIN participants (covered entities relying on the QHIN) see no change. They call the QHIN's endpoints exactly as before. The QHIN now has access to richer identity provenance and improved match accuracy via the Creda network, which translates into better $match results and fewer ambiguous responses, but the wire protocol is unchanged.

This is the recommended adoption path: institutions don't have to run Creda peers themselves to benefit from the network — their QHIN can be the entry point.

8.4.2 QHIN-to-QHIN Identity Exchange

Today, TEFCA's QHIN-to-QHIN exchange relies on each QHIN performing its own matching against incoming queries. With Creda, two QHINs that both participate exchange identity provenance directly via the peer-to-peer network. A patient query that crosses QHIN boundaries no longer requires a chain of $match calls between QHINs — both QHINs can independently navigate the same shared subgraph.

This reduces inter-QHIN query volume, improves latency, and provides both sides with full provenance for the matched identity rather than an opaque match score.

8.4.3 Backward Compatibility for Non-Creda Participants

A covered entity that does not run Creda but receives a CredaPatient resource (via a QHIN that does) sees a normal FHIR Patient with extra extensions they can ignore. Standard FHIR processing rules apply: unknown extensions are preserved if the consumer round-trips the resource, ignored if the consumer doesn't understand them, but never cause the resource to fail validation against base Patient or US Core Patient.

Creda is **additive**, not invasive. The IG is designed so that every Creda extension is optional from the consumer's perspective and the underlying Patient remains fully usable for clinical workflows even when extensions are stripped.

8.4.4 Patient Access via Individual Access Services

TEFCA's Individual Access Services (IAS) lets patients access their own data across networks. With Creda, IAS calls can return a patient's full identity provenance chain — the patient sees who has asserted what about them, which institutions have linked their records, and any contestations or amendments in their history.

This aligns with 21st Century Cures Act information blocking rules and provides patient-side transparency that does not exist in today's MPI-based system. Patients can identify incorrect links, request tombstoning of erroneous data, and verify their identity is accurate across institutions.

The IAS interface for Creda-aware patients includes:

- `Patient/[id]/$creda-provenance`: the full provenance chain.
- `Patient/[id]/$creda-tombstone`: patient-initiated right-to-be-forgotten requests, subject to the patient's authentication and the institution's privacy officer review.
- `Patient/[id]/$creda-contest`: patient-initiated link contestations when the patient identifies incorrect cross-institutional links.

These patient-facing operations require patient authentication via OAuth2/OIDC (as IAS already specifies) and are gated by the patient's own consent — patients can always access and act on their own identity chain.

9. Security and Access Control

9.1 Institutional Identity and Authentication

9.1.1 Threat Model

The Creda security architecture is designed against an explicit threat model. Each threat is enumerated here with the section that addresses its mitigations.

Threat	Description	Primary Mitigations
Malicious peer	An attacker obtains stolen credentials and operates a peer impersonating a legitimate institution	Two-credential binding (Section 9.1.4), peer reputation (Section 6.4.1), Participant Registry revocation (Section 6.1.3)
Compromised institution	A legitimate institution's credentials are used for malicious purposes (insider threat, credential theft)	Zero trust controls (Section 9.1.7), continuous verification, audit-driven detection (Section 9.4)
Curious peer	A peer follows protocol but passively observes network traffic and DHT queries to learn about patients	Demographic tokenization (Section 9.2), consent enforcement at responding peer (Section 9.3), planned PSI/oblivious DHT (Section 9.5)
Network observer	A non-participant on the network path observes peer-to-peer traffic	Noise transport encryption (Section 6.2.3), end-to-end encryption of payloads
Regulatory adversary	A subpoena or court order seeks data that should not be lawfully discoverable	Data minimization (Section 9.2), tokenization (cleartext PHI is not at most peers), institution-scoped consent (Section 9.3)
Quantum adversary	"Harvest now, decrypt later" — captures signed events today, attempts forgery once quantum computers break classical signatures	Algorithm-agile signatures and PQC migration path (Section 5.1.2)
Distributed denial of service	Attacker overwhelms peers with malformed requests, gossip floods, or DHT query storms	Rate limiting (Section 6.2.2), reputation-based load shedding (Section 9.1.8), structural validation cost-shifting

9.1.2 UDAP Certificate as Institutional Identity Anchor

Every Creda institution authenticates with a UDAP-issued X.509 certificate. UDAP (Unified Data Access Profiles) is the existing US Health IT trust framework already in use for FHIR endpoint authentication, OAuth2 dynamic client registration, and TEFCA participant identity. Creda anchors into this framework rather than inventing a new one.

The institution's UDAP certificate carries:

- **Subject DN**: includes the institution's organizational identifier (typically an OID-namespaced identifier registered in the DirectTrust bundle or equivalent).
- **Subject Alternative Names**: include the institution's FHIR endpoint URLs and any associated organizational URIs.
- **Issuer**: a CA in the DirectTrust bundle, or a Creda-recognized CA list maintained by the Participant Registry's legal coordinator (Section 6.1.3).
- **Public key**: used to verify event signatures.

The certificate fingerprint (Blake3 hash of the DER-encoded certificate) is the `institution_id` recorded on every event the institution creates. This binds every signed event to a specific certificate, and through the certificate to a specific real-world institution.

UDAP certificates are not Creda-specific. An institution that already has a UDAP certificate for FHIR endpoint authentication uses the same certificate (or a sibling certificate from the same CA) for Creda. This dramatically reduces onboarding cost — no new PKI infrastructure, no new trust framework registration.

9.1.3 SPIFFE ID for Workload Attestation

While UDAP authenticates the institution, SPIFFE authenticates the running workload. Each Creda peer pod is issued a SPIFFE Verifiable Identity Document (SVID) by the institution's SPIRE server. The SVID is a short-lived X.509 certificate (default lifetime: 1 hour, rotated automatically by the SPIRE Workload API) scoped to the Creda workload identity.

The SVID proves: "This running process is a Creda peer, deployed in an authorized k8s cluster operated by Institution X, and the deployment is current and unrevoked." Without the SVID, even an attacker with a valid UDAP certificate cannot authenticate as a Creda peer — they would also need to be running in an authorized k8s cluster with a SPIRE registration entry for the Creda workload.

SVIDs are used during the libp2p Noise handshake. The Noise XX pattern provides mutual authentication and forward secrecy. Once the secure channel is established, peers exchange UDAP certificates as application-layer evidence of institutional identity.

9.1.4 Two-Credential Binding

Each Creda peer must present **both** a SPIFFE SVID and a UDAP certificate during the peer handshake protocol. A peer that presents only one is rejected. This two-credential binding is the architectural foundation against the malicious peer threat:

- **SPIFFE SVID alone (no UDAP cert)**: the peer is a legitimate workload but has no institutional identity. Cannot create signed events. Rejected.
- **UDAP cert alone (no SPIFFE SVID)**: the peer claims an institution but cannot prove it is a legitimate Creda workload. Could be a compromised process or an attacker with stolen credentials. Rejected.
- **Both credentials, with UDAP cert linked to SPIRE registration**: the peer is verified as a legitimate Creda workload running with an authorized institutional identity. Accepted.

The link between the SVID and the UDAP cert is established during peer registration. The institution's SPIRE server is configured to issue SVIDs only to workloads that present the institution's UDAP-bound identity, and the Participant Registry records the binding. A peer attempting to present a UDAP certificate that is not bound to its SVID is rejected at the libp2p layer.

9.1.5 Key Rotation

UDAP certificates have lifetimes (typically 1-3 years). When an institution rotates its certificate, the rotation is recorded as a network event:

1. The institution generates a new key pair and obtains a new UDAP certificate from its CA.
2. The institution publishes a **key-rotation event** signed by both the old and new private keys, attesting that the new certificate is the successor of the old one. This event is recorded in the Participant Registry subgraph.
3. During a configurable transition window (default: 30 days), events signed by either key are accepted.
4. After the transition window, only events signed by the new key are accepted for new writes. Historical events signed by the old key remain valid indefinitely — the rotation is forward-looking, not retroactive.

If a key is compromised before its scheduled rotation, the institution publishes an emergency revocation event (signed by the institution's NPA-registered emergency key, kept offline) and a new UDAP cert is issued. Events signed by the compromised key after the revocation timestamp are invalid; events signed before the compromise remain valid (the revocation does not retroactively invalidate legitimate prior activity).

cert-manager (Section 7.4.6) handles the operational mechanics of certificate rotation in k8s; the spec defines the protocol-level rotation event format.

9.1.6 Patient Signing Keys

Patients with their own keys (the patient-as-participant model from Section 3.1) use a separate key infrastructure. Patient keys are not UDAP certificates — they are typically issued by a patient-facing OIDC provider with WebAuthn/passkey backing.

- **Issuance**: a patient registers with a Creda-aware patient portal or app, which provisions a key pair backed by the device's secure element (or a recoverable cloud-backed passkey).
- **Identity binding**: the patient's public key is bound to an OIDC `sub` claim issued by an identity provider trusted by the network. The binding is recorded in the patient's own identity subgraph as a special Assert event with verification method `patient-self-attestation`.
- **Trust weight**: patient keys carry weaker default trust weights than UDAP-anchored institutional keys. A patient's self-asserted address change is meaningful but should not override an insurance-card-verified address from an institution. Confidence scoring (Section 5.3) reflects this.
- **Recovery**: patient key loss is a real operational concern. The IG supports key recovery via OIDC-mediated re-enrollment — the patient re-authenticates with their identity provider, and a new key is bound to the same subgraph. The old key is revoked but historical events signed by it remain valid.

Patient keys are the cryptographic substrate for `$creda-self-verify` (Section 8.2.9.4). Their adoption rate over time is a key driver of how much the network can rely on patient-direct verification versus registrar-mediated disambiguation.

9.1.7 Zero Trust Controls Against Insider Threat

Compromised institution credentials — whether through insider threat, credential theft, or operational error — are the most difficult threat to defend against because the credentials are legitimate. Creda cannot prevent a rogue employee at a participating institution from misusing valid credentials. What Creda can do is **make misuse expensive, detectable, and bounded** by applying zero trust principles throughout the architecture.

Continuous verification, not perimeter trust. No request is trusted by virtue of originating inside an institution's network. Every Creda operation, whether from a clinician, a registrar, or an automated system, is authenticated and authorized at the point of action — not at the institutional boundary. The Bridge enforces SMART on FHIR scopes per request; Creda Core enforces consent and party-of-the-subgraph constraints per operation; gossip propagation enforces signature verification per event. There is no "trusted internal network" that bypasses these checks.

Least privilege by default. SMART on FHIR scopes are scoped narrowly. A registrar at a front desk has scopes for `Patient/$creda-disambiguate` and `Patient/$match` but not for `Patient/$creda-tombstone`. A clinician has scopes for read and `Patient/$creda-attest` for patients with whom they have a treatment relationship, not blanket read access. A privacy officer has elevated scopes for tombstoning. Each role has the minimum scopes needed for the role's function.

Per-operation auditing. Every read, write, and operation generates an audit record with the authenticated user identity, the operation, the affected resources, and the SMART scopes presented. This is enforced at the Bridge level using HAPI's standard auditing infrastructure, with records flowing to the institution's SIEM. Audit records are themselves immutable and signed.

Anomaly detection signals. Patterns indicative of credential misuse are surfaced as security signals: a registrar invoking `$creda-disambiguate` against many candidates outside their normal patient population, a clinician accessing patient subgraphs at a rate inconsistent with clinical workflow, bulk queries from accounts that historically issue single-patient lookups. The anomaly detection is not part of Creda itself — it is the institution's responsibility — but Creda's audit logs are designed to support the necessary signals.

Bounded blast radius. Even with valid credentials, an insider's blast radius is bounded by the architecture. They cannot tombstone events at other institutions (only the originating institution can amend or tombstone its own assertions, Section 3.1). They cannot retroactively modify history (the DAG is structurally append-forward, Section 7.2.2). They cannot impersonate other institutions (signatures bind events to the originating UDAP key). What they can do is misuse their own institution's authority — and that is auditable, attributable, and recoverable through Contest events (for fraudulent Links) and reputation downgrades.

Out-of-band controls. Some operations require out-of-band confirmation regardless of credentials presented. Tombstone operations require a privacy officer's attested approval (a separate signature from a different role's key). Bulk export operations require operator approval recorded in the institution's change management system. These

controls are enforced by the institution's Bridge configuration, not by Creda Core — Creda provides the hooks; the institution defines the policy.

The honest framing: zero trust does not eliminate insider threat. It makes insider threat **expensive to execute, fast to detect, and bounded in damage**. Combined with the immutable audit trail (Section 9.4), the cost of insider misuse is high enough to deter most adversaries and the detection latency is short enough to limit harm.

9.1.8 SMART on FHIR for Clinical Workflows

When a clinician at an institution accesses a Patient via the FHIR Bridge, they authenticate via SMART on FHIR — the established OAuth2 flavor for FHIR authorization. The institution's existing SMART infrastructure handles user authentication and scope authorization; the Bridge inherits the result.

The flow:

1. The clinician authenticates to the institution's SMART authorization server (typically the institution's existing OIDC IdP).
2. The clinician's application requests a SMART access token with appropriate scopes (e.g., `patient/Patient.read`, `user/Patient.$creda-attest`).
3. The application presents the access token to the Bridge.
4. The Bridge validates the token using the institution's introspection endpoint, extracts the user identity and scopes, and authorizes the operation.
5. For operations that need to create signed events, the Bridge calls Creda Core, which signs the event with the institution's UDAP key. The user identity is recorded in the event metadata for audit purposes but the cryptographic signature is institutional.

This separation — user authentication via SMART, event signing via UDAP — reflects the reality that signed events represent institutional assertions, not individual ones. A clinician's order entry is an assertion *by the institution* that the clinician acted on its behalf. The clinician's identity is auditable; the cryptographic authority is institutional.

9.1.9 DDOS Mitigation

Distributed denial of service attacks on the network can take several forms, each with specific mitigations:

- **Gossip floods**: an attacker uses a legitimate peer to flood the gossip mesh with valid events. Mitigated by per-peer rate limiting (Section 6.2.2), reputation downgrades for sustained rate limit violations (Section 6.4.1), and gossipsub's built-in mesh management which drops misbehaving peers from the mesh.
- **DHT query storms**: an attacker floods the DHT with `FIND_NODE` or `GET_PROVIDERS` queries. Mitigated by libp2p's Kademlia implementation rate limits and by reputation-aware query response (low-reputation peers get slower or rejected responses).
- **Malformed event submissions**: an attacker submits events that fail validation (bad signatures, malformed CBOR, structural violations). Mitigated by validating cheap properties first — the receiving peer checks UDAP cert validity and signature before any expensive parsing or graph traversal — and by reputation downgrades for peers that consistently send malformed events.
- **Connection exhaustion**: an attacker opens many connections to consume peer connection slots. Mitigated by libp2p's connection manager limits, per-source-IP connection caps, and prioritization of authenticated peers over pending handshakes.
- **Bridge-level DDOS**: standard FHIR endpoint DDOS protections apply at the Bridge — WAF rules, rate limiting at the institution's ingress, request size limits. These are operational concerns not unique to Creda.

The design philosophy: shift validation cost to the attacker. Cheap signature verification rejects most fraudulent traffic before it reaches expensive graph operations. Reputation downgrades amplify the cost of sustained attacks. Rate limits cap the worst case.

9.2 Patient Privacy and Data Minimization

9.2.1 Demographic Tokenization

Cleartext demographics never traverse the Creda network. Each demographic field is tokenized using a deterministic hash function with a network-wide salt:

```
token = Blake3(network_salt || field_type || normalized(field_value))
```

The `field_type` discriminator (`name_family`, `dob`, `ssn_last_four`, etc.) prevents cross-field token collisions — a name that happens to hash to the same value as a DOB cannot accidentally match. The `network_salt` is a 256-bit value distributed via the Participant Registry; it prevents rainbow-table attacks against tokens.

The tokenized form is what travels in identity events that propagate via gossip. The DHT key (Section 6.1.6) is derived from a combination of tokens. Cross-institution matching queries use tokens. Cleartext demographics exist only in the institution that originated the assertion and (subject to consent) in institutions that have explicitly retrieved them via FHIR queries.

9.2.2 Salt Rotation

The network salt is rotated annually. Salt rotation is necessary to limit the long-term value of any single salt compromise — even if an adversary somehow obtains the current salt, prior tokens remain protected by historical salts.

Rotation works via a transition window:

1. The Participant Registry's coordinator publishes the next salt several months before its activation date.
2. During the transition window (default: 6 months), peers maintain both the current salt and the next salt.
3. Tokens computed during the window use both salts; matching queries try both. Events are stored with their original token values regardless.
4. After the activation date, new tokens use only the new salt. Old tokens remain valid for matching against historical events.

The salt history is retained indefinitely — peers must be able to match against tokens from any prior salt era. The salt history is small (a few hundred bytes per year), so storage is not a concern.

9.2.3 Normalization Rules

Tokenization is only useful if the same logical input produces the same token across institutions. This requires deterministic normalization, which is part of the IG and identical across all peers.

- **Names**: lowercased, NFD-normalized Unicode (decomposed), diacritics stripped, hyphens and apostrophes removed, leading/trailing whitespace trimmed. "O'Brien" and "OBRIEN" produce the same token; "Müller" and "Mueller" produce the same token.
- **Dates**: ISO 8601 date format (YYYY-MM-DD). Time components, if present, are dropped for DOB tokenization.
- **Sex**: FHIR administrative gender code (`male`, `female`, `other`, `unknown`). Lowercased.
- **Addresses**: USPS-standardized form using libpostal-equivalent normalization. Postal codes use ZIP+4 when available, ZIP-5 otherwise.
- **SSN fragment**: digits only, no separators. Only last 4 digits are tokenized; full SSNs are never stored or tokenized in Creda.
- **Identifier strings (MRNs, member IDs)**: case-preserved (some MRN systems are case-sensitive), whitespace-trimmed, namespaced by `system` to prevent cross-system collisions.

The reference normalization library is part of Creda Core, with bindings available for the HAPI Bridge. Institutions integrating their own systems must use the reference library; ad hoc normalization will produce inconsistent tokens and break matching.

9.2.4 Cleartext Retrieval Protocol

When an institution needs cleartext demographics (e.g., a clinician viewing a patient's name during an encounter), the institution's Bridge queries the originating institution via a peer-to-peer FHIR call. The flow:

1. The requesting institution identifies the originating institution from the Assert event's `institution_id`.
2. The requesting institution's Bridge issues a FHIR `Patient/[id]/$creda-cleartext` operation to the originating institution's Bridge over an authenticated, encrypted channel (libp2p Noise transport, with mutual UDAP cert authentication at the application layer).
3. The originating institution's Bridge enforces consent: is the requester authorized for cleartext access to this patient? If yes, returns the cleartext fields requested. If no, returns a denial with rationale.
4. The requesting institution receives cleartext only for the fields explicitly authorized.

This cleartext retrieval is not part of the gossip flow. It is a synchronous, point-to-point query subject to fine-grained consent. Cleartext is never broadcast, never cached at intermediate peers, and is held at the requesting institution only as long as needed for the clinical workflow.

9.2.5 TEFCA IAS Tokenization Compatibility

TEFCA's Individual Access Services specification includes a tokenization scheme for cross-network identity matching. Creda's tokenization must align with or interoperate with TEFCA's so that institutions don't need parallel tokenization implementations.

Where the schemes are identical (normalization rules for names and dates appear to align), Creda uses TEFCA's algorithm directly. Where they diverge, the IG documents the bridging logic — a translation layer in the Bridge that produces both Creda-flavored tokens (for Creda matching) and TEFCA-flavored tokens (for TEFCA $match calls) from the same input.

This is an ongoing alignment effort that should be tracked through the Sequoia Project and the TEFCA technical workgroup. Where Creda's tokenization is more privacy-preserving (e.g., due to network salt rotation, which TEFCA does not currently specify), Creda retains its scheme and provides bridge translation.

9.2.6 Bloom Filter Optimizations

For matching workflows that need to pre-filter candidates before pulling full subgraphs, Bloom filters can be exchanged. An institution can include in its DHT query a Bloom filter over attributes it is **not** interested in (e.g., institutions the patient has not visited, payers the patient is not affiliated with) to reduce false-positive matches.

Bloom filters are a query-side optimization, **not** a security boundary. They reduce wasted work but do not provide privacy protection — the network still sees the query. They are documented here for completeness but should not be relied upon for confidentiality.

9.2.7 Minimum Necessary

Every cross-institutional FHIR request specifies the minimum data needed:

- **Element scoping**: FHIR `_elements` parameter restricts the returned resource to the elements the requester needs. A request that does not specify `_elements` receives only a minimal projection by default (e.g., `Patient.id`, `Patient.identifier`, `Patient.active`) rather than the full resource.
- **Profile-driven defaults**: per-institution policy can configure default minimum projections by SMART scope. A clinician with `patient/Patient.read` may receive a richer default projection than a researcher with `patient/Patient.read.summary`.
- **Bulk export gates**: bulk export operations require explicit approval and a documented purpose. They are not available via standard FHIR scopes; a separate operator workflow is required.

Minimum necessary is a HIPAA requirement and an architectural property in Creda. Defaulting to less data, with explicit opt-in for more, is the safer posture.

9.3 Authorization Enforcement (Security View)

This section describes authorization enforcement from the security and access-control perspective. The authorization model itself — event types, the Portable Authorization Artifact, dual-control enforcement, and the evaluation algorithm — is specified in Section 4. This section covers the security properties of that model: default postures, break-the-glass, sensitivity classifications, and cross-institutional honoring.

9.3.1 Authorization as Enforceable Predicate

An AuthorizationGrant in the DAG (Section 4.3.1) is not just a record of the patient's directive — it is an enforceable access predicate. Before a peer responds to a subgraph query, it executes the seven-step authorization evaluation algorithm (Section 4.6), collecting Grants, subtracting validated revocations, matching audience, and evaluating scope, purpose, use-mode, expiration, and volume. The dual-control model (Section 4.5) adds a second enforcement point: the Export Gate validates authorization before data egress at the source, and the Verifier validates it again at the point of use.

The evaluation is local to the responding peer — there is no remote consent service. This is essential for performance (no extra round trip per query) and for resilience (a peer continues enforcing authorization during network partitions). What follows are the security-relevant policies that parameterize this enforcement.

9.3.2 Default Authorization Posture

What if no AuthorizationGrant exists yet for a patient? The spec supports two configurable postures:

- **Deny-by-default**: no Grant = no access. Maximally privacy-preserving. Breaks emergency workflows because most patients will not have proactively created AuthorizationGrants for institutions they have never visited. Research, AI, and federal program scopes always require an explicit Grant regardless of posture.
- **Treatment-presumed-consent**: treatment relationships are presumed consented under HIPAA's TPO (Treatment, Payment, Operations) exception. Auditing is stricter to compensate — every access without explicit consent generates a high-priority audit record.

Each institution configures its own posture as a network participant. The spec **recommends treatment-presumed-consent as the default** for US healthcare deployments, to align with current practice under HIPAA TPO. Deny-by-default is appropriate for international deployments under stricter privacy regimes (GDPR-equivalent), or for specific high-sensitivity subgraphs (e.g., behavioral health, where many states have stricter consent requirements than HIPAA TPO).

The posture is not a network-wide setting — different institutions can configure differently, and the most restrictive applicable posture wins (Section 9.3.4).

9.3.3 Break-the-Glass Workflows

Emergency access requires bypassing normal consent. A break-the-glass event is a special Attest with:

- A `breakTheGlass` flag set to true.
- A free-text justification (typically referencing an emergency department workflow, a code blue event, or similar).
- The clinician's authenticated identity (from SMART), recorded in event metadata.
- Standard institutional UDAP signature.

Break-the-glass is **auditable but not preventable** by design. Patient safety supersedes authorization enforcement in true emergencies — a peer that refused emergency access because of a missing Grant would cause harm. Instead, the architecture makes break-the-glass:

- **Visible**: every break-the-glass event generates an immediate audit alert to the patient (when a patient communication channel exists) and to the institution's privacy officer.
- **Costly**: institutional privacy officers review break-the-glass events. Patterns of abuse are detected.
- **Recordable**: the event is permanent in the DAG, attributable to the specific clinician, and cannot be retroactively concealed.

Break-the-glass is the right model for healthcare. Privacy controls that fail safe (denying access during emergencies) cause harm; controls that fail open (allowing access with strict accountability) align with how clinical care actually works.

9.3.4 Consent Scope and Cross-Institutional Honoring

AuthorizationGrants specify scope (see Section 4.3.1 for the full payload):

- **Subgraph segments**: whole subgraph, specific event types, or specific date ranges.
- **Event types accessible**: read-only access to Assert and Provenance? Read access to all events? Write access (the ability to create Attest events on this subgraph)?
- **Grantee**: a specific institution by UDAP fingerprint, an institutional class (e.g., "any TEFCA QHIN"), or wildcard.
- **Expiration**: explicit date or indefinite.

Cross-institutional consent honoring: when a peer at Institution A holds events that originated at Institution B, and a third Institution C requests those events, the responding peer (A) must honor:

- **Patient's consent** for C to access the relevant subgraph segments.
- **Institution B's policies** regarding redistribution of its events.
- **Institution A's own policies** regarding the events it holds.

The most restrictive applicable consent wins. This prevents a less-strict institution from becoming a laundering point for events from a stricter institution. Practically, this means each event carries metadata about its originating institution's redistribution policies, and consuming peers honor those policies regardless of their own posture.

9.4 Audit Trail

9.4.1 The DAG IS the Audit Trail

Because every event is signed and the graph topology is structurally append-forward (Section 7.2.2), the DAG itself is a tamper-evident audit log of every identity action. Auditors can walk the chain and see exactly:

- What was asserted (the event's payload).
- By whom (the institution_id and signature).
- When (the wall-clock timestamp, plus the logical clock for causal ordering).
- Based on what (the parent references).
- With what verification (the verification_method on Assert events, the method on Link events, etc.).

Compare to today's MPI audit logs: typically separate from the data, often stored in a relational database that an administrator could modify, sometimes incomplete due to logging failures. Creda's audit trail is intrinsic to the data — there is no separate logging system to fail or be tampered with.

For tombstoned events, the audit trail preserves the structural record (this node existed, was tombstoned on this date by this party) while removing the PII payload. The audit trail of *what was forgotten* remains, even though *what was asserted* is gone.

9.4.2 Read-Side AuditEvent Generation

The DAG covers the write-side audit trail (events created). Read-side audit — who queried what, when — is covered by FHIR AuditEvent resources generated by the Bridge.

Every operation that reads identity data generates an AuditEvent:

- The requester's authenticated identity (from SMART).
- The institution issuing the request.
- The target patient subgraph(s).
- The operation performed (Patient.read, $creda-provenance, etc.).
- The SMART scopes presented.
- The response status (success, denied, partial).

AuditEvents are stored in the institution's audit infrastructure (HAPI's standard audit support, typically routing to a SIEM). They are not part of the gossip-replicated DAG — they are local to each institution and not propagated.

The two together — DAG for write-side history, AuditEvent for read-side history — give complete audit coverage.

9.4.3 HIPAA Accounting of Disclosures

HIPAA requires covered entities to provide patients with an accounting of disclosures on request. Creda's audit infrastructure supports this natively:

```
GET AuditEvent?patient=Patient/[id]&date=ge2024-01-01&category=disclosure
```

This query, against the institution's AuditEvent store, produces the accounting automatically. No separate disclosure tracking system is needed. The query can be exposed to the patient via the IAS interface (Section 8.4.4) so that patients can self-serve their accounting of disclosures from any participating institution.

9.4.4 21st Century Cures Act Compliance

The Cures Act prohibits practices that interfere with patient access to their data. Creda's design actively enables Cures Act compliance:

- **Patient-side IAS**: patients can access their full identity provenance chain via standard FHIR operations.
- **Native provenance access**: the chain is not a hidden internal structure; it is a first-class FHIR resource that patients can read.
- **Patient-direct verification and contestation**: patients can contest incorrect links, request tombstoning of erroneous data, and verify their own identity through the network.
- **No information blocking**: the architecture prevents an institution from withholding identity provenance from a patient — provenance is replicated across the network and accessible via the IAS.

The Cures Act is, in many respects, a regulatory mandate for the kind of patient-empowered identity model Creda implements. Compliance is intrinsic to the design rather than a separate compliance layer.

9.5 Future Privacy Enhancements

Several privacy enhancements are out of scope for v1 but should be tracked as future work, particularly to address the **curious peer** threat (peers that follow protocol but learn unintended information from network traffic patterns).

- **Private Set Intersection (PSI) for matching.** Today's tokenization provides confidentiality of values but not query privacy — a peer that receives a token query learns that the requesting institution is interested in a patient with that token. PSI protocols allow two parties to compute the intersection of their respective sets without revealing the elements of the sets to each other. PSI for Creda matching would let an institution determine which patients are common with another institution without either party learning the other's full patient set.
- **Oblivious DHT lookups.** Today's DHT queries reveal the queried key to the responding peers. Oblivious DHT protocols (e.g., those built on private information retrieval) hide the queried key from the responder. This prevents network-wide patterns of who is interested in which patients from being observable by curious peers.

- **Differential privacy for aggregate queries.** Population-level queries ("how many patients in this subgraph have insurance from Payer X?") can leak individual information when combined across queries. Differential privacy adds calibrated noise to aggregates, providing provable bounds on individual disclosure.
- **Zero-knowledge proofs for identity assertions.** Some identity claims could be proved without revealing the underlying data — a patient could prove they are over 18 without revealing their DOB, or prove they have a relationship with a specific institution without revealing which. ZK proofs for these claims would be a significant privacy upgrade for patient-side workflows.

These are research-active areas and their inclusion in Creda depends on the maturity of cryptographic libraries (particularly in Rust) and on institutional appetite for the operational complexity each adds. The architecture is designed to admit these enhancements without breaking changes — tokenization can be replaced with PSI tokens, DHT lookups can route through oblivious gateways, AuditEvent aggregation can incorporate differential privacy, and ZK-proof Assert events would be a new event type within the existing extensible enum.

10. System Components

This section defines the deployable artifacts that compose a Creda peer and the network-level services that govern admission. With Appendix C clarifying that most subsystems are assembled from existing components, this section focuses on how those parts compose: what processes run, in what containers, with what interfaces, and what configuration they require.

10.0 Admission Control vs. Runtime Coordination

A clarifying distinction up front, because it affects how readers interpret the rest of this section: Creda has **admission control** but does not have a **runtime coordinator**. These are different things and the network needs the first while explicitly avoiding the second.

Admission control governs who may participate. Joining the network requires executing the Network Participation Agreement (NPA), having UDAP credentials registered in the Participant Registry, and being vetted by the legal coordinator (Section 6.1.3). This is gated, not open. Vetting is essential — under HIPAA, peers exchange Protected Health Information and must have BAA coverage in place before any traffic flows. Random providers cannot join.

Runtime coordination governs how operations are routed during normal operation. Many federated systems have a runtime coordinator: a central broker through which queries are dispatched, where consensus is achieved, where writes are sequenced. Creda explicitly does not. Once admitted, peers operate peer-to-peer — events propagate via gossip, queries route via DHT, replication occurs via direct anti-entropy between peers. There is no central server through which traffic flows.

This is structurally similar to DirectTrust in the existing health IT ecosystem: a vetted trust framework with admission control, where admitted participants exchange messages directly without DirectTrust mediating each message. The trust framework administrator is essential; the runtime coordinator is unnecessary.

The components in this section reflect this split: Sections 10.1-9.4 cover the per-peer components that every participating institution operates; Section 10.5 covers the network-level admission control service that the legal coordinator operates.

10.1 Creda Core (Rust)

10.1.1 Process Architecture

Creda Core is a single Rust binary, statically-linked, exposing a gRPC API on a Unix domain socket. One process per peer, one peer per pod. The binary supports two runtime modes:

- **Daemon mode**: long-lived peer process. Listens on the libp2p port, serves gRPC, runs scheduled tasks, exposes Prometheus metrics. This is the default mode in production deployments.
- **CLI mode**: one-shot administrative operations. Subcommands include `creda init` (generate keys, write initial config), `creda snapshot` (force a snapshot to object storage), `creda registry verify` (check Participant Registry health), `creda peer list` (show currently active peers), `creda event inspect` (debug a specific event by UUID).

The same binary supports both modes — there is no separate CLI tool. CLI mode is used for ops, debugging, disaster recovery, and integration testing.

10.1.2 Module Organization

Creda Core is internally organized into modules with clear boundaries:

- **`events`**: identity event types, schema validation, signing, signature verification.

- `dag` : DAG operations — likely libgit2-backed per Appendix C.7. Subgraph traversal, root discovery, fork/split semantics, parent reference management.
- `network` : libp2p wrapper. Peer connections, gossip publish/subscribe, DHT operations, anti-entropy protocol. Hidden behind a `NetworkTransport` trait to allow alternative implementations.
- `storage` : persistence interface. Hidden behind a `Store` trait. Default implementation uses libgit2; alternative implementations (e.g., RocksDB, sled) can be plugged in for testing or specialized deployments.
- `authorization` : authorization evaluation engine. Executes the seven-step evaluation algorithm (Section 4.6) over AuthorizationGrant and AuthorizationRevocation events; backs both the responding-peer query path and the Verifier.
- `confidence` : confidence scoring engine. Implements per-field confidence (Section 5.3.2), temporal decay, attestation amplification, agreement amplification.
- `disambiguation` : question selection and answer scoring for `$creda-disambiguate` (Section 8.2.9).
- `registry` : Participant Registry client. Tracks the current set of admitted institutions and their UDAP certificates; consumes registry update events from the network.

Each module has unit tests and well-defined interfaces. Integration tests exercise the gRPC API end-to-end.

10.1.3 gRPC API Surface

The gRPC API is the contract between Creda Core and any client (the Bridge, the CLI, ops tooling). Major operations:

- `CreateEvent(EventRequest) → Event` : create any of the nine event types after validation and signing.
- `GetEvent(UUID) → Event` : retrieve a specific event.
- `GetSubgraph(SubgraphID, depth) → Subgraph` : retrieve a patient's subgraph or a portion of it.
- `GetEffectiveIdentity(SubgraphID) → ProjectedIdentity` : compute and return the effective identity per Section 5.2.4.
- `MatchByTokens(TokenSet) → Vec<SubgraphCandidate>` : match against tokenized demographics, returning candidate subgraphs with scores.
- `RequestDisambiguationQuestions(CandidateSet) → QuestionSet` : stage 1 of the disambiguation operation.
- `ScoreDisambiguationAnswers(SessionID, AnswerSet) → RefinedMatch` : stage 3 of the disambiguation operation.
- `EvaluateConsent(SubgraphID, RequestingInstitution, Scope) → ConsentDecision` : check whether a request is authorized.
- `Subscribe(TopicSelector) → Stream<Event>` : subscribe to event notifications matching a topic selector.
- `GetMetrics() → MetricsSnapshot` : ops query for current peer state.

The API is designed to be stable across minor versions. Major-version bumps are reserved for breaking changes; minor versions are additive only.

10.1.4 Trait-Based Abstractions

Each major dependency is hidden behind a Rust trait with a well-defined interface:

- `NetworkTransport` : abstracts libp2p. Allows swapping for QUIC-based or other transports if libp2p proves unfit.
- `Store` : abstracts the storage engine. Default libgit2 implementation; alternatives can be plugged in.
- `Signer` : abstracts the signing key. Default uses an in-memory key (with the private key sourced from a k8s Secret); alternative implementations can wrap an HSM, cloud KMS, or hardware token.
- `RegistryClient` : abstracts the Participant Registry. Default consumes registry update events from the network; alternative implementations can read from a static configuration file for air-gapped deployments.

The trait boundaries serve two purposes: they enable mocking in tests, and they preserve optionality for replacing components without restructuring the rest of the system.

10.1.5 Async Runtime and Concurrency

Creda Core uses **tokio** as its async runtime. All I/O — gRPC, libp2p, storage — is async. The whole process runs on a single tokio runtime; there is no separate runtime per module. Long-running tasks (scheduled jobs, gossip handling, anti-entropy partners) run as tokio tasks; CPU-bound work (signature verification, hash computation) is dispatched to a thread pool via `tokio::task::spawn_blocking` to avoid blocking the async runtime.

10.1.6 Configuration

Configuration is hierarchical:

1. **Defaults** baked into the binary. Sensible production-ready values — peers should run with no configuration changes for development and produce a working but unconfigured deployment.

2. **TOML configuration file** mounted from a k8s ConfigMap. Overrides defaults. Structured, documented, validated at startup.
3. **Environment variables** for secrets and per-pod values (UDAP cert paths, SPIFFE socket paths, peer ID overrides).
4. **CLI flags** for one-shot operations, overriding everything above.

Configuration schema is validated at startup. Bad config fails loudly before the peer accepts any traffic — no silent fallbacks to defaults that could mask misconfiguration.

10.2 Export Gate (Source-Side Enforcement)

The Export Gate is the source-side control point of the dual-control enforcement model (Section 4.5). It validates the Portable Authorization Artifact governing a data release before data leaves the source system, and emits the ExportReceipt event recording the release.

10.2.1 Role and Placement

The Export Gate runs at the data egress boundary — typically inside or adjacent to the institution's EHR, data warehouse, or FHIR endpoint, wherever clinical data actually leaves institutional control. It is a separate component from Creda Core precisely because of this placement: Core sits in the query/replication path for trust events, while the Export Gate sits in the clinical-data egress path. The two communicate (the Gate calls Core to validate authorization state and to emit the ExportReceipt) but are deployed at different points in the institution's architecture.

10.2.2 Validation Performed

Before permitting a data release, the Export Gate confirms that the governing Portable Authorization Artifact is:

- **Signed** by an institution whose UDAP certificate is current in the Participant Registry.
- **Unexpired** per the Grant's duration.
- **Correctly scoped** for the specific data and purpose of the release.
- **Addressed** to an audience that includes the requesting institution.
- **Unrevoked** — no validated AuthorizationRevocation exists for the governing Grant in the local DAG view (Section 4.6, Step 2 semantics apply).

If validation fails, data is not exported and the failure is logged. If validation succeeds, the Gate emits an ExportReceipt (Section 4.3.3) recording that the release occurred under the specific Grant, then permits the egress.

10.2.3 Implementation

The Export Gate is a small library/service (reference implementation in Rust, with bindings for common EHR integration languages) that wraps Creda Core's authorization evaluation. It is intentionally thin: it does not reimplement authorization logic, it calls Core's `EvaluateAuthorization` and acts on the result. Institutions integrate it at their egress points — as a FHIR interceptor on their HAPI endpoint, as a pre-export hook in their data warehouse, or as a sidecar that the EHR's interface engine consults before transmitting.

10.3 Verifier (Relying-Side Enforcement)

The Verifier is the relying-side control point of the dual-control model (Section 4.5). It validates identity-continuity evidence, authorization state, and provenance chain integrity at the point of use — locally, including offline.

10.3.1 Role and Placement

The Verifier runs wherever a relying party uses data it received: an EHR rendering a record obtained from another institution, a payer system adjudicating a claim, a research platform ingesting authorized data, an AI inference pipeline checking that its inputs were authorized for inference use. The Verifier confirms, at the moment of use, that the authorization under which the data was obtained still holds and that the data's provenance is intact.

10.3.2 What the Verifier Checks

For a given use, the Verifier confirms three things together:

- **Authorization validity**: the governing Portable Authorization Artifact is signed, scoped to cover the intended use, unexpired, addressed to this institution, and unrevoked.
- **Identity continuity**: the subgraph traversal confirms the authorization artifact is bound to the patient whose data is being used — the Grant references this patient's subgraph and identity continuity holds.
- **Provenance integrity**: the relevant provenance chain has no broken signatures and no missing parents; the

evidence the use relies on is intact.

10.3.3 Local and Offline Operation

The Verifier operates against the relying institution's local synchronized DAG state. It does not require a callback to the source system for routine verification. This is a deliberate and important property:

- **Resilience**: verification continues during network partitions, source-system outages, or degraded connectivity. The Verifier uses its most recent synchronized state and can flag staleness if its DAG view is older than a configurable threshold.
- **Adoption**: a consuming system can embed the Verifier SDK and check authorization locally without operating a full Creda peer. This lowers the integration bar for EHRs, payer systems, and research platforms that want to verify but not necessarily to participate as full writers in the network.

When operating offline or against stale state, the Verifier reports its confidence level and the age of its DAG view, so the relying institution can decide whether stale-state verification is acceptable for the use at hand. For high-stakes uses (e.g., a fresh authorization check before a large data export), the institution can require a current DAG view; for routine reads, recent-but-not-instant state is typically acceptable.

10.3.4 Implementation

The Verifier is delivered as an SDK/runtime with language bindings (reference implementation in Rust, with bindings for the languages common in EHR and payer environments). It embeds the authorization evaluation algorithm (Section 4.6) and the identity continuity traversal (Section 5.2.4) as a self-contained library. It maintains a local read-only replica of the relevant DAG segments — synchronized via the same replication fabric as a full peer, but without write or gossip-origination capability. A Verifier is, in effect, a read-only consumer of the trust graph optimized for point-of-use checks.

10.4 HAPI FHIR Bridge (Java)

10.4.1 Process Architecture

The Bridge is a Spring Boot application embedding HAPI FHIR's `RestfulServer` in Plain Server mode (per Section 8.3.3). One process per peer, runs in a separate container in the same pod as Creda Core.

10.4.2 Resource Providers

Custom `IResourceProvider` implementations for each FHIR resource Creda exposes:

- **`PatientResourceProvider`**: read, search, history, and the custom Creda operations (`$creda-provenance`, `$creda-attest`, `$creda-link`, `$creda-contest`, `$creda-tombstone`, `$creda-disambiguate`, `$creda-self-verify`, `$match`, `$export`).
- **`ProvenanceResourceProvider`**: read, search, history, `$creda-contest`.
- **`AuthorizationResourceProvider`**: create (as AuthorizationGrant), read, search, delete (as AuthorizationRevocation); plus the `$creda-authorize`, `$creda-revoke`, and `$creda-verify` operations.
- **`AuditEventResourceProvider`**: read, search (read-side audit only; events from Core are projected as Provenance, not AuditEvent).

Each provider delegates to Creda Core via gRPC. Providers contain no identity logic — they translate FHIR requests into gRPC calls and translate gRPC responses into FHIR resources.

10.4.3 gRPC Client to Core

The Bridge holds a long-lived gRPC connection to Creda Core over the shared Unix domain socket at `/var/run/creda/core.sock`. Connection management is handled by gRPC-Java's connection pooling. Reconnection is automatic on transient failures; sustained Core unavailability surfaces to the Bridge as 503 responses to FHIR clients.

10.4.4 SMART on FHIR Enforcement

OAuth2 token validation, scope checking, and audit log generation happen at the Bridge layer using HAPI's standard SMART support. The Bridge:

1. Extracts the bearer token from the FHIR request.
2. Validates the token via the institution's SMART authorization server (introspection or JWT verification).
3. Extracts the user identity (`sub` claim) and scopes.
4. Authorizes the requested operation against the scopes.

5. Generates an AuditEvent for the access.
6. Forwards the request to Core, including user identity as audit metadata.

Core sees pre-authorized requests with the institution's identity and the user's authenticated identity for audit metadata. Core does not re-authenticate — it trusts the Bridge's enforcement, which is appropriate because Core and the Bridge are co-located in the same pod and communicate over a private socket.

10.4.5 FHIR Validation

Inbound FHIR resources are validated against the Creda IG profiles using HAPI's built-in validator. Resources that fail profile validation are rejected with FHIR `OperationOutcome` before reaching Core. This shifts validation cost to the Bridge, keeps Core's gRPC API focused on already-valid inputs, and gives FHIR clients standard FHIR error responses.

The Bridge also handles US Core profile validation in the same pass — every CredaPatient must conform to US Core Patient (Section 8.2.1).

10.5 Peer Daemon (Runtime Composition)

"Peer Daemon" is the colloquial name for the runtime composition of Creda Core in daemon mode plus the HAPI FHIR Bridge running in the same pod. It is not a separate component — it is what we call the deployed unit when describing operational behavior.

10.5.1 In-Pod Communication

Core and Bridge communicate via gRPC over a Unix domain socket on a shared `emptyDir` volume mounted at `/var/run/creda` in both containers. This avoids network overhead for the in-pod RPC and aligns with k8s sidecar patterns. Network-level isolation between Core and the outside world is handled by k8s NetworkPolicy — only the Bridge's HTTP port and Core's libp2p port are exposed externally.

10.5.2 Scheduled Tasks Within the Daemon

Several recurring tasks run as tokio tasks within Creda Core:

- **Snapshot generation**: every 6 hours by default, write a snapshot of the local store to object storage.
- **Snapshot retention pruning**: daily, delete snapshots older than the configured retention window.
- **Anti-entropy partner selection**: continuous, select peers for anti-entropy comparison based on shared subgraph holdings.
- **Reputation decay**: hourly, apply temporal decay to peer reputation scores.
- **Registry sync**: every 5 minutes, check for Participant Registry updates and apply revocations or additions.
- **Salt transition checks**: daily, check whether the network salt rotation window has changed and update the local salt set accordingly.

These are *internal* scheduled tasks. They are distinct from the k8s CronJobs in Section 7.5, which exist for tasks that benefit from process isolation — bulk MPI imports, large retention sweeps, cross-region anti-entropy coordination. The split: continuous lightweight tasks live inside the daemon; heavyweight or risky tasks run as separate k8s Jobs that don't affect the peer's hot path.

10.5.3 Health Endpoints

The daemon exposes:

- **`/livez`** (HTTP, on a dedicated port): liveness check. Returns 200 if Core is alive (gRPC server responding) and Bridge is alive (HAPI metadata endpoint responding).
- **`/readyz`** (HTTP): readiness check. Returns 200 only if all of: peer is connected to libp2p network, Participant Registry has been synced at least once, at least one peer in the active view is reachable, and storage is healthy.
- **`/metrics`** (HTTP): Prometheus metrics endpoint covering all instrumented points across Core and Bridge.
- **gRPC reflection** (on the gRPC port, restricted to in-pod access): allows debugging tools to inspect the gRPC API.

A peer that hasn't completed bootstrap returns NotReady on `/readyz` so it doesn't receive traffic prematurely.

10.6 Container Image and Kubernetes Deployment

10.6.1 Container Composition

Each peer pod runs two containers:

- **`creda-core`**: Rust binary, peer daemon mode. Built from a multi-stage Dockerfile: Rust builder stage (using the

official `rust:1-slim` image) → distroless runtime (`gcr.io/distroless/cc-debian12`). Final image targets <100 MB.
- **`hapi-bridge`**: Java application. Built from a multi-stage Dockerfile: Maven/Gradle builder → distroless Java runtime (`gcr.io/distroless/java21-debian12`). Final image targets <250 MB (larger because of HAPI FHIR's footprint, but still much smaller than typical full Java images).

Distroless eliminates shell, package manager, and most attack surface. There is nothing for an attacker to exploit if they obtain code execution in the container — no `bash`, no `curl`, no `apt`. Debugging requires deliberate effort (e.g., `kubectl debug` with an ephemeral debug container), which is the right tradeoff for production.

The two containers share an `emptyDir` volume at `/var/run/creda` for the Unix domain socket. They do not share any other state.

10.6.2 StatefulSet Deployment

A Creda peer is deployed as a Kubernetes **StatefulSet**. The reasons:

- Stable peer identity across pod restarts. The pod's hostname is consistent, which affects how the peer presents itself in the libp2p network.
- Stable persistent volume claims. Each replica has its own PVC, retained across pod restarts.
- Ordered deployment and termination. Useful for institutions running multiple peers — they come up and shut down in a predictable order.

Each StatefulSet replica is one peer. Most institutions will run 1-3 replicas; very large institutions may run more for load distribution and redundancy. The replica count is institution-decided based on traffic patterns, not network-mandated.

10.6.3 Persistent Volume

The persistent volume holds the libgit2 repository (or RocksDB store, depending on the Appendix C.7 outcome) plus indexes and local state. Default size: 50 GB, configurable via Helm values. A peer holding events for ~1 million patients fits comfortably in this baseline; larger institutions size up.

PV loss requires snapshot rebootstrap (Section 6.2.5). PVs should be backed by reliable storage (cloud-managed PVs, or on-prem RWO storage with snapshot/replication support). Loss of a PV is recoverable but operationally expensive.

10.6.4 Resource Requirements

Baseline pod resource request:

- **CPU request**: 2 cores (1 for Core, 1 for Bridge). Limit: 4 cores.
- **Memory request**: 4 GiB (1 GiB for Core, 3 GiB for Bridge — Java's heap dominates). Limit: 8 GiB.
- **Persistent volume**: 50 GiB default.

These are starting points. Actual usage depends on patient volume, query rate, and gossip traffic. Larger institutions scale by adding StatefulSet replicas rather than increasing per-pod resources, since each replica adds both compute capacity and a separate copy of the data for redundancy.

10.6.5 Helm Chart Structure

The Creda Helm chart is the primary deployment artifact:

- **`Chart.yaml`**: standard Helm metadata.
- **`values.yaml`**: documented configuration options for institution operators. Every reasonable knob exposed without forcing users to template templates.
- **Templates**:
  - `statefulset.yaml`: the peer pod spec.
  - `services.yaml`: ClusterIP for HAPI HTTP, NodePort or LoadBalancer for libp2p, ClusterIP for gRPC reflection (intra-cluster only).
  - `configmap.yaml`: peer configuration.
  - `secret-references.yaml`: references to k8s Secrets holding UDAP certs, signing keys, registry coordinator's public key.
  - `serviceaccount.yaml` + RBAC: minimal permissions — read ConfigMaps and Secrets in the namespace, no cluster-wide rights.
  - `networkpolicy.yaml`: restrict Core's network access to libp2p peers and the Bridge; restrict Bridge's network access to authorized FHIR clients.
  - `poddisruptionbudget.yaml`: ensure at least one peer remains available during voluntary disruptions.

- **Optional sub-charts**:
  - `minio`: bundled MinIO for institutions without S3-compatible storage.
  - `prometheus-exporter`: scraping configuration if the institution doesn't run Prometheus.
  - `cert-manager-issuer`: opinionated cert-manager configuration for UDAP certificate rotation.

The chart is published to a public Helm repository and tagged per Creda release.

10.6.6 Kubernetes Operator (Future)

Once Creda has more than ~20 production deployments, a dedicated Kubernetes Operator should be developed to automate operational toil that the Helm chart cannot handle: snapshot scheduling that adapts to load, certificate rotation coordination across replicas, Participant Registry change propagation, automatic health-based scaling.

The Operator is deferred — the Helm chart is sufficient for early deployments. The trigger for building the Operator is operational evidence: when institutions repeatedly file the same operational toil tickets, those become Operator features.

10.7 Participant Registry Service (Network-Level)

The Participant Registry is a Creda subgraph (per Section 6.1.3 — the meta-DAG of who is in the network), but the *operational service* maintaining it is a real deployed component, separate from any participating institution's peer. This service is operated by the network's **legal coordinator** — typically an HIE, a nonprofit, or a multi-institution consortium.

10.7.1 Role and Responsibilities

The legal coordinator's service handles:

- **Membership applications.** Institutions wishing to join the network apply through this service. The application captures organizational identity, UDAP certificate fingerprint, BAA execution status, and points of contact.
- **NPA execution workflow.** Coordinates the legal exchange of the Network Participation Agreement, providing a workflow that institutions can complete (e.g., DocuSign integration, manual review by counsel, etc.).
- **Registry update publication.** Once an institution is admitted, the coordinator's service publishes a signed registry-addition event into the Participant Registry subgraph. The event is propagated via normal Creda gossip; existing peers learn of the new participant within seconds.
- **Revocation.** When an institution leaves the network, has its UDAP certificate revoked, or is removed for cause (e.g., persistent misbehavior, BAA breach), the coordinator publishes a signed revocation event. Peers process the revocation and reject future connections from the revoked institution.
- **Coordinator key management.** The coordinator's signing key is the trust anchor for the entire network. Compromise of this key would allow an attacker to add unauthorized institutions or revoke legitimate ones. The key must be operated with hardware-backed key storage (HSM or cloud KMS) and audit logging.

10.7.2 Why This Is Not Centralization

The legal coordinator role is **administrative**, not architectural:

- **The coordinator does not see patient data.** No PHI passes through the coordinator's service. The Participant Registry contains only institutional identifiers and certificate fingerprints.
- **The coordinator cannot create or modify identity events.** The coordinator's signing authority extends only to the Participant Registry, not to patient subgraphs. They cannot impersonate institutions or assert facts about patients.
- **The coordinator cannot censor existing participants' events.** Once an institution is admitted, their events propagate via the peer-to-peer network. Revocation is forward-looking — a revoked institution cannot create new events, but their existing events remain in the network.
- **The coordinator role can be transferred or distributed.** A different organization can take over the coordinator role, or the role can be split across multiple regional coordinators (each governing membership in their region) without changing the protocol. This is a governance decision, not a code change.

This mirrors how DirectTrust operates in the existing Direct messaging ecosystem: a coordinating body for trust framework administration, not a data intermediary. Or how the WebPKI operates: certificate authorities are admission-control authorities, but they are not in the path of every TLS connection.

10.7.3 Service Architecture

The Participant Registry service is small — likely a single Spring Boot or Rust service operated by the coordinator:

- **Web frontend** for institutions to apply for membership and check application status.
- **Admin interface** for the coordinator's staff to review applications, execute NPAs, and manage revocations.
- **Backend** that publishes signed registry events into the Creda network. The backend itself runs a Creda peer (a special-purpose peer authorized to publish to the Participant Registry subgraph).
- **HSM or KMS integration** for the coordinator's signing key.
- **Audit log** of all administrative actions, retained per the coordinator's policies and made available to participating institutions on request.

The service is not high-traffic — registry updates are infrequent (a few per week at most, even for large networks). Standard k8s deployment patterns apply: a Deployment with 2-3 replicas, a managed database for application state, an Ingress with TLS for the web frontend.

10.7.4 Coordinator Governance and Succession

The coordinator role is not technically privileged within Creda — it is a designated role with a registered signing key. The network's governance body (the consortium of participating institutions, or a designated nonprofit board) can update the coordinator role through a coordinated key transition:

1. The current coordinator publishes a coordinator-transition event signed by their key, naming the new coordinator's public key.
2. After a transition window, the new coordinator's key becomes the authoritative signer for registry updates.
3. The old coordinator's key is retired but historical events signed by it remain valid.

This makes the coordinator role recoverable — if the current coordinator becomes unable or unwilling to perform the role, governance can transition to a new one without disrupting the network. The transition does require coordination among participating institutions (since they need to update their trust configuration to recognize the new coordinator's key), but the protocol supports it natively.

11. Operations

This section covers the runbook-level details for operating Creda peers in production. Earlier sections (5, 6, 9) established the architecture; this section is for the operators who deploy, monitor, and recover the system. It assumes basic familiarity with k8s operations and focuses on what is Creda-specific.

Capacity planning guidance — sizing recommendations based on patient volume, query rates, and attestation patterns — is deferred to a separate operations guide that will be published after sufficient real-world deployment data exists to make non-speculative recommendations.

11.1 Node Bootstrap and Catch-Up

11.1.1 Day-Zero Bootstrap

The very first peer in the network is a special case — there are no existing peers to gossip with and no Participant Registry yet. The legal coordinator runs a one-time initialization workflow:

1. Generate the coordinator's signing key in an HSM or cloud KMS. This key is the network's trust anchor and must never leave hardware-backed storage.
2. Create the **genesis Participant Registry event** — a signed event that establishes the initial registry state, including the coordinator's own public key and any founding institutions. Sign it with the coordinator's key.
3. Publish the genesis event and the coordinator's public key out-of-band to founding institutions. This is the only out-of-band trust anchor in the network; everything else flows from it.
4. Deploy the coordinator's own Creda peer with the genesis state pre-loaded.
5. Founding institutions deploy their peers, configured with the coordinator's public key as the trust anchor and the coordinator's peer as their initial bootstrap peer. They sync the genesis Participant Registry event and verify it against the published trust anchor.

This is documented as a one-time procedure in the operator runbook (Section 11.5). It happens once in the lifetime of the network — successor coordinators inherit the network through key transitions (Section 10.5.4), not through new genesis events.

11.1.2 New Institution Bootstrap (Typical Case)

After the network is established, an admitted institution deploys its first peer. The sequence:

1. **Pre-conditions verified.** NPA executed, UDAP certificate registered in the Participant Registry, BAA coverage in

place with all participating institutions.
2. **Operator deploys peer** via the Helm chart, providing UDAP cert, SPIFFE/SPIRE configuration, and the network's bootstrap peer addresses.
3. **Peer initialization.** Core starts, validates configuration, opens libp2p listeners.
4. **Bootstrap peer connection.** Peer connects to one or more bootstrap peers. Mutual authentication via SPIFFE + UDAP.
5. **Participant Registry sync.** Peer pulls the current Participant Registry state from the bootstrap peer, verifies signatures back to the trust anchor, and constructs its local view of admitted institutions.
6. **Mesh and DHT join.** Peer joins gossipsub mesh on relevant topic buckets and begins Kademlia DHT bootstrap.
7. **Snapshot evaluation.** If this institution had prior peers, the new peer pulls the most recent institution-scoped snapshot from object storage. If this is the institution's first peer, snapshot loading is skipped.
8. **Anti-entropy with sibling peers** (if applicable) or with discovered peers in the network for Participant Registry catch-up only.
9. **Ready state.** All bootstrap stages complete; `/readyz` returns 200; the peer accepts FHIR traffic.

Typical bootstrap times:

- **Fresh institution, no snapshot**: 30-90 seconds, dominated by libp2p mesh joining and DHT bootstrap.
- **Institution with existing peers and a 5 GB snapshot**: 2-5 minutes, dominated by snapshot download and load.
- **Institution with existing peers and a 50 GB snapshot**: 10-15 minutes, dominated by snapshot download.

Snapshots benefit from local object storage. Cross-region snapshot pulls during bootstrap can extend these times significantly; institutions with multi-region deployments should ensure each region has local snapshot storage.

11.1.3 Replacement Peer Bootstrap

A peer pod fails — PV loss, scheduling failure, hardware issue. Same institution, new peer. Fastest recovery path:

1. New pod scheduled by the StatefulSet controller. New PVC bound (empty if PV was lost).
2. Peer initializes, connects to bootstrap peers, syncs Participant Registry.
3. Peer pulls the most recent **institutional snapshot** from object storage. The snapshot is per-institution, so it contains the events this institution's peers have stored — sufficient to restore the peer's local state.
4. Peer runs anti-entropy against sibling peers in the same institution to catch any events created since the snapshot.
5. Peer joins gossipsub mesh and DHT, enters Ready state.

Total recovery time: minutes to tens of minutes, dominated by snapshot download size. Institutions running multiple peers per StatefulSet have continuous availability throughout the recovery — sibling peers continue serving traffic while the failed pod recovers.

11.1.4 Bootstrap Failure Modes

What can go wrong during bootstrap, how to detect it, and how to recover:

Failure	Detection Signal	Recovery
Bootstrap peers unreachable	Connection failures to all configured bootstrap addresses; peer remains in `Connecting` state	Verify network connectivity, DNS resolution, and that bootstrap peer addresses are current. Update bootstrap configuration if peers have changed.
Participant Registry sync fails	Peer connects to bootstrap peers but cannot retrieve or verify Registry events	Check that the network's trust anchor (coordinator's public key) is correctly configured. If the coordinator key has rotated, update local trust anchor configuration.
UDAP cert validation fails	Bootstrap peer rejects the connection citing invalid certificate	Verify UDAP cert validity, registration in the Registry, and that the cert's intermediate chain is complete. Check for cert expiry.
Snapshot integrity check fails	Snapshot Blake3 hash does not match manifest	Snapshot is corrupted or has been tampered with. Pull a different snapshot (older one). If all snapshots fail, escalate as potential security incident.
DHT bootstrap times out	No Kademlia peers respond after configured timeout	Network partition or DHT-specific issue. Check that other peers are operational. Anti-entropy with directly-connected peers may proceed even without DHT, allowing degraded operation.
Anti-entropy backlog growing during catch-up	Backlog metric grows faster than it shrinks	Likely insufficient resources for the catch-up workload. Increase peer resource allocation or extend bootstrap timeout.

Each failure surfaces a specific stage in the readiness signal so operators can pinpoint the issue without grep-ing logs.

11.1.5 Catch-Up State Machine

The peer's bootstrap state machine is exposed via `/readyz` and Prometheus metrics:

```
Initializing → Connecting → RegistrySync → SnapshotLoad → DHTBootstrap → AntiEntropy → Ready
                                                                                    │
                                                                                    └─ Degraded (operating but with
caveats)
```

Each state has a configured timeout. A peer stuck in any state beyond its timeout transitions to a `Failed` state, generating an alert. The current state is exposed as a label on the readiness metric: `creda_peer_state{state="DHTBootstrap"} 1`.

The `Degraded` state is for cases where the peer is operational but with caveats — e.g., DHT is empty (no providers found yet), or anti-entropy has incomplete catch-up. The peer accepts traffic but the readiness response includes warnings. Operators decide whether to direct traffic to a Degraded peer based on the specific caveats.

11.2 Monitoring and Observability

11.2.1 Golden Signals

Following Google SRE's golden signals, adapted for Creda:

Latency:

- gRPC operation duration (p50, p95, p99 by operation).
- FHIR request duration (p50, p95, p99 by resource and operation).
- Gossip propagation lag (time between event creation at originating peer and local receipt).
- Anti-entropy round duration.

Traffic:

- Events received per second (by event type).
- Events created locally per second (by event type).
- FHIR requests per second (by operation).
- DHT queries per second.
- Active gossipsub topic subscriptions.

Errors:

- Signature verification failure rate.
- Schema validation failure rate.
- gRPC error rate (by code).
- FHIR 4xx and 5xx rates (by resource and operation).
- Consent denial rate (separated from other 403s — consent denials are expected, not errors).
- Anti-entropy reconciliation failures.

Saturation:

- Persistent volume utilization (% used).
- Memory utilization (RSS as % of limit).
- Gossip outbound queue depth.
- Anti-entropy backlog size.
- gRPC connection pool utilization.

All metrics are exposed via the standard `/metrics` endpoint and follow Prometheus naming conventions: `creda_<subsystem>_<metric>_<unit>`.

11.2.2 Critical Alerts

These page operators when triggered:

- **Peer not Ready for >10 minutes**: indicates a bootstrap or sustained operational failure.
- **p99 replication lag >60 seconds sustained for >5 minutes**: gossip is failing to converge in expected time. Likely a network issue, slow peer in the mesh, or capacity exhaustion.
- **Signature verification error rate >1% over a 15-minute window**: indicates either a compromised peer creating fraudulent events, a key rotation issue (peers haven't picked up the new public key), or — rarely — clock skew causing certificate validity issues.
- **Participant Registry sync stale >30 minutes**: peer cannot retrieve recent Registry updates. Could indicate coordinator service outage, network issue, or trust anchor configuration mismatch.
- **Persistent volume >85% full**: storage saturation imminent. Provision more storage or investigate retention policies.
- **Anti-entropy backlog growing unboundedly over a 30-minute window**: peer is falling behind on reconciliation. Typically indicates resource exhaustion or a runaway ingestion source.
- **Coordinator key signature mismatch**: a registry event was received signed by a key that does not match the configured trust anchor. **Severity: critical security incident.**

11.2.3 Warning Alerts

These notify but do not page:

- UDAP certificate expiring within 30 days.
- Snapshot generation skipped or failed (one occurrence).
- Peer reputation downgrade events for any specific remote peer.
- Unusual disambiguation request patterns from a single registrar (anomaly signal — see Section 9.1.7).
- Anti-entropy partner became unreachable.
- Gossip mesh degraded (active view size below configured minimum).

11.2.4 Default Grafana Dashboards

The Helm chart bundles a set of default Grafana dashboards:

- **Peer Health**: golden signals at a glance, current state, recent error rates, resource utilization.
- **Replication**: replication lag distribution, anti-entropy stats, gossip topology visualization, mesh connectivity.
- **FHIR Activity**: request rates by operation, error rates, p95/p99 latencies, top resources by traffic.
- **Network**: peer count over time, DHT health metrics, bandwidth usage, peer reputation distribution.
- **Security**: signature failures over time, consent denial rate, break-the-glass events, reputation downgrade events, unusual access patterns.
- **Storage**: persistent volume utilization, snapshot generation history, retention pruning activity, libgit2 (or RocksDB) operational metrics.

Each dashboard includes documentation on what to look for and which alerts correspond to which panels. Dashboards are designed for operators who are not Creda specialists — labels and tooltips explain the metrics in operational terms.

11.2.5 Distributed Tracing

OpenTelemetry traces span across Core and Bridge. A FHIR request entering the Bridge generates a trace; the trace continues through gRPC to Core, through the relevant operation, and into any downstream calls (gossip publish, DHT query, anti-entropy with a specific peer). Trace IDs propagate via gossip metadata when an event is created, allowing the lifecycle of a single event to be followed from creation to receipt at remote peers across the network.

Traces are exported via OTLP. Institutions route them to whatever tracing backend they operate (Jaeger, Tempo, Datadog, etc.). Sampling is configurable; default is 1% of FHIR requests fully traced, with 100% trace coverage for all errors and slow requests (>p95 latency).

11.2.6 Audit Log Routing

AuditEvent resources generated by the Bridge route to the institution's SIEM via standard mechanisms:

- **FHIR audit forwarding** to a configured FHIR endpoint (e.g., the institution's existing audit repository).
- **Syslog export** in CEF or LEEF format for SIEM ingestion.
- **NDJSON file output** to a mounted volume that the institution's log shipper can read.

Creda does not host the institution's audit log. It produces audit records in standard formats and routes them to existing infrastructure. This avoids creating a new compliance surface that the institution has to integrate into their existing audit workflows.

11.3 Failure Modes and Recovery

11.3.1 Single Peer Failure

The most common failure mode. Detection via k8s liveness probes; the StatefulSet controller restarts the failed pod automatically.

- **PV intact**: peer recovers within seconds. Restarts, replays in-flight operations from durable state, rejoins the network.
- **PV lost** (storage failure, accidental deletion, k8s claim issue): the new pod has empty storage and follows the replacement bootstrap procedure (Section 11.1.3). Recovery time depends on snapshot size.

For institutions running multiple peers, sibling peers continue serving traffic during recovery. Single-peer institutions experience an outage equal to the recovery time.

11.3.2 Network Partition

Detection signals: anti-entropy with normally-reachable peers stops succeeding; DHT queries to known providers timeout; gossip messages are not propagating to known remote peers.

Behavior during partition:

- Each side continues operating independently.
- Peers create events locally and gossip propagates within the partition.
- The DHT fragments — peers see only providers on their side.
- Effective identity computation continues with the events visible locally; some recently-asserted facts may be missing.

On reconnection:

- Anti-entropy resumes against previously-unreachable peers.
- Deltas exchange in both directions.
- The network converges within minutes for typical partition durations.

No operator action is required for transient partitions — the system self-heals. Operators should monitor for prolonged partitions (>1 hour), which may indicate a real connectivity issue requiring investigation. The metric to watch: distinct peer count visible in DHT compared to known network size from the Participant Registry.

11.3.3 Coordinator Service Outage

The Participant Registry service goes down. Behavior:

- Existing peers continue operating. The Registry updates infrequently and peers cache the current state.
- Existing institutions continue creating events, propagating, and serving FHIR requests normally.
- New institutions cannot complete the join workflow during the outage — applications can be queued at the

coordinator service but not processed.
- Revocations cannot be published. If a revocation was needed during the outage, the network must rely on peer reputation downgrades and out-of-band communication until the coordinator returns.

Recovery: bring the coordinator service back online. No protocol-level intervention needed. Queued applications and revocations are processed in order.

11.3.4 Coordinator Key Compromise

The most severe failure scenario. If the coordinator's signing key is compromised, an attacker could publish unauthorized Registry events: adding malicious institutions, revoking legitimate ones, or attempting to seize the coordinator role.

Detection:

- Peers should monitor for unexpected Registry events. Additions or revocations that the network's governance body did not authorize are flagged.
- Pattern: a sudden burst of additions or revocations is suspicious. Normal Registry activity is sparse.
- Peer reputation systems (Section 6.4.1) may also flag misbehavior from newly-added "institutions" before the compromise is detected directly.

Response:

1. The network's governance body convenes immediately.
2. A new coordinator key is generated via emergency procedure (HSM, hardware-backed, prepared in advance for exactly this scenario).
3. The governance body executes the **emergency key transition protocol** — distributing the new public key to all participating institutions out-of-band (e.g., via signed governance body communications).
4. Institutions update their trust anchor configuration to recognize the new key.
5. Events signed by the compromised key after the transition timestamp are invalid.
6. Events that the compromised key signed before the compromise (legitimate Registry updates) remain valid.
7. The compromised key is revoked retroactively from the transition timestamp forward.

Preparation: The governance body should have an emergency response plan with the new key generated and distributed-but-not-activated in advance, so that response time during a real compromise is hours, not days.

11.3.5 Mass Tombstone Propagation

A high-volume right-to-be-forgotten event — for example, a regulatory ruling requiring tombstoning of all events from a specific institution after a breach, or a class-action settlement requiring deletion of a large patient cohort.

This is operationally significant because tombstoning generates a wave of mutation events that propagate through gossip and force receiving peers to scrub their stores. At scale, this can saturate the gossip network if not orchestrated.

Handled via Argo Workflows:

1. The originating institution's privacy officer authorizes the tombstone batch.
2. An Argo Workflow runs at the institution, generating tombstone events in batches.
3. The workflow rate-limits publication to a configured ceiling (default: 100 tombstones/second per institution) to avoid saturating the network.
4. Receiving peers process tombstones in order; each peer's processing rate is also bounded so the wave doesn't compromise responsiveness for other operations.
5. Anti-entropy ensures any peer that missed tombstones during the wave catches up later.

For very large batches (>1M events), the workflow can be split across multiple sessions or extended over days. The legal coordinator may be informed for awareness, but the workflow does not require coordinator action.

11.3.6 Storage Corruption

Detection: integrity checks fail (libgit2 fsck reports broken references, Blake3 content hash mismatches on stored events, RocksDB checksum failures during read).

Response:

1. The peer is taken out of service via Helm rolling update or manual `kubectl scale` to prevent serving potentially corrupt data.
2. The corrupted store is preserved on its PV (do not delete) for forensic analysis. Snapshot the PV to long-term storage.

3. A replacement peer is launched. It bootstraps from the most recent known-good institutional snapshot.
4. Anti-entropy with sibling peers fills in events created since the snapshot.
5. The replacement peer rejoins the network as a normal peer.

If multiple peers in an institution show corruption simultaneously, suspect underlying infrastructure issues (storage hardware, k8s storage class, file system) and investigate before redeploying.

11.3.7 Disaster Recovery Drills

Institutions should perform DR drills quarterly. Standard drills:

- **Pod failure drill**: kill a peer pod via `kubectl delete pod`. Verify automatic restart and reentry into the network. Measure recovery time.
- **PV loss drill**: delete the PVC of a peer (in a non-production environment, or with operator approval in a low-traffic window). Verify rebootstrap from snapshot. Measure total recovery time.
- **Network partition drill**: use NetworkPolicy to isolate a peer from external network access for 30 minutes. Verify the peer continues serving local traffic. Restore connectivity. Verify anti-entropy convergence.
- **Coordinator unavailability drill**: simulate coordinator service outage by blocking access to it. Verify peers continue operating with cached Registry state.
- **Snapshot integrity drill**: deliberately corrupt a snapshot in object storage, verify integrity check catches it, verify recovery via a different snapshot.

Drill outcomes feed into operational confidence and reveal gaps before real incidents. Results should be documented and tracked over time as the operational maturity metric.

11.4 Integration Testing in Production

Verifying that a new peer is functioning correctly without affecting real patients requires a separate testing surface. Creda supports this via **test patient subgraphs** containing synthetic data that propagate normally through the network.

11.4.1 Test Patient Subgraphs

A test patient subgraph is structurally identical to a real patient subgraph but is flagged as synthetic via a special extension on every event:

```
http://creda.health/StructureDefinition/test-data
```

The extension carries:

- `purpose`: `integration-testing`, `load-testing`, `compliance-validation`, etc.
- `originatingTest`: identifier of the test plan that generated the data.
- `expirationTime`: when the test data should be tombstoned (typically 30-90 days after creation).

Test events propagate through the gossip network like real events but are filtered out of:

- FHIR query results to clinical systems (the Bridge filters by the `test-data` extension when responding to clinical SMART scopes).
- Confidence scoring for real patient subgraphs (test events never contribute to real patients' confidence scores).
- Audit reports for HIPAA accounting (test events are tracked separately).

Test events are **fully visible** in:

- Operator-mode FHIR queries (with a special `test-data` SMART scope).
- Network-level metrics (so test traffic shows up in latency, throughput, and replication metrics).
- Anti-entropy and replication (test events participate in normal replication so the network can verify replication correctness).

11.4.2 Synthetic Data Generation

The reference test data generator is published alongside Creda Core. It generates synthetic patient subgraphs with:

- **Realistic demographics** drawn from public-domain name lists, randomly-generated DOBs in plausible distributions, fictional addresses.
- **Realistic event chains** simulating typical patient journeys: registration at one institution, referral to another, insurance card update, periodic attestations.
- **Configurable scale**: from a single test patient to millions, for load testing.

- **Configurable scenarios**: identity disputes (deliberately conflicting demographics across institutions), link/contest cycles, deceased patients, right-to-be-forgotten patterns.

Synthetic data is generated with a deterministic seed so the same test scenario is reproducible across runs.

11.4.3 Integration Test Workflows

Standard integration tests run by a new peer's operator before promoting the peer to production traffic:

1. **Smoke test**: create a single test Assert event, verify it propagates to known peers within 5 seconds, verify the FHIR Bridge returns it via standard FHIR queries (with operator scope).
2. **Round-trip test**: create test events of every event type, verify each is correctly propagated, validated, and reflected in effective identity computation.
3. **Disambiguation test**: create two test patients with deliberately ambiguous demographics, invoke `$creda-disambiguate`, verify questions are returned and answers are scored correctly.
4. **Authorization enforcement test**: create test patient with restrictive AuthorizationGrants, verify access from unauthorized institutions is denied, verify access from authorized institutions succeeds, verify a revocation takes effect within Bound 1 (Section 4.7).
5. **Anti-entropy test**: create test events while anti-entropy is observed, verify replication completes within expected time bounds.

These tests run automatically on a new peer's deployment via a CronJob in the Helm chart, gated by an operator flag. Results are exposed as metrics and can be checked before directing real traffic.

11.4.4 Test Data Retention and Cleanup

Test events have a configured `expirationTime`. A scheduled task in each peer (Section 10.3.2) tombstones expired test events automatically. This prevents test data accumulation over time and ensures the network's storage growth reflects real traffic, not test residue.

Institutions can also explicitly tombstone test data on demand via the standard `$creda-tombstone` operation with `legalBasis=test-cleanup`.

11.5 Legal Coordinator Operations Runbook (Stub)

The legal coordinator's operational responsibilities are distinct from institutional peer operations and warrant a separate runbook. This section is a placeholder; the full coordinator runbook will be published as a separate document.

Key topics to cover in the full runbook:

- **Network genesis procedure** (one-time, documented in Section 11.1.1).
- **Membership application review workflow**: BAA verification, UDAP certificate validation, application approval and registration.
- **Revocation procedures**: criteria for revocation, governance approval requirements, technical execution.
- **Coordinator key management**: HSM/KMS configuration, key rotation schedule, emergency key generation and standby procedures.
- **Coordinator service operations**: deployment, monitoring, backup, disaster recovery for the Registry service itself.
- **Governance interface**: how the coordinator interacts with the network's governance body for policy decisions, dispute resolution, and coordinator succession.
- **Audit and reporting**: regular reports the coordinator produces for the governance body and participating institutions.
- **Incident response**: procedures for coordinator key compromise (Section 11.3.4), governance disputes, and other coordinator-specific incidents.

The coordinator role is critical to network trust; the runbook should be developed before the network's first production deployment and reviewed annually by the governance body.

12. Migration and Adoption Path

The previous sections answer how Creda works. This section answers how it becomes real — how a network of vetted, peer-to-peer institutional participants grows from zero to nationally significant scale, what existing alternatives the architecture must demonstrate value against, and what the realistic timeline looks like.

The audience for this section is broader than the engineering team. HIE leadership, standards body participants, executive sponsors, and institutions evaluating adoption all have legitimate questions about strategy, sequencing, and risk. The technical specification is the foundation; this section translates the foundation into a path forward.

12.1 Adoption Philosophy

12.1.1 Bottom-Up, Not Top-Down

Creda spreads through voluntary institutional adoption, not regulatory mandate. Each institution that joins improves the network for everyone, but no institution is forced to participate. This is structurally similar to how DirectTrust adoption progressed in the early 2010s: a vetted trust framework that institutions joined when the value proposition was clear, growing organically until participation became the de facto standard for clinical messaging.

The alternative — top-down adoption driven by federal mandate — has been tried repeatedly for patient identity (most prominently the perpetually-debated National Patient Identifier) and has consistently failed for political and constituent-pressure reasons. A bottom-up approach sidesteps this by not requiring federal intervention to begin functioning.

12.1.2 Additive, Not Replacing

Institutions do not rip out their existing MPI to adopt Creda. They add Creda alongside their MPI as a second source of identity provenance for cross-institutional cases. The MPI remains authoritative for the institution's own patient population; Creda becomes authoritative for cross-institutional identity resolution. Over time, as Creda accumulates richer provenance and broader coverage, it may subsume more of the MPI's function — but this is a multi-year evolution, not a forklift upgrade.

This staging matters operationally. A health system CIO is not going to schedule a downtime window to replace their MPI with Creda. They will, however, deploy a Creda peer alongside the MPI if the integration is straightforward, the marginal value is clear, and the operational risk is bounded. The architecture is designed to support this co-existence indefinitely.

12.1.3 Three Adoption Tiers

Institutions can engage at different levels:

- **Observer**: consumes Creda data through a QHIN that participates in the network. No direct integration. Benefits from improved match rates and provenance access via existing QHIN-mediated workflows. Most institutions will start here.
- **Light participant**: institution runs a Creda peer, reads from the network, and makes limited writes — typically Attest events on existing identity matches. Useful when an institution wants direct provenance access without taking on the operational complexity of being a primary identity writer.
- **Full participant**: institution is a primary writer for its own patients, creating Assert events at registration, Link events when matches are confirmed, and the full range of identity events through clinical workflows. This tier provides the most value to the network and the most direct value to the institution.

Different tiers allow institutions to start small and grow into deeper participation. An institution can begin as an Observer through their QHIN, become a Light participant within a year as integration capacity allows, and graduate to Full participant over multi-year integration projects. The network's value scales as participants graduate to higher tiers.

12.1.4 Network Effects Compound

Each institution that joins increases the value for all participants — more patients with provenance chains, more attestations, more matching candidates. A network of three institutions provides limited value compared to one of three hundred. Adoption strategy must address this chicken-and-egg problem directly: how does the network reach critical mass when early participants face the worst value proposition?

The answer in this design is **leverage QHINs as multipliers**. A QHIN that becomes a Creda peer instantly contributes its full participant network as Observer-tier members. A single QHIN integration can move the network from "interesting pilot" to "covers half the country" in one onboarding cycle. This is why the Phase 2 strategy concentrates on QHIN recruitment rather than direct institution-by-institution adoption.

12.2 Phase 1: Foundation

The first year focuses on building the foundation: reference implementations, founding institutional commitment, and standards body engagement.

12.2.1 Reference Deployment with Founding Institutions

The network launches with 3-5 founding institutions:

- **An HIE serving as the legal coordinator** — bringing the governance role and the operational discipline to run the Participant Registry service.
- **Two large health systems as primary writers** — providing realistic data volume, integration challenges, and the institutional credibility needed to recruit later participants.
- **A payer as an early attestation source** — adding non-provider perspective and demonstrating the multi-stakeholder value proposition.

Founding institutions invest more (debugging time, feedback cycles, governance contribution, willingness to operate against rough edges) and gain reputation as Creda pioneers. Their commitment is encoded in their NPA terms — they accept higher operational risk in exchange for governance influence and early adopter recognition.

12.2.2 Open-Source Reference Implementations

The Creda FHIR IG, Creda Core source code, and HAPI Bridge source code are published under an OSI-approved license (Apache 2.0 recommended) from day one. Open source is not optional for this project — it is a precondition for:

- **Security review confidence**. Institutions cannot adopt closed-source identity infrastructure for critical workflows. Public review by security researchers is a prerequisite for institutional trust.
- **Standards body acceptance**. HL7, ONC, and Sequoia are unlikely to recognize a vendor-controlled IG. Open governance models are a baseline requirement.
- **External contribution**. The network benefits from contributions beyond the founding team — bug reports, integration tooling, language bindings, deployment patterns. Closed source forecloses this.
- **Avoiding vendor lock-in concerns**. Institutions adopting Creda need confidence that they can operate the system independently of the original developers if necessary.

The license choice is Apache 2.0 (permissive, patent grant, broadly compatible) over GPL variants (which create distribution complications with HAPI FHIR, which is Apache 2.0).

12.2.3 HL7 FHIR Engagement

The Creda IG is published through HL7's standard process. The path:

1. **Initial draft** published as a public IG on `http://creda.health/fhir/ig/v1` with full source.
2. **HL7 FHIR Foundation IG submission** — the first level of HL7 recognition, providing standards namespace and balloting infrastructure.
3. **Comment ballot** — formal HL7 review cycle.
4. **STU (Standard for Trial Use)** — HL7's recognition tier for IGs in production use but not yet final.
5. **Normative** — final standards status, achieved after multiple years of trial use and balloting.

Engagement should begin during the IG's draft phase, not after publication. Active participation in the HL7 Patient Administration work group, the Security work group, and the FHIR Infrastructure work group ensures Creda is informed by ongoing FHIR evolution and that FHIR contributors see Creda as a collaborator rather than a competitor.

12.2.4 Sequoia and TEFCA Alignment

Creda's tokenization aligns with or extends TEFCA Individual Access Services tokens (Section 9.2.5). The Sequoia Project — which administers TEFCA's Common Agreement — is the appropriate engagement venue for tokenization compatibility, QHIN integration patterns, and eventual TEFCA recognition.

Engagement during Phase 1 should focus on:

- Ensuring Creda's tokenization extensions are compatible with TEFCA's reference implementation.
- Identifying any TEFCA Common Agreement provisions that interact with Creda's consent model.
- Building relationships with Sequoia's technical workgroup leadership for Phase 2 QHIN recruitment.

12.3 Phase 2: First QHIN Integration

The single highest-leverage adoption moment is when a QHIN becomes a Creda peer.

12.3.1 Why QHINs Are the Multiplier

Every covered entity in a QHIN's network instantly benefits from Creda's improved matching and provenance, without those institutions having to do anything beyond their existing QHIN integration. A single QHIN onboarding can scale the network's effective coverage by orders of magnitude.

The QHIN does not need to deploy Creda peers at every participant — only the QHIN itself runs a Creda peer, integrated with its existing identity resolution infrastructure. Participants continue using standard QHIN APIs; the QHIN's matching backend now consults Creda's provenance graph alongside its existing logic.

12.3.2 QHIN Recruitment Strategy

Identify QHINs facing matching pain:

- Smaller or regional QHINs are often more agile than the large ones and feel match rate problems more acutely (less data to work with, less existing infrastructure to defend).
- Specialty-focused QHINs (e.g., behavioral health networks under TEFCA) face particularly hard matching cases that Creda's provenance approach addresses well.
- Newer QHINs without legacy MPI investments may find Creda integration easier than mature QHINs with deeply-embedded existing systems.

Pilot proposal structure:

1. **Baseline measurement**: measure the QHIN's current match rate against a representative test cohort (synthetic patients or anonymized real cases).
2. **Pilot deployment**: integrate Creda alongside the QHIN's existing MPI for a defined cohort or duration.
3. **Comparative measurement**: measure match rate against the same cohort with Creda available.
4. **Decision point**: if Creda demonstrably improves match rate (target: 5+ percentage points improvement on cross-institutional matches), the QHIN converts the pilot to production. If not, the pilot ends without commitment.

This structure is honest about the value proposition. Creda either delivers measurable improvement or it doesn't. Pilots that fail are useful — they reveal what needs to improve before broader adoption.

12.3.3 Regional HIE Pilot (Parallel Track)

Parallel to QHIN recruitment, a regional HIE pilot serves a different purpose: producing operational data for capacity planning, exposing edge cases at a manageable scale, and demonstrating that the architecture works as designed in production conditions.

A regional HIE has:

- Tighter participant scope (dozens to hundreds, not thousands).
- More cohesive governance (often a single state or multi-state board).
- Existing operational discipline for cross-institutional workflows.

The HIE can serve as the legal coordinator for its regional Creda deployment. Regional pilots can later federate into the national network through coordinator transition (Section 10.5.4), or remain regionally scoped indefinitely if the participants prefer.

12.3.4 Pilot Success Metrics

Pilots succeed or fail on numbers, not narrative. Required metrics:

- **Match rate improvement**: percentage point improvement over the baseline MPI for cross-institutional matches. Target: 5+ percentage points for production pilots.
- **Provenance coverage**: percentage of identities in the pilot scope with multi-institutional provenance chains. Target: 30%+ within 6 months of pilot launch.
- **Disambiguation success**: percentage of ambiguous matches (those with low MPI confidence) resolved by `$creda-disambiguate` to high-confidence outcomes. Target: 60%+ resolution rate.
- **Replication health**: p99 replication lag, anti-entropy backlog, peer availability. Target: p99 replication lag under 5 seconds during pilot conditions.
- **Incident rate**: privacy and security events per million transactions. Target: zero unexplained incidents during pilot.

Pilot agreements specify these metrics and target thresholds upfront. Missing thresholds is not failure — it is data — but conversion from pilot to production requires meeting them.

12.4 Phase 3: Network Growth

Phase 3 is where adoption transitions from "carefully orchestrated pilots" to "organic growth." The strategy shifts from recruitment to enablement.

12.4.1 QHIN-to-QHIN Adoption

Once one QHIN demonstrates value through a successful pilot, other QHINs face competitive pressure. The argument shifts from "should we adopt this experimental thing?" to "are we falling behind by not adopting it?" Each new QHIN brings its full participant network with it.

The ideal cadence: 2-3 additional QHIN onboardings in Year 2 of the network's existence, accelerating to most major QHINs by Year 4-5. This is aggressive but achievable if the first pilot demonstrates clear value and reference implementations are mature.

12.4.2 EHR Vendor Integration

Major EHR vendors integrating Creda support into their patient registration and identity management workflows is a multi-year arc but high-leverage. Once Epic, Oracle Health (Cerner), Athenahealth, or other major vendors offer Creda integration as a built-in capability, individual institutions get Creda by upgrading their EHR — they do not deploy additional infrastructure.

Vendor integration motion:

- **Reference integration with one vendor first** — likely a smaller or more agile vendor where decision cycles are shorter.
- **Demonstrable customer demand** — institutions referencing Creda in their RFPs and contract renewals.
- **Standards body recognition** — HL7 IG status, TEFCA endorsement — provides political cover for vendor integration decisions.
- **Open-source SDK and reference adapters** — making vendor integration low-cost in engineering effort.

Major vendor integrations are 3-5 year arcs. They are unlikely to drive Phase 2 success but are essential for Phase 4 maturity.

12.4.3 Specialty Network Adoption

Some specialties have particularly acute identity problems:

- **Behavioral health**: stricter consent rules (42 CFR Part 2), patients move between providers frequently, cross-institutional context is critical for safety.
- **Addiction treatment**: similar consent complexity, with the additional challenge that patients often present at multiple facilities under stress conditions where identity verification is hard.
- **Pediatrics**: linking to maternal records, custody/guardianship complications, pediatric-to-adult provider transitions that span identity changes.
- **Rare disease care**: small patient populations, multi-institutional consultations are routine, identity errors have outsized clinical impact.

Specialty networks may adopt Creda earlier than general medical networks because the pain is more acute and the cost of identity errors is higher. Specialty-focused pilots in Phase 3 can produce strong value demonstrations and serve as advocates for broader adoption.

12.5 Phase 4: Patient-Side Adoption

Patient signing keys (Section 9.1.6) become useful only when patients can actually use them. Phase 4 — beginning around Year 3-4 of the network's existence — focuses on patient-side infrastructure.

12.5.1 Patient Keys via Existing Patient-Facing Infrastructure

Patients do not install a "Creda app." They get a Creda key as part of their normal patient portal sign-up. Strategy:

- **Integrate patient key generation into patient portals** (MyChart, FollowMyHealth, athenaPatient, etc.) so that patients receive a Creda-compatible key when they create their portal account.
- **Use TEFCA IAS infrastructure** as the standardization point. Patient keys issued through IAS workflows benefit from TEFCA's existing patient identity verification standards.
- **Anchor on WebAuthn / passkeys**, which are increasingly device-native (Apple, Google, Microsoft all support passkeys as of 2024-2025). A passkey on the patient's phone can serve as their Creda key, with no separate hardware or app required.

Patient key adoption follows portal adoption. Patient portal coverage is already 60%+ of insured patients in the US; Creda key coverage trails this and grows as portals integrate the feature.

12.5.2 `$creda-self-verify` Rollout

Even after patient keys exist, institutional support for `$creda-self-verify` rolls out incrementally. The likely sequence:

- **Year 3-4**: Self-verify supported at a small number of forward-leaning institutions, primarily in patient-initiated identity correction workflows (a patient finds an error in their record and requests correction directly).
- **Year 4-6**: Self-verify supported at major academic medical centers and large health systems for routine registration verification.
- **Year 6+**: Self-verify becomes the preferred path for ambulatory registration; registrar-mediated `$creda-disambiguate` remains the fallback for ED/inpatient/non-portal-using populations.

Registrar-mediated disambiguation will remain the primary path for years. Patient-direct verification is the long-arc destination, not the immediate state.

12.6 Phase 5: Maturity

12.6.1 Network-Wide PQC Migration

Once PQC adoption is sufficient — measured by % of new events signed with PQC algorithms (target: 80%+ over a sustained period) — the network can deprecate classical-only signatures via a coordinated cutoff date. This is a years-long process aligned with industry-wide PQC adoption schedules and NIST's eventual recommendations on classical signature deprecation.

The architecture supports this migration without breaking changes:

- Hybrid signatures (`Ed25519MlDsa65`) provide an early transition path.
- Pure PQC signatures (`MlDsa65`) become the default once libraries and HSMs broadly support FIPS 204.
- The deprecation cutoff is published as a future-dated network-level policy update; institutions have years to migrate.

This is the longest-arc item in Creda's roadmap — likely 10+ years from network launch to full classical-signature retirement.

12.6.2 Becoming the Default

The success criterion for Creda is not "adoption rate" but "assumed substrate." At network maturity, Creda is the assumed substrate for cross-institutional patient identity, with traditional MPIs serving local-only roles. Institutions plan new health IT projects on the assumption that Creda exists, the way they currently assume FHIR exists.

This may take a decade. The migration is not done until:

- TEFCA Common Agreement references Creda directly (not just compatible tokenization).
- Major EHR vendors ship Creda as a default integrated capability.
- Patient-facing apps ship Creda key support as an expected feature.
- Regulatory standards (ONC certification, CMS reporting) reference Creda's provenance model.

These are years away. The architecture is designed to remain stable across the transition, so that institutions adopting in Year 2 are not stranded when the network reaches Year 10.

12.7 Critique of Existing Alternatives

Stakeholders evaluating Creda will compare it against existing alternatives. Honest engagement with those alternatives — what they offer, where they fall short for the cross-institutional decentralized identity problem — is essential to making the case for Creda. This section addresses the most commonly proposed alternatives directly.

12.7.1 National Patient Identifier

The National Patient Identifier (NPI for patients, not to be confused with the provider NPI) has been proposed and rejected repeatedly since HIPAA's 1996 enactment, which Congress blocked from funding. The premise: every patient gets a unique federal identifier, and identity matching becomes trivial.

What it offers: theoretical simplicity. If every patient had a single federal identifier and presented it consistently, matching becomes a primary key lookup.

Where it falls short:

- **Politically infeasible.** Three decades of failed attempts. The consensus that patients should not be assigned permanent federal numbers tied to their healthcare records is durable.
- **Doesn't solve the actual problem.** Even if every patient had a federal identifier, they would not always present it correctly. Misspellings, transposed digits, fraudulent presentation, and unavailable identifiers (unconscious patients, pediatric cases, undocumented patients) would still require demographic-based matching.
- **Creates a single point of failure.** A federal patient identifier registry would be the highest-value target for state-level adversaries and insider threats.

Creda's approach: identity provenance does not require a single identifier. It requires a way to recognize that two assertions refer to the same person, with cryptographic accountability. The network functions without any federal registry.

12.7.2 Centralized Federal MPI

A variant of the NPI proposal: a central federal MPI that institutions query, where a federal authority (or contracted entity) operates the matching service. Some TEFCA-related proposals have included elements of this.

What it offers: top-down standardization, single source of truth, federal authority backing.

Where it falls short:

- **Same political feasibility issues** as the NPI. A centralized federal identity service for healthcare faces persistent constitutional and constituent-pressure concerns.
- **Reintroduces the architectural problems** that decentralization was designed to avoid: single point of failure, single point of trust, single point of compromise.
- **Concentrates patient data.** A central MPI must hold demographic data for matching; this data becomes a high-value target.
- **Slow to evolve.** Centrally-administered systems improve at the speed of the central administrator's roadmap, not at the speed of participant innovation.

Creda's approach: vetted decentralization. Admission control via the legal coordinator (Section 10.5) provides the trust framework benefits of a centralized model; peer-to-peer operations provide the resilience and innovation benefits of a decentralized one.

12.7.3 Blockchain-Based Patient Identity

Various proposals have been made to put patient identity on a public blockchain (Ethereum, Bitcoin) or on a permissioned blockchain platform (Hyperledger Fabric, R3 Corda). Some of these proposals share genuine architectural insights with Creda — Merkle DAGs, signed assertions, distributed trust.

What blockchain approaches offer:

- Tamper-evidence (which Creda also provides).
- Decentralization (which Creda also provides).
- Cryptographic accountability (which Creda also provides).

Where they fall short for healthcare identity specifically:

- **Public blockchains are wrong for PHI.** Even tokenized PHI on a public blockchain is inappropriate — the immutability that blockchains tout becomes a liability when right-to-be-forgotten requirements apply. Tombstoning conflicts with public blockchain's foundational immutability.
- **Permissioned blockchains add overhead without proportional value.** Hyperledger Fabric et al. provide Byzantine fault tolerance via consensus protocols (PBFT, Raft) — but Creda doesn't need consensus, only eventual consistency. The consensus overhead of permissioned blockchains is wasted work for Creda's use case.
- **Cryptocurrency-adjacent design baggage.** Many blockchain frameworks bring cryptocurrency-related concepts (gas, tokens, mining/validator economics) that have no place in healthcare and complicate operational deployment.
- **Limited FHIR/Health IT integration.** Blockchain approaches typically require greenfield infrastructure and have not integrated with the existing FHIR/UDAP/SMART/TEFCA ecosystem that institutions actually operate.

Creda's approach: take the genuine insights from blockchain (Merkle DAGs, signed assertions, append-forward semantics, content addressing where appropriate) and apply them to the specific shape of healthcare identity, while explicitly avoiding consensus overhead, immutability conflicts with right-to-be-forgotten, and cryptocurrency baggage. The result is "blockchain-inspired" but not blockchain.

12.7.4 Improved Existing MPIs

Vendors like Verato, NextGate, and Health Catalyst offer increasingly sophisticated MPI products with referential matching, machine learning, and SaaS deployment. These products are real, deployed, and measurably better than legacy MPIs.

What improved MPIs offer:

- Better matching algorithms than legacy MPIs (genuinely meaningful improvements).
- Operational simplicity for individual institutions (SaaS deployment, vendor support).
- Integration with existing FHIR and Health IT workflows.

Where they fall short:

- **Each is a silo.** An institution running Verato has better matching internally, but cross-institutional matching with a competitor running NextGate still requires coordination through QHINs or HIEs. The fragmentation problem is unchanged.
- **Vendor lock-in.** Switching MPIs is expensive; institutions are constrained by their MPI vendor's roadmap.
- **No provenance.** Even the best MPIs return match scores without provenance. There is no way to inspect why a match was made or to dispute it through evidence-based contestation.
- **Vendor concentration risk.** A small number of MPI vendors hold a disproportionate share of identity matching for US healthcare. Compromise of any one vendor is a national-scale incident.

Creda's approach: Creda is not a competitor to improved MPIs at the institutional level. Institutions can continue running Verato or NextGate for their internal MPI needs. Creda complements them at the cross-institutional layer — providing the provenance, decentralization, and disagreement-tolerance that improved MPIs lack. In the long arc, MPIs may become local-only systems with Creda providing the cross-institutional substrate; in the medium term, they coexist.

12.7.5 Vendor-Controlled Identity (Epic Care Everywhere etc.)

The dominant EHR vendor — Epic — provides cross-institutional identity through Care Everywhere, leveraging Epic's market share to create a de facto identity layer for the institutions running Epic.

What vendor-controlled identity offers:

- Working, deployed, real cross-institutional matching for Epic-using institutions.
- Strong vendor-side investment and continuous improvement.

Where it falls short:

- **Excludes non-Epic institutions.** Care Everywhere covers Epic-to-Epic flows. Institutions on other EHRs are second-class participants or excluded entirely.
- **Vendor concentrates trust.** Patient identity for a large fraction of US healthcare flows through Epic's infrastructure. This is a private, for-profit company holding identity authority for a public-good function.
- **No patient agency.** Patients cannot inspect, dispute, or participate in their identity provenance through Care Everywhere.
- **Not interoperable beyond Epic's ecosystem.** Care Everywhere is fundamentally an Epic-internal capability, not an industry standard.

Creda's approach: Creda is vendor-neutral by design. Open-source reference implementations, FHIR-based interfaces, and bottom-up adoption mean that Creda works the same regardless of which EHR an institution runs. Epic, Oracle Health, athenahealth, and smaller vendors can all integrate Creda — and once they do, their institutional customers benefit from a common substrate rather than vendor-fragmented islands.

12.7.6 The Honest Comparison

The above alternatives are not strawmen. Each represents real work by serious people addressing real problems. The case for Creda is not that these alternatives are bad — many of them are genuinely good — but that they each address only part of the problem space:

- NPI/centralized federal MPI: politically infeasible and architecturally fragile.
- Blockchain: right insights, wrong overhead, conflicts with regulatory requirements.
- Improved MPIs: solve institutional matching but not cross-institutional fragmentation.
- Vendor-controlled identity: works for one ecosystem but excludes the rest.

Creda occupies the niche that none of these address: a vendor-neutral, decentralized, FHIR-aligned, regulation-compatible substrate for cross-institutional patient identity provenance. It is complementary to most existing approaches (institutions can keep their MPIs) and explicitly competes only with the centralized federal alternatives that have failed politically for three decades.

12.8 International Adaptation

Creda is **US-first by design**. Several architectural decisions are tightly coupled to US healthcare infrastructure:

- **UDAP authentication** is a US Health IT trust framework. Other jurisdictions have different institutional PKI standards (e.g., the EU's eIDAS framework, the UK's Spine framework).
- **TEFCA / QHIN integration** is US-specific. Other jurisdictions have their own cross-institutional exchange frameworks.
- **HIPAA-aligned consent posture** (treatment-presumed-consent under TPO) reflects US regulation. GDPR-aligned jurisdictions require deny-by-default consent.
- **42 CFR Part 2, ADA, and other US regulatory specifics** influence the data minimization and consent scope decisions.

Adaptation paths to other jurisdictions are deliberately preserved:

- **Pluggable authentication.** UDAP is wrapped behind an `AuthenticationProvider` trait; alternative implementations (eIDAS, Spine, etc.) can be plugged in without changing other components.
- **Configurable consent posture.** Each institution configures its own posture; jurisdictions with stricter regimes default their institutions to deny-by-default.
- **Localized normalization.** Address normalization, name handling, and tokenization rules can be parameterized per jurisdiction. The libpostal-based address normalization handles international addresses; name handling rules can be extended for non-Latin scripts and naming conventions.
- **Independent legal coordinators per jurisdiction.** Each jurisdiction operates its own legal coordinator and Participant Registry, federated through cross-coordinator agreements. International identity exchange between jurisdictions follows the same NPA model that intra-jurisdictional adoption uses.

International expansion is not a Phase 1-5 priority. The US deployment must reach Phase 3 maturity before international adaptation becomes practical. When it does, the architectural hooks exist; the work is in standards adaptation, governance translation, and regulatory navigation specific to each jurisdiction.

12.9 Realistic Timeline

A summary of the phases and expected duration:

Phase	Description	Duration	Milestone
Phase 1	Foundation: reference implementations, founding institutions, standards engagement	Year 1	First production deployment between founding institutions
Phase 2	First QHIN integration, regional HIE pilot	Year 2	First QHIN converts pilot to production
Phase 3	Network growth via QHIN-to-QHIN adoption, EHR vendor integrations begin, specialty network adoption	Years 2-5	Coverage of >50% of US insured population through QHIN participation
Phase 4	Patient-side adoption: keys integrated into patient portals, self-verify rollout	Years 3-7	Patient key coverage reaches plurality of insured patients
Phase 5	Maturity: PQC migration, "default substrate" status	Years 7-12+	Creda referenced in TEFCA Common Agreement, major EHR vendor default integration

These are honest projections, not aspirations. The 12-year horizon for full maturity reflects the historical pace of US Health IT infrastructure transitions (FHIR's adoption from publication to ubiquity took ~10 years). Earlier phases can move faster if reference implementations are strong and pilots produce clear value; later phases depend on multi-institutional and multi-vendor coordination that does not accelerate easily.

13. Open Questions

This section enumerates design decisions and operational details that remain unresolved as of this spec version. These are the genuine gaps — places where further prototyping, real-world data, governance input, or ecosystem evolution is required before answers can be locked in.

The purpose of this section is honesty. A specification that pretends every detail is settled invites failure when the unsettled details surface during implementation. By naming what is open, the engineering team and reviewers can prioritize the work needed to close each question, and stakeholders can see the spec's actual maturity.

Each question includes the relevant section, a description of what is unresolved, and the conditions under which it will be closed.

13.1 Storage and DAG Layer

13.1.1 libgit2 vs. RocksDB as Storage Foundation

Reference: Appendix C.7

The question: Should Creda Core's DAG and storage layer be built on libgit2 (using Git's data model directly) or on RocksDB with a custom Merkle-DAG implementation?

Why it's open: libgit2 offers significant code reduction and inherits decades of Git hardening, but it requires reconciling UUID-based addressing with Git's content-addressing and adapting Git's repository organization to handle millions of patient subgraphs. RocksDB offers more flexibility but requires building DAG primitives ourselves. Until parallel prototypes are built and compared on lines of code, performance, and operational characteristics, both options remain live.

Closure condition: Build prototype implementations of Creda Core on both backends with equivalent functionality (event creation, retrieval, subgraph traversal, signature verification, anti-entropy). Compare on: total lines of Creda-specific code, throughput for representative workloads, recovery time after PV loss, debugging tooling availability. Decide before locking in production architecture. Estimated effort: 2-4 engineer-weeks per prototype.

13.1.2 Tombstone Integrity Tradeoff

Reference: Section 7.2.2

The question: Tombstoned nodes lose content integrity (the original signature no longer verifies because the signed content has been scrubbed). The graph topology and references remain intact, but cryptographic verification of "this node originally contained X" is no longer possible after tombstoning. Is this acceptable to compliance reviewers, security architects, and auditors?

Why it's open: This tradeoff is the right answer for our regulatory constraints — right-to-be-forgotten requires actual content destruction — but it leaves a gap in the cryptographic audit story. An auditor cannot retroactively verify what a tombstoned event originally said. We have not yet had this design reviewed by privacy counsel, healthcare auditors, or institutional security architects.

Closure condition: Review the design with: (a) privacy counsel familiar with HIPAA, GDPR, and state-law right-to-be-forgotten requirements; (b) representative institutional security architects from founding institutions; (c) HL7 Security work group reviewers during the IG ballot process. Document any required adjustments — for example, an option to retain a hash of the original content (without the content itself) for "what was tombstoned" audit purposes, while still satisfying the legal requirement that the PHI itself is destroyed.

13.1.3 Bucket Count for Topic Gossip

Reference: Section 6.2.4

The question: Creda specifies 1,024 topic buckets for gossipsub subscription, hashing patient subgraphs into buckets to limit topic cardinality. The right number depends on traffic patterns we don't have yet — too few and per-bucket traffic is excessive; too many and gossipsub overhead dominates.

Why it's open: Optimal bucket count is a function of patient population per institution, write rate per patient, and desired ratio of "events received but not relevant to me" overhead. Without real traffic data, 1,024 is an educated guess.

Closure condition: Run load tests on pilot deployments measuring per-bucket bandwidth, gossipsub mesh overhead at varying bucket counts, and the tradeoff curve. Adjust default bucket count based on results. The bucket count is a tunable parameter so production deployments can be reconfigured if needed, but the protocol-level default should be informed by pilot data before broader adoption.

13.2 Identity and Matching

13.2.1 TEFCA Tokenization Alignment Specifics

Reference: Section 9.2.5

The question: Section 9.2.5 commits to "align with or extend TEFCA IAS tokenization." The actual technical alignment requires reviewing TEFCA's reference implementation in detail and identifying every divergence in normalization rules, salt management, and field tokenization scope.

Why it's open: TEFCA's tokenization specification has been evolving. We have not yet performed a line-by-line comparison between Creda's tokenization rules (Section 9.2.3) and the current TEFCA reference implementation. Until that comparison is done, we cannot claim full TEFCA compatibility, only a stated intent.

Closure condition: Engagement with the Sequoia Project's technical workgroup, formal review of TEFCA's published tokenization specifications, and a documented compatibility matrix listing every alignment and every divergence. Where Creda diverges, the IG documents the bridging logic for translation between formats.

13.2.2 Confidence Scoring Calibration

Reference: Section 5.3

The question: The confidence scoring model specifies inputs (verification method weights, institutional credibility weights, attestation amplification, agreement amplification, temporal decay) but the actual numerical weights and curves are unspecified. What weight does a government-ID-verified assertion carry vs. a self-reported one? How much does the tenth attestation amplify confidence vs. the third?

Why it's open: Calibration requires real-world data. The right weights are the ones that produce match rates that align with manual clinical review of borderline cases. Without pilot data, any weights we set now are guesses.

Closure condition: Pilot deployments collect ground-truth match data (cases reviewed by clinicians or registrars, with their judgments recorded). Calibration runs against this data, tuning weights to minimize false positives and false negatives. The calibrated weights become defaults; institutions can override based on local policy. This is iterative — weights should be re-evaluated annually as data accumulates.

13.2.3 Disambiguation Question Generation Algorithm

Reference: Section 8.2.9.2

The question: Section 8.2.9.2 describes the criteria for question selection (differentiating between candidates, avoiding PHI leakage, accommodating cognitive load) but does not specify the algorithm. Selecting good questions is a research-grade problem — it requires understanding which facts are memorable to patients, which differ reliably between candidates, and how to construct multiple-choice distractors that don't leak other candidates' data.

Why it's open: This is a novel application — no existing system has done provenance-grounded patient disambiguation at the level of granularity Creda requires. Implementation requires prototyping and iteration.

Closure condition: Prototype the question selection algorithm against synthetic test cases. Validate that questions: (a) actually differentiate candidates in the test set, (b) do not leak demographic data through distractor selection, (c) are answerable by patients with realistic memory (validated through user testing with synthetic patient personas). Iterate until the algorithm meets quality thresholds, document the algorithm in a separate design note, and publish reference implementation.

13.2.4 Patient Self-Verify Trust Weight

Reference: Section 5.3.2

The question: Patient-originated assertions (via `$creda-self-verify` or other patient-direct paths) carry "lower default confidence" than institutional ones, but the specific weighting is undetermined. How much should a patient's signed self-attestation count compared to a clinician's institutional Assert backed by a government ID check?

Why it's open: This is a policy question as much as a technical one. Stronger weight on patient assertions empowers patients but creates risk of identity manipulation (a patient with a stolen key could assert false demographics). Weaker weight protects against manipulation but undermines the patient agency model.

Closure condition: Governance body input from founding institutions and patient advocacy representatives. Document the rationale and recommended default weight; allow institutional override. Re-evaluate annually as patient key infrastructure matures.

13.3 Network and Consensus

13.3.1 Anti-Entropy Partner Selection

Reference: Section 6.2.5

The question: Section 6.2.5 specifies tiered anti-entropy scheduling (15 minutes for active subgraphs, 6 hours for warm, 7 days for dormant) but does not specify how peers choose their anti-entropy partners. Random selection? Reputation-weighted? Geographically aware? Selection by overlap in held subgraphs?

Why it's open: Different selection strategies have different convergence and bandwidth characteristics. Random is simple but inefficient. Overlap-aware concentrates work where it matters but requires DHT-coordinated partner discovery. Geographic awareness reduces cross-region traffic but slows cross-region convergence.

Closure condition: Network simulation with representative peer counts and traffic patterns. Compare strategies on convergence time, bandwidth efficiency, and resilience to partial outages. Pick a default strategy with explicit knobs for institutional override.

13.3.2 DHT Replication Factor

Reference: Section 6.1.5

The question: Each patient subgraph announcement is held at how many DHT nodes? Kademlia's default replication factor is k=20, but Creda's traffic patterns may suggest a different value. Higher k provides more resilience to peer churn; lower k reduces network overhead.

Why it's open: The right value depends on peer churn rates (how often peers join and leave) and DHT query rates (how often subgraph lookups happen). We don't have this data yet.

Closure condition: Measure peer churn and DHT query rates in pilot deployments. Adjust replication factor based on observed patterns. The value is a tunable parameter that can be adjusted post-launch without protocol changes.

13.3.3 Cross-Institutional Event Ordering Edge Cases

Reference: Section 7.2.3

The question: When two institutions concurrently create Link events asserting incompatible identity relationships (e.g., one links Patient A → B, the other links Patient A → C, where B and C are clearly distinct people), Section 7.2.3 says these surface as competing assertions for institutions to resolve. But the operational mechanics — who notifies whom about the conflict, how it appears in the FHIR Bridge, how it's audit-logged, what UI the registrar sees — are underspecified.

Why it's open: Concurrent conflicting Links are rare but high-stakes. The operational handling needs careful design with clinical workflow input.

Closure condition: Operational design workshop with clinical informaticists and HIE operators. Document the notification, presentation, and resolution workflows. Implement in the FHIR Bridge with appropriate `$creda-disambiguate` integration for the resolution path.

13.4 Portable Authorization

13.4.1 Revocation Latency Bounds 2 and 3

Reference: Section 4.7

The question: Section 4.7 commits to three revocation propagation latency bounds. Bound 1 (gossip, ~1-2 seconds under normal conditions) follows from the gossip design. Bounds 2 (anti-entropy catch-up) and 3 (post-partition convergence) are stated as architectural commitments but are partition- and load-dependent.

Why it's open: Bounds 2 and 3 cannot be validated without real network conditions. The worst-case post-partition bound is inherently partition-duration-dependent and therefore not a fixed number.

Closure condition: Pilot deployments instrument revocation propagation latency under normal, degraded, and post-partition conditions. Conformance tests verify Bound 1 under normal conditions. Document realistic distributions for Bounds 2 and 3 from pilot data, and establish operational alerting thresholds for revocation lag.

13.4.2 Export Gate Integration Surface

Reference: Section 10.2

The question: The Export Gate must sit at each institution's data egress boundary, but institutions have heterogeneous egress architectures (FHIR endpoints, interface engines, data warehouses, direct EHR integrations). The reference integration patterns are not yet specified for each.

Why it's open: Egress architecture varies widely across institutions. A single integration pattern will not fit all, and the reference implementations for the common cases (HAPI interceptor, interface-engine hook, warehouse pre-export gate) have not been built.

Closure condition: Build and document reference Export Gate integrations for the three or four most common egress patterns, validated with founding institutions. Publish integration guides per pattern.

13.4.3 Verifier Stale-State Policy

Reference: Section 10.3.3

The question: The Verifier can operate offline against stale DAG state and reports the age of its view, but the policy for when stale-state verification is acceptable — and who decides — is left to the relying institution. There is no network-level guidance on acceptable staleness by use type.

Why it's open: Acceptable staleness is use-dependent (a routine read tolerates more staleness than a fresh authorization check before a bulk export) and risk-tolerance-dependent (institutions differ). A universal threshold would be wrong for someone.

Closure condition: Publish recommended staleness thresholds by use type (routine read, sensitive read, pre-export check, AI/research use) as operational guidance, with the relying institution retaining override authority. Pilot data informs the recommendations.

13.5 Security and Cryptography

13.5.1 PQC Algorithm Finalization

Reference: Section 5.1.2

The question: Section 5.1.2 references ML-DSA-65 (FIPS 204) and SLH-DSA-256s (FIPS 205) as PQC algorithm choices. NIST's PQC ecosystem continues to evolve, and final algorithm selection should track NIST's recommendations and HSM/library availability at the time of implementation.

Why it's open: PQC standardization is recent and ongoing. Cryptanalysis of current candidates may reveal weaknesses requiring algorithm changes. HSM and library support for the selected algorithms is uneven.

Closure condition: Continuous monitoring of NIST PQC announcements, cryptanalysis publications, and HSM/library support. Algorithm choices are revisable through the algorithm-agile signature design (Section 5.1.2) — a decision made today does not lock the network into a future-broken algorithm. Re-evaluate annually until PQC adoption stabilizes industry-wide.

13.5.2 Salt Rotation Governance

Reference: Section 9.2.2

The question: The network salt rotates annually, but who decides when, how the next salt is published, and what happens if a salt rotation is contested?

Why it's open: Salt rotation governance is a network-wide decision that requires coordination among participating institutions. The legal coordinator publishes the salt, but the decision authority and contest procedures are not specified.

Closure condition: Governance body procedure document, developed in conjunction with the legal coordinator runbook (Section 11.5). Specify: who proposes the next salt, who approves it, the publication window, the transition window, and what happens if an institution objects to a rotation.

13.5.3 Coordinator Key Compromise Emergency Procedures

Reference: Section 11.3.4

The question: Section 11.3.4 describes the high-level response to coordinator key compromise (governance convenes, new key activated, transition executed) but the actual emergency communication channels — how the governance body distributes the new public key to all participants in hours rather than days — are not specified.

Why it's open: Effective emergency response requires pre-staged channels and procedures. We have not designed these in detail.

Closure condition: Develop the emergency response runbook before network launch. Specify: pre-registered emergency contact channels for each participating institution, the out-of-band signing chain for emergency announcements, the threshold of governance body members required to authorize key transition, and the verification procedures institutions follow before accepting the new key. Run tabletop exercises with founding institutions before production.

13.5.4 Insider Threat Detection Thresholds

Reference: Section 9.1.7

The question: Section 9.1.7 references anomaly signals (registrar invoking disambiguation outside normal patterns, unusual access volumes, etc.) but the specific thresholds, signal weights, and alert routing are institution-decided.

Why it's open: Anomaly detection is institution-specific. A "normal" disambiguation pattern at a large urban academic medical center is different from a small rural clinic. Universal thresholds would generate false positives at one site and miss real signals at another.

Closure condition: Reference implementations and recommended starting configurations published as part of the operational guide, with documentation of how to tune for local patterns. Pilot deployments produce the empirical baselines that institutional security teams can adapt.

13.6 FHIR and Integration

13.6.1 R4 to R5 Migration Timing

Reference: Section 8.2

The question: Section 8.2 commits to FHIR R4 with R5 conformance planned for v1.1. The actual timing depends on US Core's R5 baseline maturity and broad ecosystem adoption.

Why it's open: US Core 7.0+ is moving to R5 but adoption is still mixed across the US Health IT ecosystem. Releasing an R5-only Creda IG too early would alienate institutions still on R4. Releasing too late means the IG lags behind FHIR's evolution.

Closure condition: Track US Core R5 adoption metrics through HL7 and ONC reporting. When R5 reaches majority adoption among US Core implementers (target: 50%+), publish Creda IG v1.1 with R5 conformance, supporting both R4 and R5 during a transition window.

13.6.2 CapabilityStatement Evolution

Reference: Section 8.2.11

The question: As new event types and operations are added (event types are extensible per Section 3.4), the CapabilityStatement must evolve. The mechanics of capability negotiation — how a Creda v1.0 peer interacts with a v1.1 peer that supports new operations — needs more specification.

Why it's open: We have not yet versioned the protocol or the IG. Versioning conventions, backward compatibility guarantees, and capability negotiation patterns need explicit design.

Closure condition: Versioning and compatibility design document, published before v1.1. Cover: semantic versioning rules for the IG, the protocol, and the CapabilityStatement; backward compatibility commitments; deprecation policies; how peers advertise supported versions and how clients handle mixed-version networks.

13.7 Adoption and Operations

13.7.1 First QHIN Target

Reference: Section 12.3

The question: Section 12.3 describes QHIN recruitment strategy in the abstract but does not name a specific first target. Identifying the right first QHIN — one with willingness, capacity, and compatible technical posture — is a business-level question.

Why it's open: This depends on relationships and ongoing conversations, not technical decisions. The right first QHIN may be smaller and more agile rather than larger and more prominent.

Closure condition: Business development outcome, tracked separately from this spec.

13.7.2 Capacity Planning Data

Reference: Section 11

The question: Section 11 deferred capacity planning guidance because we don't have real-world data on what a peer holding 5 million patients with active gossip looks like operationally. The 50 GB persistent volume default and 4 GiB memory baseline are starting points without empirical backing.

Why it's open: No production deployment exists yet to measure.

Closure condition: Pilot deployments instrument detailed capacity metrics. After 6-12 months of pilot operation, publish a capacity planning guide with actual measurements: storage growth per million patients, memory usage at varying gossip subscription counts, CPU usage at peak query rates, network bandwidth profile. Update spec defaults if pilot data warrants.

13.8 Patient and Edge Cases

13.8.1 Patient Key Recovery Flow

Reference: Section 9.1.6

The question: Section 9.1.6 mentions OIDC-mediated re-enrollment for patient key loss but does not specify the security ceremony. What identity proofing is required for re-enrollment? How is the old key revoked? How are events signed by the recovered key linked to events signed by the original key for the same patient?

Why it's open: Patient key recovery is a high-risk workflow — done badly, it becomes an attack vector for identity hijacking. The right ceremony depends on the patient identity proofing infrastructure available through TEFCA IAS, which is itself evolving.

Closure condition: Design and document the recovery ceremony with input from the IAS technical workgroup, FIDO Alliance recommendations on passkey recovery, and security review by founding institutions. Include the recovery flow in the FHIR IG and in operational runbooks.

13.8.2 HIPAA Privacy Rule Edge Cases

Reference: Sections 3.4, 8.3, throughout

The question: Specific scenarios where HIPAA, state law, or clinical practice create complexity:

- Minor patients aging into adulthood (consent transfer, parental access termination).
- Divorce, remarriage, custody changes affecting consent inheritance.
- Deceased patient family access (HIPAA permits limited family disclosure of deceased patient information; Creda's consent model needs to handle this).
- Emancipated minors (jurisdiction-specific rules for adolescent consent).
- 42 CFR Part 2 substance use treatment records (stricter consent than general HIPAA).
- Court-ordered access (e.g., custody disputes, criminal investigations).

Why it's open: Each scenario is its own design problem. Some are addressed by the existing consent model but require explicit configuration; others may require new event types or scope semantics.

Closure condition: Per-scenario design notes developed in conjunction with privacy counsel and clinical informaticists. Some scenarios may extend the IG with new profiles or scope definitions; others may be addressable through institutional consent configuration without protocol changes. Maintain an active list of edge cases as institutions encounter them in practice.

13.8.3 Multi-Region Snapshot Replication

Reference: Section 6.4.2

The question: Section 6.4.2 mentions cross-region deployment but the specifics of snapshot replication for institutions operating multi-region peers are underspecified. Does each region maintain its own snapshot store? Are snapshots replicated cross-region for disaster recovery? Who owns the cross-region replication relationship?

Why it's open: Multi-region deployment is a Phase 3+ concern. We have not designed it in detail because the v1 deployment scope is single-region.

Closure condition: Design document for multi-region deployment, developed when at least one founding institution requires multi-region operation. Cover: per-region snapshot storage, cross-region replication patterns, regional failover procedures, and consistency guarantees for cross-region anti-entropy.

13.9 Tracking and Closure

This list of open questions is **not exhaustive** — additional questions will surface during implementation, pilot deployment, and standards body review. The spec maintainers commit to:

- Adding new open questions as they are identified.
- Closing questions in version-controlled commits with documentation of how each was resolved.
- Reviewing this section at each minor version bump to ensure the list reflects current state.

Each closed question should result in: an update to the relevant spec section with the resolved design, a removal from this list, and a note in the spec's change log explaining the resolution and citing any prototyping, governance, or review work that informed it.

Open questions are not failures. They are the work that remains. Naming them honestly is how the spec earns the trust required for institutional adoption of the resulting system.

Appendix A: Prior Art and References

[To be written — IPFS/libp2p, W3C DID/VC, Git Merkle DAG, CRDT literature, TEFCA/Carequality specs, relevant academic papers]

Appendix B: Glossary

[To be written — Creda-specific terminology definitions]

Appendix C: Build vs. Buy — Existing Components for Each Technical Decision

This appendix annotates every significant technical decision in the spec with the existing library, standard, or service that should be used to implement it. The goal is to minimize code Creda has to write, maintain, and secure. Each entry identifies the spec section, the decision, the recommended existing component, and any adaptation required.

The principle: if a section of the spec describes building something that already exists in a mature, maintained, and appropriately-licensed form, we should use that thing. New code should be reserved for what is genuinely Creda-specific: the healthcare-specific event semantics, the consent model, the disambiguation operations, and the integration glue.

C.1 DAG, Storage, and Cryptographic Primitives

Spec Section	Decision	Use This	Notes
4.1 (Identity Event Node)	Signed DAG with parent references	**libgit2**	Git's data model is a signed DAG with parent references. Use libgit2 as a library (not Git as a server). One repository per institution; patient subgraphs as named refs within the repo. Saves thousands of lines of custom DAG code and inherits 20 years of Git hardening.
4.1.1 (Canonical CBOR)	Deterministic serialization	**`ciborium` crate**	Rust CBOR with canonical encoding mode (RFC 8949 Core Deterministic Encoding). Successor to the older `serde_cbor`. Don't write canonicalization manually.
4.1.2 (PQC)	Algorithm-agile signatures	**`pqcrypto` Rust crate family** + **`ed25519-dalek`**	`pqcrypto-mldsa` for ML-DSA-65 (FIPS 204), `pqcrypto-sphincsplus` for SLH-DSA (FIPS 205), `ed25519-dalek` for classical. Wrap behind a single `CryptoSignature` trait.
4.1.2 (Hash function)	Blake3 with PQC margin	**`blake3` crate**	Official Rust implementation, well-maintained, hardware-accelerated.
4.1.4 (UUIDv7)	Time-ordered UUIDs	**`uuid` crate with `v7` feature**	Don't roll your own. The crate handles the timestamp encoding and randomness correctly.
4.2.5 (Index structures)	Secondary indexes	**RocksDB column families**	If using RocksDB directly, column families provide native secondary index support. If using libgit2, consider supplemental indexes via `redb` (pure Rust, embedded).
4.3.1 (Demographics struct)	Tokenized demographics	**TEFCA IAS tokenization reference implementation**	Don't invent tokenization. Adopt TEFCA's scheme (or extend it where it lacks something Creda needs) so institutions don't run parallel tokenizers.
4.3 (Confidence scoring)	Per-field confidence model	**Fellegi-Sunter probabilistic record linkage**	The math has been settled since 1969. Reference implementations exist in `splink` (Python) and various MPI vendors. Port the algorithm; don't reinvent it.

C.2 Networking and Replication

Spec Section	Decision	Use This	Notes
5.1.5 / 5.2.1 (DHT, gossip, transport)	Peer-to-peer overlay	`rust-libp2p`	Already chosen. Provides gossipsub, Kademlia DHT, Noise transport, NAT traversal, mplex/yamux. The single most important "don't reinvent" decision in the spec.
5.1.4 (Gossip protocol)	Event propagation	**libp2p gossipsub**	Don't implement custom gossip. Gossipsub already handles mesh management, message dedup, fanout control, and peer scoring.
5.1.5 (DHT)	Subgraph routing	**libp2p Kademlia**	Mature implementation with provider records, iterative queries, and routing table maintenance.
5.1.8 (Anti-entropy / Merkle root)	Replica sync protocol	**Git's smart HTTP wire protocol**	If using libgit2, Git's pack-negotiation protocol is exactly the "compare two replicas, transfer the delta" algorithm we described. Run it over libp2p streams instead of HTTP.
5.2.1 (Peer discovery / partial views)	Active/passive view management	**libp2p gossipsub mesh management**	Subsumes HyParView for our needs. Don't implement HyParView separately.
5.2.3 (Transport encryption)	Encrypted authenticated transport	**libp2p Noise transport**	Built into libp2p. Don't roll TLS or custom crypto.
5.4.2 (Cross-region / NAT)	NAT traversal and relay	**libp2p AutoNAT + Circuit Relay v2**	Built-in. Don't implement hole-punching.

C.3 Storage and Operational Layer

Spec Section	Decision	Use This	Notes
6.3.1 (Embedded KV store)	Local persistence	**libgit2 (preferred) or RocksDB**	If we adopt the libgit2-based DAG storage, the KV store decision largely goes away — Git handles storage. If we don't, RocksDB via the `rust-rocksdb` crate.
6.3.3 (Snapshot generation)	Periodic state export	**Git bundles**	If using libgit2: `git bundle` is a single-file export of a repo's history. Already designed for this exact use case (transferring history between clones). Snapshot = bundle, replay = unbundle.
6.4.2 (Workflow orchestration)	Operational task scheduling	**Argo Workflows + k8s CronJobs**	Already chosen.
6.4.3 (Deployment packaging)	k8s deployment artifact	**Helm chart + later a Kubernetes Operator (e.g., via `kube-rs` or Operator SDK)**	Standard. Don't roll deployment automation.
6.4.4 (Object storage)	Snapshot storage	**MinIO (on-prem) / cloud S3-compatible**	Already chosen.
6.4.5 (Observability)	Metrics, traces, logs	**Prometheus + Grafana + OpenTelemetry**	Already chosen. Use the standard OTel SDKs for Rust and Java.
6.4.6 (Identity / certs)	Workload identity, cert rotation	**SPIRE + cert-manager**	Already chosen.
6.5 (Retention tasks)	Scheduled cleanup	**k8s CronJob**	Don't build custom schedulers.

C.4 FHIR Layer

Spec Section	Decision	Use This	Notes
7.3 (HAPI FHIR Bridge)	FHIR server	**HAPI FHIR (Java)**	Already chosen. Plain Server mode with custom resource providers.
7.2.1 (US Core conformance)	US Core profiles	**US Core IG (HL7)**	Don't redefine Patient profiles. Inherit from US Core; layer Creda extensions.
7.2.3 (CredaProvenance profile)	Provenance resource	**FHIR Provenance + US Core Provenance**	Already aligned.
7.2.7 (`$creda-link` , etc.)	FHIR Operations	**HAPI's `@Operation` annotation framework**	Standard HAPI mechanism for custom operations.
7.2.9 (Disambiguation Q&A)	Multi-stage operation flow	**FHIR Parameters + session token pattern**	Idiomatic FHIR.
7.2.11 (CapabilityStatement)	Capability advertisement	**HAPI's auto-generated CapabilityStatement**	Customized for Creda extensions but built on HAPI's generator.
7.2.12 (Subscription)	Real-time notifications	**HAPI's Subscription support** + **FHIR R5 SubscriptionTopic when we move to R5**	Don't write a notification service.
7.2.13 (Bulk Data export)	$export operation	**HAPI's Bulk Data implementation**	NDJSON output, async job tracking — all built in.
7.4.1 (QHIN integration)	TEFCA participation	**Existing QHIN SDK / Sequoia Project tooling**	Whatever Sequoia publishes. Don't fork the QHIN-to-QHIN protocol.

C.5 Security and Identity

Spec Section	Decision	Use This	Notes
8.1.2 (UDAP)	Institutional auth	**HL7 UDAP reference implementations**	UDAP work group has reference clients/servers. Use them rather than implementing UDAP from spec.
8.1.3 (SPIFFE/SPIRE)	Workload identity	**SPIRE +** `rust-spiffe` **crate / Java SPIFFE library**	Standard.
8.1.6 (Patient signing keys)	Patient PKI	**WebAuthn / FIDO2 passkeys + OIDC**	Don't invent patient-side PKI. Browser/OS-level passkey support is mature; pair with OIDC for identity binding.
8.1.8 (SMART on FHIR)	Clinical workflow auth	**HAPI's SMART on FHIR support** + `smart-on-fhir` **JS client for apps**	Existing.
8.1.9 (DDOS)	Rate limiting, mesh management	**libp2p gossipsub peer scoring** + **standard k8s ingress WAF**	Don't write rate limiters.
8.2.1 (Tokenization)	Demographic tokenization	**TEFCA IAS tokens** (via reference implementation)	Reuse, don't reinvent.
8.2.3 (Address normalization)	USPS-standardized addresses	**libpostal** (via `postal-rs` bindings)	Mature address normalization library. Used by OpenStreetMap and others.
8.2.6 (Bloom filters)	Pre-filter optimization	`bloomfilter` **Rust crate** or `probabilistic-collections`	Standard.
4.3 / 9.3 (Authorization)	Authorization representation	**FHIR Consent resource**	Embed FHIR Consent in the AuthorizationGrant payload rather than a parallel schema.
8.4.1 (DAG as audit trail)	Tamper-evident log	**sigstore Rekor pattern**	Rekor is a transparency log used by sigstore for software supply chain. The "signed entries in an append-only Merkle tree, publicly verifiable" pattern is exactly what we need. Borrow the design (and possibly the code, which is open source).
8.4.2 (Read-side AuditEvent)	FHIR-side audit	**HAPI's AuditEvent infrastructure**	Built in.
8.5 (PSI for matching)	Privacy-preserving matching	**Microsoft APSI**, **Google's Private Join and Compute**	Research-grade libraries exist. When this becomes scope, don't implement PSI from papers.
8.5 (ZK proofs)	Zero-knowledge identity claims	`arkworks-rs` **ecosystem** or `bellman`	When in scope, use established Rust ZK libraries.

C.6 What Creda Genuinely Has to Build

After applying the above, what's left for Creda's engineering team to write:

Creda-specific event semantics layer:

- The `IdentityEventType` enum and per-type payload schemas (Section 3.4) — the healthcare-specific event types and their semantics.
- Validation logic for each event type (party-of-the-subgraphs constraint for Contest, signature-by-originating-

institution for Amend, etc.).
- The effective identity computation algorithm (Section 5.2.4) — traversal that respects amendments, contests, and tombstones.

Confidence scoring engine:

- Per-field confidence computation (Section 5.3.2) implementing Fellegi-Sunter math adapted to the per-field and per-attestation model. The math is borrowed; the application to Creda's event model is new.
- Temporal decay (Section 5.3.3) and disagreement flagging (Section 5.3.4) — orchestration over the confidence inputs.

Disambiguation logic:

- Question selection algorithm for `$creda-disambiguate` (Section 8.2.9.2) — choosing differentiating questions from candidate provenance chains.
- Answer scoring against candidate chains.

Consent enforcement:

- The authorization evaluation algorithm (Section 4.6) — the seven-step evaluation over AuthorizationGrant and AuthorizationRevocation events, plus the Export Gate and Verifier enforcement points. The data model uses FHIR Consent; the evaluation and dual-control logic is Creda-specific.

Integration glue:

- The Bridge's translation layer between FHIR resources and DAG events (most of Section 8).
- Translating SMART scopes to Creda operation authorizations.
- Bridging TEFCA IAS tokens and Creda's tokenization where they diverge.

Bootstrapping and registry:

- The Participant Registry as a Creda subgraph — the meta-DAG of who is in the network. The DAG mechanics are libgit2; the participant lifecycle (NPA execution, certificate registration, revocation) is Creda-specific.

Operational integration:

- The k8s Operator (when we build one) — automating snapshot scheduling, certificate rotation, Participant Registry sync.
- The Helm chart and reference deployment configurations.

That's a manageable scope. Roughly: a healthcare-domain event-semantics layer, a confidence/matching engine, a question-selection algorithm, a consent evaluator, FHIR-to-DAG translation glue, and operational tooling. Everything else is assembled from existing parts.

C.7 Reconsidering libgit2 as the DAG Foundation

The single largest "don't reinvent" opportunity is using libgit2 as the storage and replication foundation rather than building on RocksDB directly. The case for it:

Wins:

- Signed DAG with parent references — already the Git data model.
- Content-addressed integrity verification — already Git's design.
- Efficient delta storage and packing — Git's packfiles are highly optimized.
- Garbage collection for orphaned objects — `git gc`.
- Sync protocol — Git's pack-negotiation is exactly our anti-entropy protocol.
- Bundles as snapshots — `git bundle` is our snapshot format.
- Decades of hardening — security review, performance optimization, edge case handling.
- Existing tooling and developer familiarity — anyone who knows Git can reason about Creda's storage.

Tensions to resolve:

- **UUID addressing vs. content addressing.** Creda uses UUIDs for tombstone compatibility. Git uses content hashes. The reconciliation: store events as Git objects (commits or blobs), and maintain a Git ref namespace mapping UUIDs to content hashes. UUID→hash lookup is one ref read. Tombstoning replaces the object content; the ref is updated to point to the new (scrubbed) object. This loses Git's content-addressing integrity for tombstoned objects, exactly as our spec already accepts in Section 7.2.2.
- **Multiple roots per patient.** Git supports multiple roots in a single repository (orphan branches). A patient subgraph with multiple independent roots is a set of orphan branches connected by Link events expressed as merge commits.
- **Repository organization.** One Git repo per patient is impractical at millions of patients. One Git repo per

institution, with each patient's subgraph as a ref namespace (e.g., `refs/creda/patient/[uuid]/heads/main`), works at scale and is how GitHub manages billions of refs across far fewer repositories.

- **Signature model.** Git supports GPG and SSH commit signing. UDAP X.509 certificates are not directly supported. Either extend libgit2's signing interface or sign at the application layer and store the signature as a commit trailer (already a Git convention).

Recommended action: Build a small libgit2-backed prototype of Creda Core in parallel with the v1 implementation. Compare it against the RocksDB-backed approach on:

- Lines of code in Creda Core.
- Performance for representative read/write/sync workloads.
- Operational characteristics (backup, restore, debugging).
- Integration complexity with the rest of the stack.

If the libgit2 approach wins on most axes (which seems likely given the natural fit), promote it to the v1 path. If RocksDB wins, document why and move on. Either way, the comparison is worth the engineering investment because the storage layer is foundational and switching later is expensive.

C.8 Summary

The honest accounting:

- **Networking layer**: 100% existing components (libp2p).
- **FHIR layer**: 100% existing components (HAPI, US Core, FHIR base).
- **Cryptographic primitives**: 100% existing libraries (ciborium, blake3, ed25519-dalek, pqcrypto).
- **Identity and auth**: 100% existing standards and implementations (UDAP, SPIRE, SMART, WebAuthn, OIDC).
- **Tokenization**: 100% existing scheme (TEFCA IAS), possibly extended.
- **Storage and replication**: Very likely libgit2; otherwise RocksDB. Either way, existing.
- **Observability, deployment, operations**: 100% existing tooling (Prometheus, Grafana, OTel, Helm, k8s, Argo, MinIO, cert-manager).
- **Audit trail**: Pattern from sigstore Rekor; resource format from FHIR.

What Creda actually writes from scratch:

- Healthcare-specific event semantics (~200-500 lines per event type).
- Confidence scoring engine adapting Fellegi-Sunter to per-field model (~1,000-2,000 lines).
- Question selection for disambiguation (~500-1,000 lines).
- Consent evaluation algorithm (~500 lines).
- FHIR-to-DAG translation glue in the Bridge (~3,000-5,000 lines).
- Participant Registry mechanics (~1,000 lines).
- Helm chart, Operator, deployment automation (~1,000-3,000 lines).

Rough total: 8,000-15,000 lines of genuinely new code, plus integration with potentially hundreds of thousands of lines of existing libraries. That's the right ratio for a project that wants to be deployable, maintainable, and trusted in a regulated environment.

---

1. DirectTrust, "Direct Secure Messaging," https://directtrust.org/what-we-do/direct-secure-messaging.↩